Decentralized Crypto World Bank

A Blockchain-Based Banking Platform
with AI-Enhanced Security and Assistance

Thesis ID: P26101123

by

Md. Bokhtiar Rahman Juboraz

20301138

Md. Mahir Ahnaf Ahmed

20301083

A pre-thesis 1 report submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science

Department of Computer Science and Engineering
BRAC University
February 2026

It is hereby declared that

1. The report submitted is our own original work while completing degree at BRAC University.
2. The report does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The report does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Md. Bokhtiar Rahman Juboraz	Md. Mahir Ahnaf Ahmed
20301138	20301083

The pre-thesis 1 report of final year project titled "Decentralized Crypto World Bank: A Blockchain-Based Banking Platform with AI-Enhanced Security and Assistance" submitted by

1. Md. Bokhtiar Rahman Juboraz (20301138)
2. Md. Mahir Ahnaf Ahmed (20301083)

Of Spring, 2026 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science on February 20, 2026.

Examining Committee:

Supervisor:
(Member)

Mr. Annajiat Alim Rasel
Senior Lecturer
Department of Computer Science and Engineering
BRAC University

Project Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam
Professor
Department of Computer Science and Engineering
BRAC University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi
Chairperson and Associate Professor
Department of Computer Science and Engineering
BRAC University

This project is designed to be functional on public blockchain testnets, with no intention for monetary profit, nor to create institutional leverage over decentralized financing. No personal banking data, or private financial data, or codes of existing projects, are used in the development of the prototype platform and training AI/ML data. The full project prototype build is to be demonstrated in the final defence of the project; the platform is expected to be ready for testing and troubleshooting by pre-thesis 2. All the development workflow can be traced by the official GitHub repository for the project, and the planning, coding, testing, and usage of different dependencies for this project is considered open-source with MIT licensing, and will be open for contribution after the submission of the final build.
For the training of the AI models, AI-assisted financial decision-making, risk assessment, privacy features, on-chain and off-chain data processing and data storing, we acknowledge the ethical considerations while making decisions for the development of the project.

Abstract

Modern Institutional Banking is established in layers to obtain better control of global financial flows to create intentional restrictions for equal access to financing capabilities and centralized system. To address the complex layered or tier-based architecture that the global market follows for better governance and balance in distribution of financing, and to demonstrate that a decentralized crypto currency system can be established as a more viable solution to global financing, more fair and equal distribution with advanced security system, *Crypto World Bank* project is built with the intention to show a connecting bridge between Decentralized banking system for transparency in traditional institutional layer system for adaptability of Blockchain technology for the global market. The project reveals that it is possible to build a programmable lending platform with better transparency in global financial flow with better security implementation and how Artificial Intelligence can enhance the experience for better adaptability with the growing demand of AI assistance and security. The integration of a combined stack of blockchain technology into the layered-based banking system with AI enhanced security and assistance has not been recorded or attempted before and the purpose of the project is to show that it is possible to make this combined complex system sustainable and functional while making it adaptable with the growing global financial development demands.

Keywords: Blockchain, Decentralized Finance, Deposit Mobilization, Explainable AI, Financial Sustainability, Group Lending, Hierarchical Lending, Institutional DeFi, Multi-Entity Co-Lending, Oracle-Mediated Risk Analytics, Reserve Management, Smart Contracts

Dedicated to our families for their unwavering support throughout our academic journey.

We would like to express our sincere gratitude to our panel members for their guidance and support throughout this project. We also thank our supervisor, Mr. Annajiat Alim Rasel, Senior Lecturer at the Department of Computer Science and Engineering, BRAC University, for his guidance and support. Finally, we thank the Department of Computer Science and Engineering at BRAC University for providing us with the resources and academic environment to pursue this work.

Table of Contents

List of Figures

List of Tables

Utilization (U): $U = \frac{L}{L+B}$

Kinked rate ($U \le U^*$): $r_b(U) = r_0 + \frac{U}{U^*} \cdot r_1$

Kinked rate ($U > U^*$): $r_b(U) = r_0 + r_1 + \frac{U-U^*}{1-U^*} \cdot r_2$

Collateral Ratio: $CR = \frac{C}{D}$

Loan-to-Value: $LTV = \frac{\text{Loan Amount}}{\text{Collateral Value}}$

APR (simple): $APR = r_{\text{period}} \times N$

Compound interest: $A = P\left(1 + \frac{r}{n}\right)^{nt}$

EMI: $EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$

Health Factor: $HF = \frac{C \times LT}{D}$

Group Health Factor: $HF_{\text{group}} = \frac{\text{total\_group\_collateral} \times LT}{\text{total\_group\_outstanding\_debt}}$

Institutional Health Factor: $HF_{\text{inst}} = \frac{\text{on\_chain\_reserve} - \text{min\_reserve}}{\text{outstanding\_borrowing}}$

Reserve Ratio: $RR = \frac{\text{Reserves}}{\text{Total Deposits}}$

Platform Solvency Ratio: $PSR = \frac{\text{TotalReserveBalance}}{\text{TotalOutstandingLoans}}$

Credit Velocity: $CV = \frac{\text{Capital Recycled per Period}}{\text{Total Reserve}}$

Interbank rate cap: $r_{\text{IB}}(U) \le r_{\text{downward}}(U) - \delta$

Upward deposit rate cap: $r_{\text{up}} < r_{\text{down}} - \delta$

FX conversion: $\text{Amount}_B = \text{Amount}_A \times \frac{P_A}{P_B}$

Treasury swap: $\text{Amount}_{\text{to}} = \text{Amount}_{\text{from}} \times \frac{P_{\text{from}}}{P_{\text{to}}} \times (1 - \text{spread})$

Syndicated share: $\text{Share}_i = \frac{C_i}{\sum_j C_j}, \quad \text{Payout}_i = \text{Total} \times \text{Share}_i$

Junior tranche share: $0.10 \le \frac{J}{J+S} \le 0.30$

Net settlement: $\text{Net}_i = \sum_j O_{ij} - \sum_j O_{ji}$

Oracle commit–reveal: $h = \text{keccak256}(s \,\|\, \text{nonce})$

Group loan share: $\text{Share}_i = \frac{\text{LoanTotal}}{N_{\text{members}}}$

On-chain credit score: $CS = \sum_i w_i \cdot \text{feature}_i$

Isolation Forest score: $s(x, n) = 2^{-E(h(x))/c(n)}$

Random Forest fraud probability: $P(\text{fraud} \mid x) = \frac{1}{T}\sum_{t=1}^{T} f_t(x)$

Net interest split: $\text{NetInterest} = \text{InterestCollected} - \text{DepositorYield} - \text{InsuranceAllocation}$

Net present value (NPV): $\text{NPV} = \sum_{t=0}^{3} \frac{B_t - C_t}{(1+r)^t}$

Return on investment (ROI): $\text{ROI} = \frac{\sum_{t=0}^{3} B_t - \sum_{t=0}^{3} C_t}{\sum_{t=0}^{3} C_t} \times 100\%$

SHAP (Shapley value): $\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$

AA: Account Abstraction (ERC-4337)

ALM: Asset-Liability Management

AI: Artificial Intelligence

AI/ML: Artificial Intelligence / Machine Learning

AML: Anti-Money Laundering

APR: Annual Percentage Rate

API: Application Programming Interface

BRAC: Bangladesh Rural Advancement Committee

CAR: Capital Adequacy Ratio (Basel III; see PSR for this platform)

CBDC: Central Bank Digital Currency

CCIP: Cross-Chain Interoperability Protocol (Chainlink)

CCS: ACM Computing Classification System

CR: Collateral Ratio

CVL: Certora Verification Language

DeFi: Decentralized Finance

DID: Decentralized Identifier (W3C)

DON: Decentralised Oracle Network (Chainlink)

EIP: Ethereum Improvement Proposal

EIP-7702: Ethereum Improvement Proposal 7702 — Session Key standard enabling scoped, time-bound wallet authorisations for an autonomous agent

EMI: Equated Monthly Installment

ERC: Ethereum Request for Comments (token / interface standard)

EVM: Ethereum Virtual Machine

FL: Federated Learning

FATF: Financial Action Task Force

FX: Foreign Exchange

GNN: Graph Neural Network

HF: Health Factor

KYC: Know Your Customer

LLM: Large Language Model

L2: Layer 2

LTV: Loan-to-Value Ratio

MFI: Microfinance Institution

MiCA: Markets in Crypto-Assets Regulation (EU)

ML: Machine Learning

MCP: Model Context Protocol — tool-calling interface used by the AI agent to interact with CWB banking operations

NIM: Net Interest Margin

PRISMA: Preferred Reporting Items for Systematic Reviews and Meta-Analyses

PoS: Proof of Stake

PoR: Proof of Reserve (Chainlink on-chain reserve verification)

PSR: Platform Solvency Ratio

QRC: QR code

RBAC: Role-Based Access Control

RLS: Row-Level Security (PostgreSQL policy feature enforcing append-only access on audit tables)

RR: Reserve Ratio

SAR: Suspicious Activity Report

SBT: Soulbound Token (non-transferable ERC standard)

SHAP: SHapley Additive exPlanations

SOFR: Secured Overnight Financing Rate

SSE: Server-Sent Events

SSI: Self-Sovereign Identity

SoK: Systematization of Knowledge

SWC: Smart Contract Weakness Classification

TVL: Total Value Locked

U: Utilization

UUPS: Universal Upgradeable Proxy Standard (ERC-1822)

VC: Verifiable Credential (W3C)

XAI: Explainable Artificial Intelligence

ZKP: Zero-Knowledge Proof

zkAML: Zero-Knowledge Anti-Money-Laundering proof

zkEVM: Zero-Knowledge Ethereum Virtual Machine — a ZK rollup that inherits Ethereum L1 security via cryptographic validity proofs

zk-SNARK: Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

Chapter 1

Introduction

The *Crypto World Bank* is a research-based project for exploring how blockchain technology can be shaped to fit into institutional financial systems with all the added benefits that come with a decentralized financing system. Based on our findings that had similar motivation to our study, existing DeFi protocols show isolated functionalities, such as Aave for lending, Uniswap for exchange, and MakerDAO for stablecoins. This project specifies a tier-based, hierarchically governed platform that includes a mixture of institutional financing with DeFi, accommodating deposit mobilization, credit allocation, interbank liquidity, installment repayment, FX facilitation, solidarity group lending, reserve enforcement, syndicated co-lending, and risk-based governance covering all four tiers that systematically mirrors real-world institutional financing. At the top, Tier 1 represents the World Bank reserve; the core banking hierarchy is specified in Solidity and being improved until final deployment. Another significant feature—the multi-entity contract suite—is fully specified in Chapter 4, which is scheduled for practical development in Phases II–III.

The system is designed for three categories of participant: a programmable World Bank reserve as Tier 1, national and local development banks serving in Tiers 2–3, and retail clients in Tier 4. Retail clients get access to savings, group loans, and installment lending features. This hierarchical structure in this project was motivated by unbanked and MSME financing gaps [9, 13]. The project is not built with any motivation to deploy only in regions with access to advanced technologies; rather, proper distribution of access to finance and fairness is the leading cause of this attempt to find suitable ground to adapt DeFi among the general population. The whole project is open source and available in our GitHub repository. The added benefits of blockchain: it provides state visibility, tamper-evident audit trails, and programmable rule enforcement among institutions having competition, as demonstrated by the renowned World Bank FundsChain programme [25] and JPMorgan Kinexys [26]. From the study of institutions, open ledger infrastructure and institutional hierarchy are not mutually exclusive.

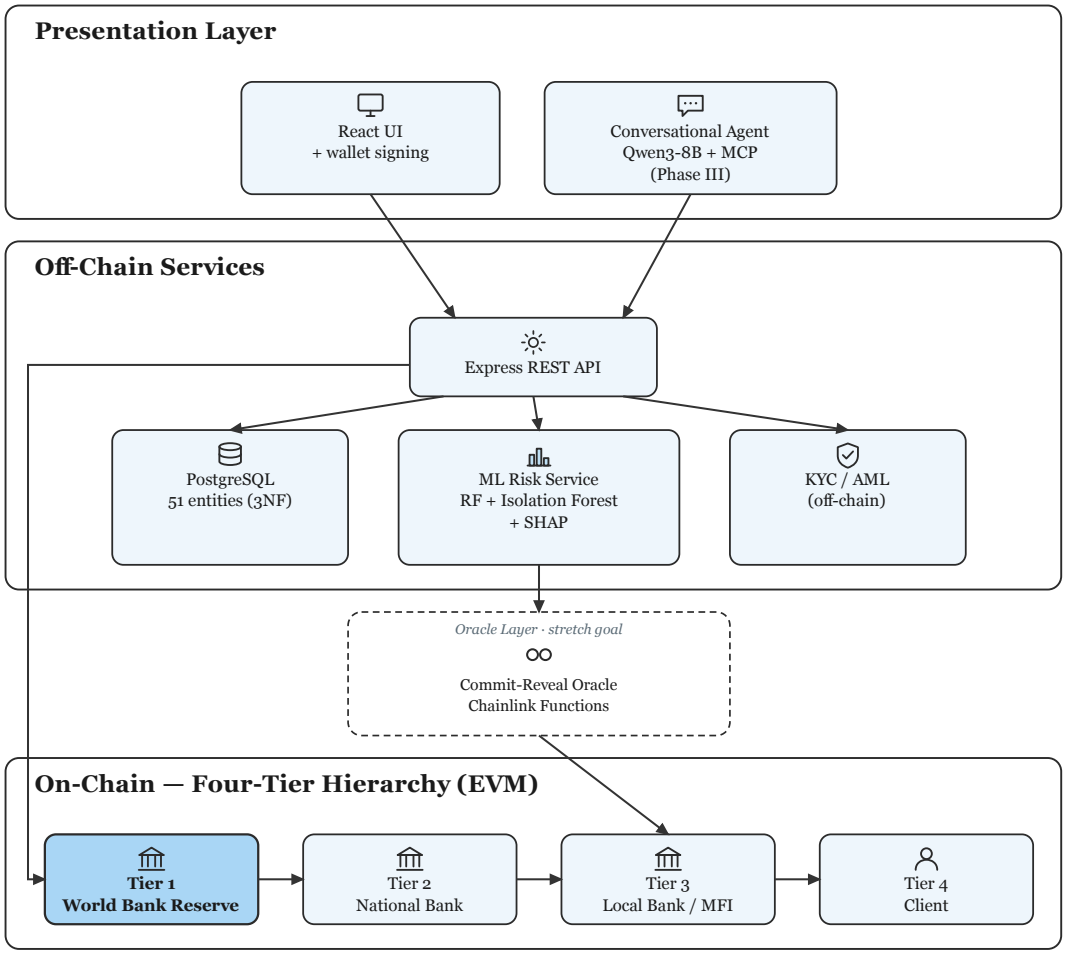

Figure 1.1: System overview.

1.1 Background

Local lenders, or our target users, for this aspect of lending through layered institutions—supranational bodies that is provided by development finance channels have structural costs. Idle capital is usually locked by correspondent banking in pre-funded nostro/vostro accounts [15], cross-border transactions that take around two to five days cost roughly $45 per transaction [101], charges for remittance corridors average 6.49%, which is above the UN SDG target of 3% [17]. Due to unequal access to traditional banking systems, 1.4 billion adults remain unbanked [9] and MSMEs face a $4.5 trillion annual financing gap [13].

In decentralized financing, lending, collateral, and interest can be automated by enabling rules on-chain transparently. However, institutions like Aave v3 that holds $26.3 billion TVL [18] and the broader DeFi lending market exceeds $55 billion [8], use flat pool design that ignore institutional hierarchy, inter-tier exchange, and accessing clients via local bank lend path which is more generalized for normal people with less knowledge about DeFi.

Contribution of smart-contract and blockchain. Smart-contracts can develop lending rules as immutable, atomic, with auditable on-chain logic. This technology replaces the bilateral ledger reconciliation with a system of shared record, when a local bank initiates a loan, the World Bank's reserve ratio updates on the same transaction status [25]. DeFi extends this feature to financial services without licensed custodians. Here in The *Crypto World Bank* adds three dimensional feature that is missing from the existing institutional protocols: institutional hierarchy (four-tier capital flows), multi-entity operations (solidarity groups, syndicated loans, tranched pools), and a connection to the evolving development finance framework identified as absent in the literature [1].

Optional conversational interface. Every user interacts with this system primarily through the React UI and wallet credential signing. For development purposes and demonstration, a locally hosted agent (Qwen3-8B + MCP, 17 banking tools) provides an alternate path to get the essential tasks done, including all the customer services that require system interaction information guidance and assistance in acquiring a loan or exchange finances with the system. In the final build it is planned to be replaced with secured API-based services.

1.2 Rational of the Study or Motivation

Real development finance depends on layered institutions, yet settlement friction, opaque reserves, and correspondent-banking idle capital leave both intermediaries and unbanked clients underserved [17, 90]. Flat DeFi protocols automate lending at scale but ignore institutional hierarchy, cross-tier flows, and the Local-Bank-to-client path that microfinance and development banks use in practice [1]. This study is motivated by the opportunity to encode that hierarchy on a programmable ledger—with enforced reserves, role-based governance, and auditable capital movement—while supporting (not replacing) human credit judgment through lightweight analytics and an optional conversational interface. The thesis specifies a reusable architecture; it does not claim a particular national deployment target.

1.3 Problem Statement

Three primary forces of motivation for this study: development-finance has restricting transparency and settlement; the absence of multi-tiered banking functions in DeFi (deposits, solidarity and lending); the keen interest in combining blockchain auditability with ML computational analytics. There are six structural inefficiencies

noticeable in development finance routes for individual borrowers from national and local banks:

1. **Lack of transparency:** there is very little chance of the lower tier users to verify capital management, reserve levels or lending decisions are made by the higher tiers, very less transparency on how money flows through the different levels of systems. Only the top tiers get the information and records of reserves and audited at most quarterly.
2. **Sluggish settlements and idle capital:** cross-border transactions are quite expensive, average $45 per transaction [101], and time consuming taking two to five days [23].
3. **Inconsistent risk evaluation:** fraud and credit assessment are completely relied on human judgement, borrowers are generally re-evaluated at each tier or institutions based on information available at that tier, no unified credit history.
4. **Access and trust barriers:** the cost of inter-institutional trust is quite expensive for legal and audit process. It creates congestions in developing countries where investors earn returns roughly 3% below compared to developed markets [103].
5. **No programmable savings:** depositors in developing economies have no real-time access to visibility into how savings are processed and deployed.
6. **No on-chain group credit:** without individual collateral, borrowers cannot access programmable on-chain mutual group lending with shared responsibility. Real-world organizations like BRAC exist but does not function like DeFi.

DeFi enabled automatic lending transparency at scale such as Aave v3 with $26.3 billion TVL, $17.7 billion borrows [18], Compound v3 ($1.4 billion) [19], MakerDAO with $10.5 billion stablecoin issuance [20]. Despite large market coverage they exhibit four limitations for institutional development finance:

- **Flat lending models.** Aave/Compound uses one protocol for many asset pools (USDC, ETH, etc.). There is no cross-tier allocation, no institutional hierarchy, no difference in role-based governance. Here our CWB models institutional relationships that go deep into what kind of services to provide to what kind of roles/institutions to increase liquidity in overall lending and exchange situations.
- **No AI risk analytics.** DeFi relies on collateral ratio, utilization curves. There is no behavioral fraud detection or explainable credit support system.
- **No tiered governance.** DeFi protocol treats tokens as a measurement of voting, not based on what is the role of the client or user or institution in the bigger to smaller picture as it lacks relation to a deeper level like institutions.

- **Single-level institutional DeFi.** Maple & Goldfinch [21] with $2.6–3.8 billion TVL and $680 million originations in 18+ countries respectively serve institutions, but none models a full multi-tier banking system with interbank flows, lack hierarchical capital flows.

1.4 Objective

The objectives of this project are:

1. Design and specify all features and constraints of a four-tier lending architecture on an EVM-compatible blockchain and complete a full testnet deployment (World Bank → National Bank → Local Bank → Client). The whole development cycle is planned for Phases I–IV.
2. Design not only pool lending, but specify an extended banking product suite on the hierarchical system (deposits, savings, and solidarity group lending).
3. Investigate a lightweight off-chain analytics layer using ML, training and testing the models in practical testnet for fraud detection, anomaly detection, explainable review, and an optional MCP-based conversational interactive agent for ease of use.
4. Specify programmable on-chain lending with borrowing limits based on financial behaviour analysis, installments, and role-based access control and information via smart contracts.
5. Plan and execute testnet deployment using Polygon zkEVM Cardona, Ethereum Sepolia, and evaluate hierarchical controls in the final thesis phase.
6. Document governance for future improvements, security analysis and feature improvements, role-based regulatory considerations, and a controlled rollout toward sandbox piloting.

1.5 Proposed Solution

The Crypto World Bank will implement a four-tier on-chain lending model: Tier 1 World Bank reserve custodian → Tier 2 National Banks → Tier 3 Local Banks processing retail loans → Tier 4 clients building on-chain credit history (Figure 1.1), with six standard banking functions enforced on-chain—deposit mobilization, credit allocation, payment and settlement, risk intermediation, liquidity management, and ancillary services—rather than discretionary audit alone. Banks also borrow from peers, park surplus with parent tiers, and syndicate large loans; the platform replicates four-tier downward flow, same-tier pools, upward deposits, and multi-bank co-funding on-chain (Figure 1.2).

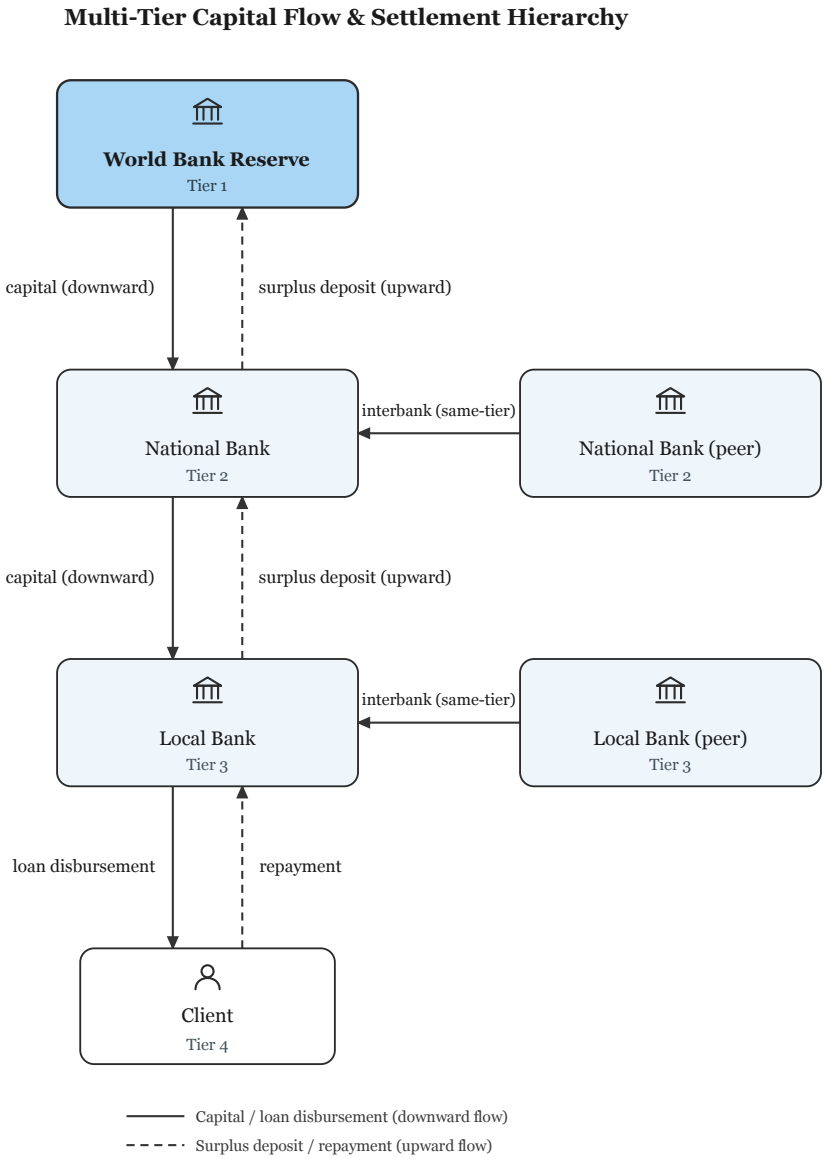

Figure 1.2: Cross-tier lending flows.

1.6 Methodology in Brief

There are four Agile phases to this methodology for this project. This report is designed with the architecture specification. The development phase will start from Phase II after pre-thesis 1. In this report there is architecture, contracts, database, and ML pipeline. The Phase II development phase will demonstrate the loan cycle. Phase III will start the implementation of ML scoring and the Chainlink oracle (Section 4.2.3). Phase IV will show the evaluation, metrics, and the overall feasibility of the project in the real world. Stack: Solidity, React, Express/FastAPI, scikit-learn on Polygon zkEVM Cardona. There is also an AI chat agent that will be built as an optional feature that will run alongside the project to help the clients navigate

better with the system.

1.7 Scopes and Challenges

In scope now. In our development phases, we have calculated that we can implement a limited amount of our full vision: the four-tier lending architecture, loan lifecycle workflow, borrowing limits for the clients and the banks, the design of Oracle fraud analytics, and the training and the live demonstration of the AI/ML workflow. It can be shown to a limited amount on the prototype builds only.
Out of scope for now. Mainnet deployment, live retail payments in the real world, production KYC/AML at the banking level and client interaction, stress testing the whole project, and providing an MVT checklist before the product is ready is not quite possible yet.
Main risks. There are a couple of risks that we have noticed for our project, starting with the fraud labels may not be completely accurate based on the limited amount of data we have for our system architecture. We need more synthetic data that will be collected over time. If regulations have changed—mitigated by testnet-only builds—then we might have to change the architectural elements. The gas toll on micro-loans may not be sustainable. The ML layer with SHAP may keep borrowers in the loop in the system since it is not 100% accurate and accuracy of the system depends on the training process. Rest of the economics and trade-offs are discussed in Section 3.7.

Economic sustainability. On Cardona, 10–15 txs on a typical loan cost under $0.15—small next to interest earned. Mainnet micro-loans would need tighter gas work.

Institutional capture. Another huge issue that needs to be addressed is that Tier 1 and the National Banks, which have huge reserves and influence over the rates, reserves, and who gets credit, might influence the overall system and cost-to-performance ratio. It might temper the trust and relationship with the retail customers and smaller banks.

Stablecoin-first lending. Retail clients are not to be carrying high-priced positions like ETH, since the prices of Ethereum might fluctuate. MiCA and the U.S. GENIUS Act support the idea that stablecoins should be more generalized for authorized cryptocurrency. There are many settlement projects like *mBridge* and *Agora* [61, 62], moving the value of cryptocurrencies across borders, but it is not a replacement for our lending system. The architectural system for stablecoin pools

and L2 deployment (Section 5.3) gives the liquidity move fluidly from the top tier to the bottom, and the surplus to go up from the bottom.

Chapter 2

Literature Review

2.1 Preliminaries

Important terms—XAI, PoS, CBDC, solidarity/group lending, DeFi, smart contracts, over-collateralization, ALM, flash loans, ZKP, and Health Factor—have been introduced in Glossary appendix. This section has been expanded in appendix D that will be presented in the final versions of the complete project file.

2.1.1 Review Methodology (PRISMA-Style Frame)

We did our research to find the appropriate papers and highly qualified papers that goes without project from ACM, IEEE, Elsevier, MDPI, arXiv, and BIS/World Bank (2017–2026) with PRISMA-style screening [52]. Primary topics for this search: DeFi lending, fraud ML, tiered finance, and smart-contract security.

2.2 Review of Existing Research

We explain our literature review, and our findings from the papers and the work of the authors, in the subsections shown below.

2.2.1 Decentralized Lending and DeFi Protocol Design

For a big picture, Werner et al. [1] show that DeFi lenders use flat pools with no bank-style tiers; we have found a gap that can be filled by introducing our project. According to Bastankhah et al. [2], adaptive borrow rates work better than Aave-style curves for financial flows, which we introduced in our kinked-rate module. Dao et al. [39] show how on-chain limits can get off-chain credit scores that in general are applied at our Local Bank level.

2.2.2 Machine Learning, Explainable AI, and Extended AI Security

For fraud detection, Palaiokrassas et al. [3] engineer behavioral features that do a really good job in DeFi transaction fraud detection, not just hashes. We plan to use Random Forest in Phase III that will further strengthen the fraud detection level with the mixture of hash functions that is built into the DeFi protocol. Moreover, for the situations when the fraud detection level is scarce, we will use Liu et al. [6] Isolation Forest. We also studied a very in-depth comparison of SHAP versus LIME by Adom et al. [4], which shows a significant advantage by using SHAP for loan detection; approvers in bank and authority level will see SHAP screening results before signing.

2.2.3 Hierarchical Distribution, Cross-Tier Flows, and Group Lending

Tan [5] demonstrated how CBDC flows in bank-to-bank and bank-to-client financial transactions. We extended this idea in our project by showing tier-based reserve checks. For layered reserve contracts, *Project Pine* [107] has shown a prototype that is similar to our `InterBankLendingPool` and the syndicated-style loan flow. We also integrated the style of microcredit admin cost with smart contracts based on the study of Asaduzzaman et al. [36]. The structure of `GroupLendingPool` function is the on-chain version, without claiming that we are trying to replace BRAC-style social microfinancing enforcement.

2.2.4 Smart Contract Security and Layer 2 Scalability

To improve the security system of the project, we are following Atzei et al. [7], which lists the main Ethereum contract attacks. Since the tier-based system is very complex, many transactions on Cardona might be triggered. Gas prices can be cut to 30–50% by transaction batching [102], with further DeFi gas optimisation [37].

2.2.5 Monetary Policy, Tokenization, and Institutional Landscape

Very large institutions like World Bank FundsChain [25] and similar institutions show that development finance is already moving on-chain for auditability; we are further improving this by encoding rules, not just managing disbursement logs.

2.2.6 Graph ML, Federated Learning, Regulation, and Identity Primitives

We noticed the usability and rise of stablecoin and went further deep into it and try to explore how stablecoin pools for retail clients can be strengthened by using MiCA and the U.S. GENIUS Act [58, 60]. For our project, soulbound tokens will be used that fit into our Credit Passport idea, which was extended by the idea of Buterin et al. [94]. This introduces a secured identity on-chain, a portable reputation, without the fear of exposing personal data.

2.2.7 Oracle-Mediated ML and Optional LLM Interfaces

Lastly, for an optional feature to our project, future studies can be made and features can be added to the system. Caldarelli [96] describes how ML scores still have some issues that go beyond price oracles; we focus on this part and extend our commit-reveal or Chainlink path targets. We collected some reputable databases, and more to be added such as the BCCC fraud dataset [99] that goes with our system. This section will be continued and extended after pre-thesis 2.

Table 2.1: Literature review summary (1/2).

Author & Focus	Methodology	What they found & how we use it
Palaiokrassas et al. (2024) [3] · DeFi fraud	XGBoost, NN on 54M+ txs	We plan to use behavioural features in Phase II for Random Forest, as it lifted F1 to 0.76–0.85
Adom et al. (2025) [4] · XAI lending	LIME / SHAP comparison	Approvers get a SHAP brief before signing, as it gives a clearer result than LIME on loan data
Tan (2023) [5] · CBDC hierarchy	Developing-nation model	CBDC backs the four-tier layout and works as a bank-to-user tier
Liu et al. (2008) [6] · Anomaly detection	Isolation Forest	Flags odd wallets without many labels; backup when fraud data is thin
Atzei et al. (2017) [7] · SC security	SoK of Ethereum attacks	Lists re-entrancy and access-control risks; guides OpenZeppelin guards
Werner et al. (2022) [1] · DeFi SoK	Survey of 12+ protocol types	Main gap CWB targets is DeFi lending having no bank hierarchy
Bastankhah et al. (2024) [2] · Adaptive lending	Dual fast/slow control	Dynamic rates support our kinked-rate modules as it beats static curves
Hartmann & Hasan (2021) [33] · P2P lending	Smart-contract origination	Gas is cut in on-chain P2P, which is a minor input for loan-facing clients
Pranto et al. (2022) [34] · Privacy-preserving fraud ML	Incentive-based fraud detection	Lack of raw-data sharing features by banks leaves future cross-bank options
Wang et al. (2021) [105] · SC vulnerabilities	ML on bytecode	Common bytecode bugs spotting by ML helps provide extra security over manual audit
Liao et al. (2019) [106] · Audit automation	Classification + fuzzing	Workflow idea for Phase V is inspired from ML prioritizing fuzzing speed

Table 2.2: Literature review summary (2/2).

Author & Focus	Methodology	What they found & how we use it
BIS et al. (2020) [35] · CBDC design	Retail distribution analysis	Standard CBDC two-tier is extended to four-tier with on-chain reserves
Kiff et al. (2020) [80] · CBDC models	IMF architecture survey	Matching National/Local roles; retail CBDC flow implemented through supervised banks
Bindseil (2020) [79] · CBDC rates	ECB tiered remuneration	Upward deposit spread idea inspired from tiered rates that protect bank intermediation
Dong et al. (2023) [78] · Supply-chain finance	Three-tier game model	Capital can move down a chain, parallel to World Bank → client allocation
Project Pine (2025) [107] · Hierarchical SCs	NY Fed / BIS prototype	IBLP and syndicated loans used on-chain layered reserve contracts prototype
CPMI/FSB [15, 23] · Cross-border settlement	Nostro/vostro cost analysis	Tiered on-chain flow is better than legacy settlement as it is slow and costly
World Bank (2025) [25] · Dev. finance blockchain	13-project pilot	Programmable lending rules added with dev-banks system
Chen et al. (2024) [31] · Multi-chain lending	Cross-chain design	Scaling by splitting loads throughout chain is later option, not for Phase I
Yang & Cui (2023) [32] · Protocol evaluation	Quantitative risk metrics	Gives utilisation and liquidation benchmarks for health checks tier
Asaduzzaman et al. (2020) [36] · Smart micro-credit	Smart-contract MFI	Model for `GroupLendingPool` is smart contracts, which cuts the admin costs in Bangladesh
Wang et al. (2021) [102] · Tx batching	iBatch middleware	Per call saved 15–59% in gas price by which we deploy on Cardona L2
Kim & Kim (2024) [37] · Gas optimisation	DeFi parameter tuning	∼18% gas cut on Ethereum pools helps our L2 deployment
Yuan (2024) [38] · SHAP credit default	RF, GBM + SHAP	Deploying on Cardona L2 as batching saved 30–50% gas; SHAP is also used for our decline path

Feature	Aave	Comp.	Maker	Maple	Gfinch	CWB
Institutional hierarchy	✗	✗	✗	✗	✗	●
Cross-tier capital flow	✗	✗	✗	✗	✗	⧗
Same-tier interbank lending	✗	✗	✗	✗	✗	⧗
Syndicated / co-lending	✗	✗	✗	Part.	Part.	⧗
Upward capital repatriation	✗	✗	✗	✗	✗	⧗
Solidarity group lending	✗	✗	✗	✗	Part.	●
AI/ML fraud detection	✗	✗	✗	Man.	Man.	●
SHAP explainability	✗	✗	✗	✗	✗	●
ZKP KYC compliance	✗	✗	✗	✗	✗	●
Kinked interest rate curve	✓	✓	Gov.	✓	✗	⧗
Role-based access control	Part.	Part.	Gov.	✓	Part.	●
Institutional / L2 focus	✗	✗	✗	✓	Part.	●
TVL / Status (2026)	$26B	$1.4B	$10B	$2.6B	$680M	Planning

Table 2.3: Protocol comparison.

2.3 Summary of Key Findings

Four points stand out after reviewing and Table 2.3:

- **Flat DeFi vs. tiers.** Large pool lenders [1, 8] have no hierarchical bank currently, and it opens a door to explore how a hierarchical system would behave with DeFi. CBDC and Project Pine work [5, 107] prove that nobody combines a four reserve-enforced tier with group lending and cross-tier financial flows and an ML oracle in a singular architecture.
- **ML and security.** ML and security layers combined with behavioural fraud features and SHAP [3, 4, 6] further extend the usability, feasibility, and adapt-

ability of our project. Based on these literature reviews, we further improve our Phase III pipeline; contract attack catalogues [7] and L2 gas batching [102, 37] justify OpenZeppelin guards to be added to our project.

- **Groups and institutions.** We explored smart-contract microcredit in Bangladesh [36] and World Bank on-chain pilots [25] that gave us the idea and improved the implementation of `GroupLendingPool` function and institutional lending system.
- **Regulation.** For regulation improvement and securities, MiCA and GENIUS [58, 60] are accepted by stablecoin-first pools. Soulbound credit tokens [94] fit the Credit Passport without putting KYC on-chain.

Chapter 3

Requirements, Impacts and Constraints

3.1 Final Specifications and Requirements

The functional and non-functional requirements are shown in Table 3.1 and Table 3.2; based on them, the software engineering with in-depth architecture and system analysis is specified in Chapter 4. The implementation plan has been specified in Section 3.5.1. The MVT checklist has been specified in Table 3.3, and the complete features list is in Table 3.4.

Table 3.1: Functional requirements.

ID	Requirement	Description
FR-01	Four-tier hierarchy	World, National, and Local Bank contracts with role-gated downward capital allocation and client onboarding at Local Bank tier.
FR-02	Loan lifecycle	Request → ML score commit–reveal → approver decision → disbursement → installment schedule → repayment.
FR-03	Reserve enforcement	Each tier retains a minimum reserve ratio before allocating capital downward; on-chain revert on breach.
FR-04	Borrowing limits	Rolling six-month and one-year limits enforced on-chain; off-chain projection in `BORROWING_LIMIT`.
FR-05	Risk scoring	Off-chain RF + IF + stacking; SHAP explainability; immutable `LOAN_RISK_ASSESSMENT` audit row per inference.
FR-06	Group lending	`GroupLendingPool` with mutual liability, consent, and per-member share disbursement (Section **??**).
FR-07	Event synchronisation	Blockchain event listener projects on-chain state into PostgreSQL for dashboards and agent context.
FR-08	Dual client interface	React dashboard and MCP conversational agent call the same Express.js API and contracts.

Table 3.2: Non-functional requirements.

ID	Category	Constraint / target
NFR-01	Performance	ML inference $\leq$50 ms on laptop hardware; API p95 $\leq$500 ms for read endpoints at MVT scale.
NFR-02	Security	RBAC on contracts and API; `ReentrancyGuard`; human approver gate before disbursement; agent write-path confirmation.
NFR-03	Scalability	Modular contract deployment; PostgreSQL indexing (Appendix **??**); L2/testnet gas target sub-cent per retail lifecycle step.
NFR-04	Maintainability	TypeScript/React/Express monorepo at repository root; OpenZeppelin primitives; documented phase gates G1–G4.
NFR-05	Usability	Mobile-first React UI; five-stage onboarding funnel; Authority Brief for approvers (Section 4.2.10).
NFR-06	Compliance (design)	Tiered KYC ladder; GDPR data minimisation (hash-only on-chain); EU AI Act Art. 86 explainability artifacts.
NFR-07	Reliability	Pause mechanism on contracts; zero-cost demo substitutes with documented upgrade path (Section 3.6).
NFR-08	Interoperability	EVM-standard contracts; WalletConnect; ERC-20/ERC-4626 patterns; MCP tool schema for agent integration.

3.1.1 Minimum Viable Thesis (MVT) Checklist

Table 3.3: MVT deliverable checklist.

#	Deliverable	Priority	Phase
1	Four-tier bank contracts on testnet	Must	I
2	Loan lifecycle E2E (request, approve, disburse, repay)	Must	II
3	On-chain reserve and borrowing limits	Must	II
4	ML risk scoring (RF + IF + stacking)	Must	III
5	SHAP explainability and Authority Brief UI	Must	III
6	Risk assessment audit row per inference	Must	III
7	Commit–reveal score gate before approval	Must	III
8	BCCC fraud-detection benchmark	Must	III
9	300-client simulation or gas-cost table	Must	IV
10	README with reproduction instructions	Must	IV
11	MCP agent (3–5 tools + confirmation gate)	Must	III
12	LLM evaluation protocol	Should	IV
13	Contract audit reports (Slither + Mythril)	Must	IV
14	Foundry fuzz and invariant tests	Must	IV
15	Safe multisig for World Bank admin	Must	I
16	Tenderly monitoring alerts	Should	IV

3.1.2 List of Features

Table 3.4 lists the complete list of features of the full project. The Must features will be completed during the development phases, Should and Stretch features are extensions of the main Must features, and Specified features are documented but not yet implemented, Future Work features are to show the potential improvements of the project and some of these features might be deliverable by the end of the project as special features.

Table 3.4: Feature inventory.

#	Feature	Priority	Phase
Core platform and hierarchy			
1	Four-tier banking hierarchy (World → National → Local → Client)	Must	I
2	Role-based access control (RBAC) on contracts and API	Must	I

Table 3.4 (continued on next page)

Table 3.4 (continued)

#	Feature	Priority	Phase
3	World Bank Reserve — global capital custody	Must	I
4	National Bank — jurisdiction-level allocation	Must	I
5	Local Bank — retail operations hub	Must	I
6	Downward capital allocation (`allocateCapital`)	Must	I
7	Hierarchical bank registration (National → Local)	Must	II
8	Reserve-ratio enforcement before downward allocation	Must	II
9	MockUSDC / testnet stablecoin for lending	Must	I
10	2-of-3 Safe multisig for `WorldBankAdmin`	Must	I
11	Contract pause / unpause (emergency controls)	Must	I
12	Single-chain deployment (Sepolia / Amoy)	Must	I–IV
Retail lending lifecycle			
13	Loan request submission (on-chain + UI)	Must	II
14	Approver review workflow (approve / reject / request info)	Must	II
15	Loan disbursement to client wallet	Must	II
16	Installment schedule generation	Must	II
17	Installment payment processing	Must	II
18	On-chain loan state machine (`PENDING_SCORE` → `ACTIVE`)	Must	II
19	Borrowing limits (six-month / one-year rolling caps)	Must	II
20	Collateral-based lending (LTV)	Must	II
21	Credit-based lending (no collateral, SBT-gated)	Should	II
22	Kinked interest rate model (utilization-based)	Should	II
23	Credit tier schedule (Bronze → Diamond)	Should	II
24	Overdue installment checks (cron / Chainlink Automation)	Should	II
25	Loan history and transaction tracking UI	Should	II
26	Income verification document upload and review	Should	II
Deposits and savings products			
27	SavingsVault — variable-yield savings (ERC-4626)	Should	II
28	FixedDeposit — term deposits (30/90/180/365 days)	Specified	II+
29	Current / checking account (atomic transfers)	Specified	II+
30	Deposit-to-lending pool integration	Should	II
31	Interest split (depositor / InsuranceFund / protocol)	Should	II
32	Withdrawal gated by reserve ratio	Must	II
33	InsuranceFund (5% interest capture, default coverage)	Should	II
Group and solidarity lending			
34	Group formation (3–20 members)	Must (C2)	II
35	Multi-signature group consent	Must (C2)	II
36	Per-member disbursement share	Must (C2)	II
37	Shared collateral / mutual liability	Must (C2)	II
38	Over-indebtedness controls (DTI, cooling-off, max two group loans)	Should	II
39	Group credit history on SBT	Should	II
Multi-entity and cross-tier operations			
40	InterBankLendingPool — same-tier peer lending (1/7/30 day)	Should	II
41	UpwardDepositFacility — surplus repatriation to parent tier	Should	II

Table 3.4 (continued on next page)

Table 3.4 (continued)

#	Feature	Priority	Phase
42	SyndicatedLoan — multi-bank co-funding	Stretch	III
43	TranchedPool — senior/junior tranches	Stretch	III
44	TreasurySwap — cross-tier treasury FX	Should	III
45	FXModule — oracle-priced retail FX	Should	III
46	Dual-currency facility (fiat ↔ crypto)	Specified	III
47	Cascading repayment / interest across tiers	Should	II
48	NettingEngine — multilateral settlement	Future Work	—
49	Trade finance / escrow letters of credit	Future Work	—
Credit passport and identity			
50	On-chain Credit Passport (Soulbound Token)	Must	II
51	Credit score, tier, and repayment history on SBT	Must	II
52	Tier-based max loan limits and rate modifiers	Should	II
53	Wallet authentication (MetaMask / WalletConnect)	Must	I
54	EIP-712 typed-data login + JWT session	Must	I–II
55	Five-stage retail onboarding funnel	Should	I–II
56	Tiered KYC (Level 1 / Level 2)	Must	I–II
57	Off-chain KYC document storage (hash-linked)	Must	I–II
58	ERC-4337 account abstraction (gasless onboarding)	Stretch	III
59	Groth16 zkKYC / zkAML circuits	Future Work	—
AI/ML risk and explainability			
60	Random Forest fraud probability scoring	Must (C3)	III
61	Isolation Forest anomaly detection	Must (C3)	III
62	Stacking meta-learner (calibrated composite score)	Must (C3)	III
63	SHAP explainability (top-k features)	Must (C3)	III
64	Authority Brief UI for approvers	Must (C3)	II–III
65	Commit–reveal oracle score gate before approval	Must	III
66	`LOAN_RISK_ASSESSMENT` audit row per inference	Must	III
67	BCCC-DeFiFraudTrans benchmark evaluation	Must	II–III
68	Chainlink Functions DON oracle	Future Work	—
69	GNN (GraphSAGE) ablation	Future Work	—
70	Federated learning across banks	Future Work	—
Conversational agent			
71	Qwen3-8B local LLM chat interface	Must	III
72	MCP tool server (3–5 tools at MVT)	Must	III
73	Human confirmation gate for all write tools	Must	III
74	Read tools: account state, loan status, requirements	Must	III
75	Write tools: loan application, installment payment	Must	III
76	RAG over policy documents (Q&A path)	Must	III
77	Live on-chain context injection per turn	Must	III
78	Three-tier prompt assembly (Stable / Context / Volatile)	Must	III
79	Prompt injection scanning	Must	III
80	`AGENT_ACTION_LOG` audit trail	Must	III
81	Full 17-tool MCP suite	Future Work	—
82	EIP-7702 scoped session keys for agent	Stretch	III
Frontend, API, and data layer			
83	React + TypeScript dashboard (mobile-first)	Must	I–II
84	Reserve transparency dashboard	Should	III

Table 3.4 (continued on next page)

Table 3.4 (continued)

#	Feature	Priority	Phase
85	Five-state transaction UX machine (idle → success/error)	Should	II
86	PostgreSQL event listener (on-chain → DB sync)	Must	I–II
87	Real-time notifications (WebSocket / Socket.IO)	Should	III
88	Client–bank chat	Should	II
89	Express.js REST API with RBAC	Must	I–II
90	FastAPI ML inference service	Must	II–III
91	API rate limiting and correlation-ID tracing	Should	II
92	The Graph subgraph (optional query layer)	Future Work	—
Security, governance, and compliance			
93	Five-layer defense-in-depth architecture	Specified	I–IV
94	ReentrancyGuard + CEI pattern on contracts	Must	I–II
95	LiquidationEngine (health factor, liquidator bonus)	Should	III
96	SAR workflow (anomaly → freeze account)	Should	III
97	Governance timelock + snapshot voting	Specified	III+
98	Slither + Mythril static analysis	Must	IV
99	Foundry fuzz + invariant test suite	Must	II–IV
100	Tenderly monitoring alerts	Should	III–IV
101	Regulatory read-only audit access	Specified	III+
102	GDPR-style data minimisation (hash-only on-chain)	Specified	I–IV
Evaluation and operations			
103	300-client Foundry / Hardhat simulation	Must	IV
104	LLM red-team evaluation protocol	Should	IV
105	Open-source README with reproduction steps	Must	IV

3.2 Societal Impact

The existing global financial system does not provide equal access to financial lending capabilities; tier-based financial flows are often kept hidden from a general client who has little to no financial literacy, and bankers often try tactics to misdirect clients into taking loans since transparency is not their priority. In general, the rich have the ability to back up loans of large amounts by showing their assets and potential profit margins and are given priorities, and the poor who might need financial help as loans for critical issues are ignored for the higher profit margins of banks. Programmable smart contracts are being utilized to generalise the loan limits and loan request submitting and approval system that improves equal access globally to those who need it regardless of financial class in a society and who have the ability to repay a loan despite their reputation set by typical judgemental views of traditional bank authorities. All the retail clients are treated equally; if the client can provide necessary documents for proof of income and stability and stakes, he or she may get loan approval from the *Crypto World Bank* branch that has been set for that local client. The benefit of the smart contracts in CWB is that they are completely transparent for all the tiers, so clients can understand how currency is

being made sustainable and functional for all; the blockchain comes with the added benefit of decentralisation, so no single party holds the ability to stop financial flow for one section of the world as a restriction of equal access to currency. It is not just a project to show technical design; it is made to prove that blockchain technology and cryptocurrencies can be introduced to the general mass for universal usage, and a lending platform can be built to improve the traditional tier-based banks.

3.3 Environmental Impact

One of the widely popular major complaints of using a blockchain network to build projects and products on it is the energy consumption and its effect on the environment. Running a proof-of-work (PoW) blockchain calculation consumes a massive amount of energy by the servers or personal computer systems, and the waste of energy compared to the end results of calculations is hard to justify.
That is why our project, CWB, runs on proof-of-stake (PoS) EVM, which is 99.9% more energy efficient compared to PoW consensus chains. The power consumption of Bitcoin's PoW network is estimated at 120 to 150 TWh annually, which is 0.4–0.5% of total global electricity consumption, where our approach to serve a worldwide unified lending platform should not exceed electricity consumption of a few small commercial buildings to keep the servers and network operational.
During development, Ethereum Sepolia will be used for demonstration and to prepare the testnets, and in future Polygon zkEVM Cardona is preferred for more efficient gas cost optimisation and more efficient settlement. The ML layer risk score containing fraud detection, anomaly detection and explainability runs on mid-tier commodity hardware or free cloud services; these are not very expensive services as we planned with lightweight models. ML models are not processed on-chain with the smart contracts; the loan approval, financial flow and other banking services are off-chain work, which reduces stress of on-chain calculations to minimize complexities. During the pre-thesis stage, the total estimation of carbon emission and electricity usage is hard to determine; this information will be more predictable after the final execution of the prototype.

3.4 Ethical Issues

The ethical statements have been clarified at the beginning of the paper. The intention of CWB is to demonstrate how this technology and development stack can be used to create a blockchain-based lending platform with AI-enhanced security and assistance. There is no intention to store and use user data, monetary profit, or to build services for financial gains. The full development process and code are

open source and available for use publicly in the official GitHub repository for the project.
For the banking system, client data are mostly controlled and processed for on-chain use using hashes, and the AI-assisted techniques that have been utilized here only help the clients to navigate and suggest how to use the platform; all major decisions regarding loan approval are controlled by human approval gates. The ML layers that use client data are not stored and only used for each session time when a user visits the platform; the risk scores and SHAP explainability are designed to not expose any critical data about the users and the platform adapters.
For professional-grade usage of the platform in the future, clients are managed by fair rules and regulation; the treatment for the lending system and money allocation system follows the traditional banking system. No extraordinary social, psychological, financial systems or laws have been applied here, as this build is a prototype which may evolve in future for more robust usage and more advanced adaptability strategies may be used.

3.5 Project Management Plan

3.5.1 Implementation Phase Plan and Deliverables

Phase I: Foundation and Infrastructure (Weeks 1–4)

Phase I focuses on the foundation layers, such as the smart contracts of World Bank, National Bank, and Local Bank, setting the right format and design of the database and schemas, and successfully deploying the smart contracts on Sepolia to show the contracts are functional. Priority follows the MoSCoW classification:

Table 3.5: Phase I task register.

Task	Priority	Development Task	Effort (days)
DT-I.01	Must	World Bank contract — reserve management, national bank registration	5
DT-I.02	Must	National Bank contract — borrow from World Bank, lend to Local Banks	5
DT-I.03	Must	Local Bank contract — borrow from National Bank, lend to users	5
DT-I.04	Must	RBAC — WorldBankAdmin, NationalBankAdmin, LocalBankAdmin, BankUser, RetailClient	3
DT-I.04a	Must	`MockUSDC.sol` — mintable ERC-20 for testnet loans (Section 3.6)	1
DT-I.05	Should	Gas cost management — initiator-pays pattern; Polygon low-fee design	2
DT-I.06	Should	`InsuranceFund` — 5% interest capture, claims, default coverage	4
DT-I.07	Should	`getReserveSummary()` on each tier contract (PSR, reserve, insurance fund)	2
DT-I.08	Should	Chainlink Price Feeds (fiat/USD, ETH/USD via `AggregatorV3Interface`)	3
DT-I.09	Should	Chainlink Proof of Reserve — WorldBankReserve attestation	2
DT-I.10	Should	Append-only `audit_logs` PostgreSQL role + RLS policies	2
DT-I.11	Could	Polygon zkEVM Cardona migration — RPC, chain IDs, factory addresses	1
DT-I.12	Must	Phase I schema migration — 13 M1 tables; M2/M3/stub tables deferred	4
DT-I.13	Must	Core indexes and referential integrity constraints	2
DT-I.14	Must	React frontend shell — Vite build, routing, layout components	3
DT-I.15	Must	WalletConnect — connection, address display, network switching	2
DT-I.16	Should	Sessions and assets tables in frontend (dashboard stubs)	2
DT-I.17	Should	Credit tier and interest rate schedule display	2
DT-I.18	Must	Safe 2-of-3 multisig + `WorldBankAdmin` role transfer (MVT item 15)	1

Phase I effort: 31 Must days; 51 all priorities.

Phase II: Core Banking and On-Chain Lifecycle (Weeks 5–9)

For Phase II, we focus on building the React user interface to be responsive with loan requests and other banking functionalities such as loan approval, installments, registering accounts, and setting borrowing limits. Moreover, this phase builds the building blocks for Phase III that will incorporate the conversational agent and the tools that the agent has access to.

Table 3.6: Phase II task register.

Task	Priority	Development Task	Effort (days)
DT-II.01	Must	Loan request submission with blockchain transaction	5
DT-II.02	Must	Loan approval and rejection workflow with bank approver	5
DT-II.03	Must	Installment payment system with automated schedule generation	8
DT-II.04	Must	Borrowing limit enforcement (on-chain authoritative check per client SBT)	5
DT-II.05	Should	Client–bank real-time chat system	5
DT-II.06	Should	Income verification document upload and review	5
DT-II.07	Must	Hierarchical bank registration (National → Local)	5
DT-II.08	Should	Bank user management and approver designation	3
DT-II.09	Should	Loan history and transaction tracking pages	5
DT-II.10	Could	QR code generation for wallet addresses	2
DT-II.11	Should	Responsive UI polish and error handling	2
DT-II.12	Should	Authority Brief UI (SHAP breakdown for bank approver dashboard)	5
DT-II.13	Should	Installment due-date checks: Node.js cron (MVT demo path) or Chainlink Automation upgrade when LINK funded	5
DT-II.14	Should	Credit tier schedule in Credit Passport SBT (Bronze → Diamond)	3
DT-II.15	Should	`interest_rate_tier` table integration with Kinked Rate Model	2
DT-II.16	Should	`agent_action_log` + `SESSIONS` schema migration (agent audit trail foundation)	3
Phase II total estimated effort (Must-have):			*28 days*
Phase II total estimated effort (all priorities):			*68 days*

Phase III: AI/ML Oracle Pipeline and Conversational Agent (Weeks 10–13)

Phase III is the continuation of Phase II, using all lending functionalities such as loan request, approval workflow, and adding the ability of the MCP agent to interact with these tasks. The ML layer is also integrated with the oracle link and a connection between the on-chain and off-chain is established. Implementation of conversational agent harnessing including MCP server, conversational UI, and confirmation gates from both client and bank authority.

Table 3.7: Phase III task register.

Task	Priority	Development Task	Effort (days)
DT-III.01	Future Work	Chainlink Functions oracle (production upgrade): DON-based score commit when supported; commit–reveal is MVT demonstration path	6
DT-III.02	Must	RF + Isolation Forest + SHAP pipeline (FastAPI); Authority Brief explainability	7
DT-III.03	Must	MCP tool server — minimal (3–5) tool subset wired to Express.js API; one write demo E2E	4
DT-III.04	Must	Agent chat interface (Qwen3-8B + tool calling + human-gate)	8
DT-III.05	Should	Session-key management (EIP-7702) and scope enforcement	5
DT-III.06	Must	Three-tier prompt constructor in Express.js	5
DT-III.07	Must	Prompt injection scanning + confirmation audit hook (Hook 1)	5
DT-III.08	Should	Session lineage + `get_session_history` MCP tool	5
DT-III.09	Should	Context compression path + toolset scoping + Hook 2	9
DT-III.10	Should	(Optional) The Graph query layer + WebSocket event listener for Reserve Transparency Dashboard	4
DT-III.11	Should	SAR workflow: Isolation Forest → aml-alert → `freezeAccount`; compliance review queue integration	4
DT-III.12	Should	Reserve Transparency Dashboard: live queries to event-listener index (Graph optional later)	3
DT-III.13	Could	GraphSAGE GNN ablation	5
DT-III.14	Could	Federated learning aggregator	7
Phase III total estimated effort (Must-have):			*29 days*
Phase III total estimated effort (all priorities):			*77 days*

Phase IV: Verification, Evaluation, and Finalization (Weeks 14–16)

Implementing the Foundry fuzz suite, fully functional 300-client simulation, testnet deployment (local + remote), and the LLM agent evaluation reports, and finally paper refinements and finalization.

Table 3.8: Phase IV task register.

Task	Priority	Development Task	Effort (days)
DT-IV.01	Must	300-client Foundry simulation on Anvil/Hardhat (deterministic seed); gas-cost table and reserve-dynamics manifest to `deployments/testnet/`; optional Sepolia replay for explorer evidence	7
DT-IV.02	Should	LLM evaluation protocol: task accuracy, action accuracy, human-gate bypass (Section 4.2.16; MVT item 12)	5
DT-IV.03	Future Work	Certora reserve proofs: CVL specs for Invariant 1 (reserve never undercollateralised) and Invariant 2 (no overallocation downstream); specs in Appendix 6.2	5
DT-IV.04	Should	Foundry invariant + fuzz suite: solvency, role segregation, capital-flow direction; 10,000 fuzz runs per CI build	4
DT-IV.05	Should	AML pre-check (Hook 3): IF anomaly gate before disbursement writes; compliance queue routing; requires Phase III IF pipeline	3
DT-IV.06	Could	Groth16 zkKYC (Circom 2.0): ID possession proof without PII; circuit I/O specification	5
DT-IV.07	Must	Final paper revision: tool-count and phase refs consistent; master checklist complete; bibliography verified	4
DT-IV.08	Must	Red-team testing: 20 injection payloads; 20 bypass attempts; 10 session-scope scenarios; results in evaluation subsections	3
DT-IV.09	Must	Slither + Mythril on World/National/Local Bank contracts; triage and archive (MVT item 13)	2
DT-IV.10	Should	Tenderly alerts: 3 demo-critical triggers + README reproduction (MVT items 10, 16) 30	2
Phase IV total estimated effort (Must-have):			*16 days*
Phase IV total estimated effort (all priorities):			*40 days*

Table 3.9: Four-phase roadmap.

Phase	Focus	Weeks	Tasks	Key deliverables
I	Foundation and infrastructure	1–4	DT-I.01–18	Contract hierarchy, `MockUSDC`, core DB schema, React shell
II	Core banking and on-chain lifecycle	5–9	DT-II.01–16	Lending workflow, SBT limits, cron/Automation triggers
III	AI/ML oracle pipeline + conversational agent	10–13	DT-III.01–14	Commit–reveal oracle, ML pipeline, MCP agent (minimal toolset), dashboard
IV	Verification, evaluation, finalization	14–16	DT-IV.01–10	Foundry, simulation, evaluation pack, testnet deploy

3.5.2 SDLC Stage Mapping

Table 3.10: SDLC stage mapping.

#	SDLC Stage	Project Activity	Deliverable
1	Planning	Feasibility studies; professor consultations; guideline review	Feasibility Report; Project Plan
2	Requirements	System analysis; use case definitions; constraints identification	Use case diagrams; UC-1 through UC-11 (eleven grouped use cases)
3	Design	Four-layer architecture; DB schema (51 entities, 3NF); smart contract interfaces	Architecture diagrams; ERD; DFD
4	Development	Phase I–IV: smart contracts, frontend, backend, AI/ML, agent	Source code; DApp prototype
5	Testing	Hardhat unit tests; Foundry fuzz; integration testing; AI/ML + agent evaluation	Test reports; model metrics
6	Deployment	Ethereum Sepolia (Phases I–IV demonstration); Certora on Sepolia (Phase IV stretch); frontend (Vercel); backend local-first (Render optional)	Testnet prototype manifest
7	Maintenance	Monitoring; model retraining; bug fixes; iteration	Updated docs; retrained models

3.6 Risk Management

Table 3.11: Design vs. demonstration path.

Area	Full design	Demo path (Ph. I–IV)	Risk mitigated
Network	Polygon zkEVM Cardona	Sepolia primary; Amoy secondary	Testnet instability
Stablecoin	USDC / USDT	`MockUSDC.sol`	No official testnet USDC
Oracle	Chainlink DON	Commit–reveal relay	Oracle cost / latency
ML delivery	Chainlink Functions	FastAPI + local relay	Integration complexity
Simulation	300 wallets live	Foundry/Anvil + 5–10 live demo	Faucet funding
Hosting	Cloud API + ML	Local stack + Vercel frontend	Cold-start / cost

3.7 Economic Analysis

3.7.1 Prototype Cost Model

Table 3.12: Economic feasibility.

Cost Category	Estimate	Notes
Blockchain deployment	$0	Public testnets — no real cryptocurrency required
Frontend hosting	$0	Vercel free tier or localhost for demo
Backend hosting	$0	Render free tier or localhost
AI/ML training	$0	Local machine or Google Colab free tier
Development tools	$0	Hardhat, VS Code, Git — all open-source
Total prototype	**$0**	Entire prototype operates at zero financial cost

3.7.2 Gas Cost and Break-Even

Table 3.13: Gas cost assumptions.

Parameter	Estimate	Notes
On-chain steps per loan lifecycle	27–32	Full retail lifecycle
Gas price (planning)	≈30 Gwei	—
ETH price (planning)	$2,500	—
Total gas (Cardona / L2)	≈$0.011	<0.01% of interest on $25,000 USDC loan at 8% APR
Total gas (Ethereum mainnet)	$2.40–$4.80	Same lifecycle; supports L2 design choice [37]
Average loan size	$25,000	Break-even base case
Default rate assumption	8%	—
Infrastructure cost	$500/month	Pilot-scale assumption
Break-even active loans	≈200	Covers default losses at base case

3.7.3 Net Present Value and Return on Investment

Table 3.14: NPV and ROI summary.

Year	Benefits ($k)	Costs ($k)	Net ($k)	PV factor @ 10%
0 (setup)	0	6.0	−6.0	1.000
1	48.0	6.0	42.0	0.909
2	72.0	8.0	64.0	0.826
3	96.0	10.0	86.0	0.751
NPV (sum of discounted net)				**$149.6k**
ROI (3-yr benefits − costs) / costs				**620%**
Break-even loans (base case)				**≈300**

Chapter 4

Proposed Methodology

This project paper is constructed to match with the final year project evaluation rubrics. Full development work has been distributed to 4 phases, and priority of all the capability sections has been divided to Must, Should and Future work. The Must and Should priority sections are to be completed by the end of the development and testing phase, Future Work sections have been specified only, to show the Potential of the project by extending its functionality and development scopes.

Table 4.1: Capability priority summary.

Capability	**Priority**	**Primary interface / phase**
Four-tier smart contracts	Must (Ph. I–II)	React UI + wallet; Hardhat/Foundry tests
On-chain loan lifecycle	Must (Ph. II)	React forms; `LoanController` state machine
Chainlink Functions oracle	Future Work	FastAPI → DON → on-chain score commit; commit–reveal is MVT demonstration path
RF + Isolation Forest + SHAP	Must (Ph. III)	Approver Authority Brief; client decline reasons
Authority Brief UI	Should (Ph. II–III)	Bank approver dashboard (React) — *not* agent-only
MCP conversational agent	Must (Ph. III)	Qwen3-8B + 3–5-tool MCP server; one write demo E2E
LLM evaluation protocol	Should (Ph. IV)	Red-team + task accuracy (Section 4.2.16; deferred)

4.1 Design Process or Methodology Overview

We have developed a plan to adapt lightweight Agile/Scrum Methodology for the prototype build with an estimated development window of 16 weeks. This iterative approach enables us to explore and improve the demonstrable features that include the minimum requirements and advanced research oriented features. These feature development phases are organised in four incremental stages (Section 3.5.1).
Agile Methodology application process:

- **Phase duration:** each phase is designed for 3–5 weeks, total 4 phases, total estimated time — 16 weeks.
- **Team Size:** 2 developers
- **Weekly Sync:** One weekly meeting will be conducted on Tuesday. Completion of each feature, testing and next development plans will be discussed in these meetings, based on the opinion and suggestions received from the supervisor.
- **Phase Planning and review:** After completion of each phase, a long reflective phase meeting will be arranged, to organize and plan upcoming weeks.

4.1.1 Process Model Selection

Based on the Software engineering process for a blockchain project, a development plan of Agile/Scrum with incremental phases has been chosen. Other models such as Waterfall, Spiral, V-Model have some flaws that do not meet the requirements to successfully build all the features.

Table 4.2: Process model comparison.

Model	Fit for CWB	Limitation	Decision
Waterfall	Clear phase documents	Cannot absorb evolving smart-contract and ML scope mid-project	Rejected as primary
Incremental	Matches four phases + gates G1–G4	Still needs within-phase flexibility	**Adopted** (phase structure)
Iterative / Agile	Weekly sync; phase retrospectives	Requires discipline on scope	**Adopted** (within-phase)
Spiral	Strong for high-risk R&D	Overhead excessive for 16-week student project	Not adopted
V-Model	Maps to test levels in Ch. 5	Too rigid for research prototyping	Used for *verification planning* only

The project is built in the main branch from the root section. It follows a conventional **Model–View–Controller (MVC)** architecture for separation in root. **Model** — `contracts/`: on-chain state machines and PostgreSQL (off-chain projections), **View** — `frontend/` (React components), **Controller** — `backend/`. For version control the repository is managed with git command, with consistency and compatibility checks, validated by `npm run test:contracts` (Hardhat) before each merge into main branch.

4.1.2 Implementation Feasibility and Demonstration Scope

Full project scope is documented with what will be completed and demonstrated before the defense, through completion of the project in the phase task registers (DT-I.01–IV.10), the MVT checklist (Table 3.3), and the module inventory in Chapter 4. The phases have been designed with balance in mind, across the time window and complexity of tasks. All tasks are distributed across four phases; estimated completion time is 16 weeks, including all features and bug fixes. However, the demonstration prototype may be completed well before the 16-week window.
All details about phase task distribution can be found in Section 3.5.1, the Chapter 3 MVT checklist (Table 3.3), and the full feature list (Table 3.4). It covers demonstration criteria for each phase, involving frontend, backend, and conceptual theory through practical implementation of each aspect. Those details are not repeated in

this section.

Features flagged as **Future Work** are designed for later delivery, given more time and manpower for stable development. Some features are over-engineered relative to the graded prototype and are retained as design depth for high-end complexity rather than MVT deliverables. Examples include: the **full 17-tool MCP suite**, which is complex and time-consuming and may require a team of 6–8 developers over several months; **GNN (GraphSAGE) ablation** and **federated learning across banks**, which belong to the research literature and could form an MSc-level research section in future; **Groth16 zkKYC / zkAML circuits**, still unstable in practice but heavily impactful for efficiency and privacy; and **NettingEngine** multilateral settlement plus **trade finance / escrow letters of credit**, which require further manual research and study.

The MVT list may be enhanced while developing the project, as it is difficult to determine which experimental features will be functional. The design and concept of the project are quite novel—similar projects are not widely available online—so the experimental phases may require considerable tweaking and change based on challenges encountered during development.

4.2 Preliminary Design or Design (Model) Specification

4.2.1 System Design Overview

This chapter documents the technical dependencies and logical design based on in-depth system analysis of the project. The chapter demonstrates a four-tier banking system with on-chain lending flows and off-chain data and AI/ML layer integration and optional conversational AI agent support. All technical specifications and logical explanations of their interconnectivity, and how they work as a complete system, have been documented here sequentially and structurally. The tables contain concise information and the figures are provided for visualization of each complicated section.

4.2.2 High-Level Architecture

The system uses a 4-layer architecture: React for frontend, application services by Express.js and FastAPI, persistence by PostgreSQL, and Solidity for on-chain contracts. The layers are summarized in Table 4.3 and visually demonstrated in Figure 4.1, including how they interact.

Table 4.3: Four-layer architecture detail.

Layer	Technology	Key components	Responsibility
Presentation	React, TypeScript, WalletConnect / RainbowKit	Web UI, wallet signing, approver dashboard	User interaction and transaction initiation
Application services	Express.js, FastAPI	REST API, ML scoring service, MCP agent server, event listener	Business logic, AI/ML inference, agent tool calls, API orchestration
Persistence	PostgreSQL, Redis (cache)	Lending projections, audit logs, session and agent tables	Off-chain data storage and read models synced from chain events
On-chain	Solidity 0.8.20 (EVM)	World / National / Local Bank contracts, `LoanController`, deposit and pool modules	Authoritative banking state, RBAC, loan lifecycle, reserve rules

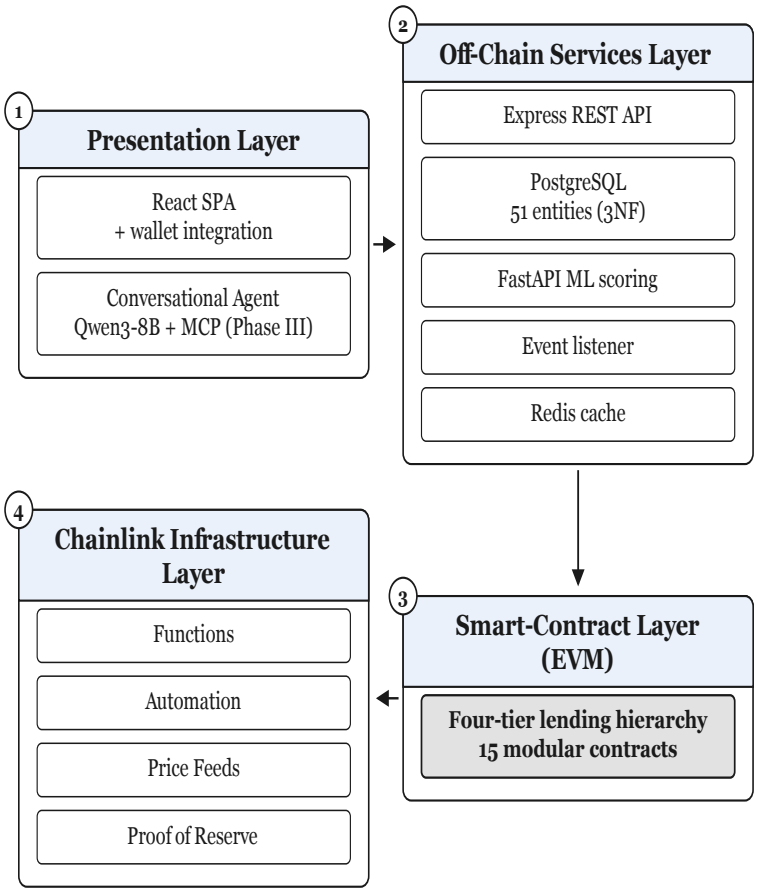

Figure 4.1: Four-layer architecture.

Figure 4.2 shows a generalized interaction of the different components. This diagram will be improved during project development as more features are integrated; it is the current baseline. The component diagram and extended modules are documented in Appendix A.2.

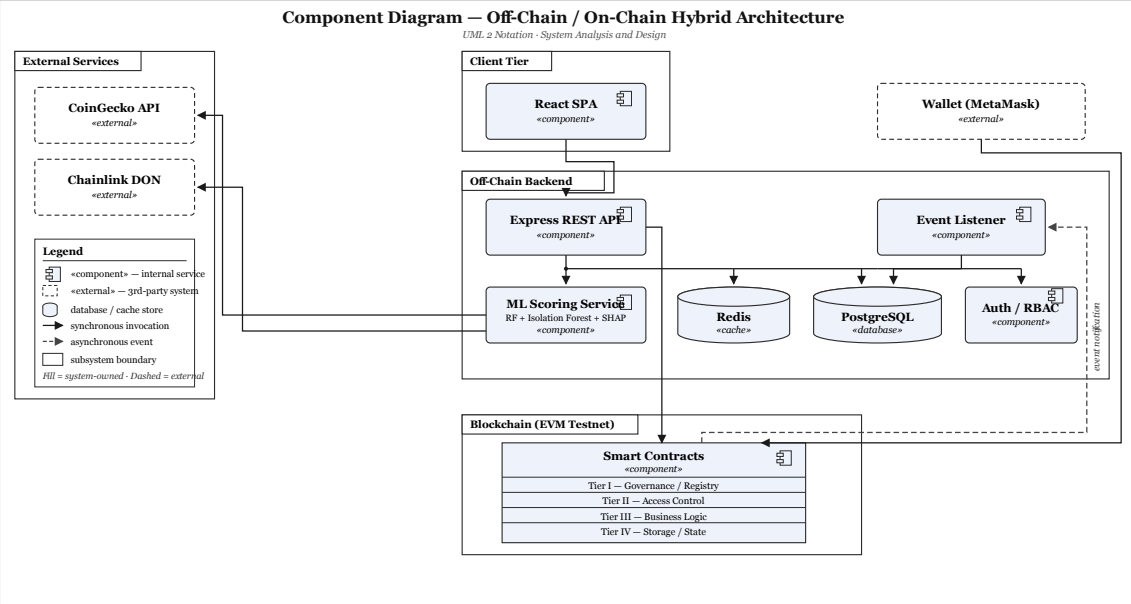

Figure 4.2: Component diagram.

Table 4.4: Smart contract inventory.

Contract	Module	Status	Primary responsibility
WorldBankReserve	Tier 1 core	Specified (Phase I)	Global reserve custody, national-bank registration, downward capital allocation.
NationalBank	Tier 2 core	Specified (Phase I)	Local-bank registration, upward borrowing, downward allocation.
LocalBank	Tier 3 core	Specified (Phase I)	Client onboarding, approver management, owns `LoanController`; `freezeAccount` (Phase II+).
LoanController	Lending state machine	Ph. I shell; Ph. II	Loan request, approval, disbursement, installment state machine (DT-II.01–03); deployed by `LocalBank`.
SavingsVault	Deposits	Specified (Phase II)	Variable-yield savings, reserve-gated withdrawals.
FixedDeposit	Deposits	Stub (Phase II+)	Term deposits with deterministic early-withdrawal penalty.
GroupLendingPool	Group credit	Specified (Phase II)	Solidarity-group formation, consent, mutual liability.
FXModule	Treasury / FX	Planned (Phase III)	Oracle-priced retail FX with disclosed spread.
InsuranceFund	Risk buffer	Ph. I (often II)	Default coverage buffer; Chainlink Proof of Reserve.
CurrentAccount	Payments	Stub (Phase II+)	Transactional accounts and atomic peer transfers.
InterBank-LendingPool	Multi-entity	Specified (Phase II)	Same-tier interbank liquidity pools per tier.
UpwardDeposit-Facility	Multi-entity	Specified (Phase II)	Upward surplus repatriation with yield spread.
SyndicatedLoan	Multi-entity	Planned (Phase III)	Multi-lender syndicate consent and disbursement.
TranchedPool	Multi-entity	Planned (Phase III)	Senior/junior tranching and waterfall recovery.
TreasurySwap	Multi-entity	Planned (Phase III)	Institutional oracle-priced asset swaps.
LiquidationEngine	Risk / liquidation	Planned (Phase III)	Health-factor monitoring; third-party liquidator bonus when $HF < 1.0$ (Section 4.2.8).
NettingEngine	Multi-entity	Future Work	ERD stubs only; Section 6.2

4.2.3 Blockchain Platform Selection

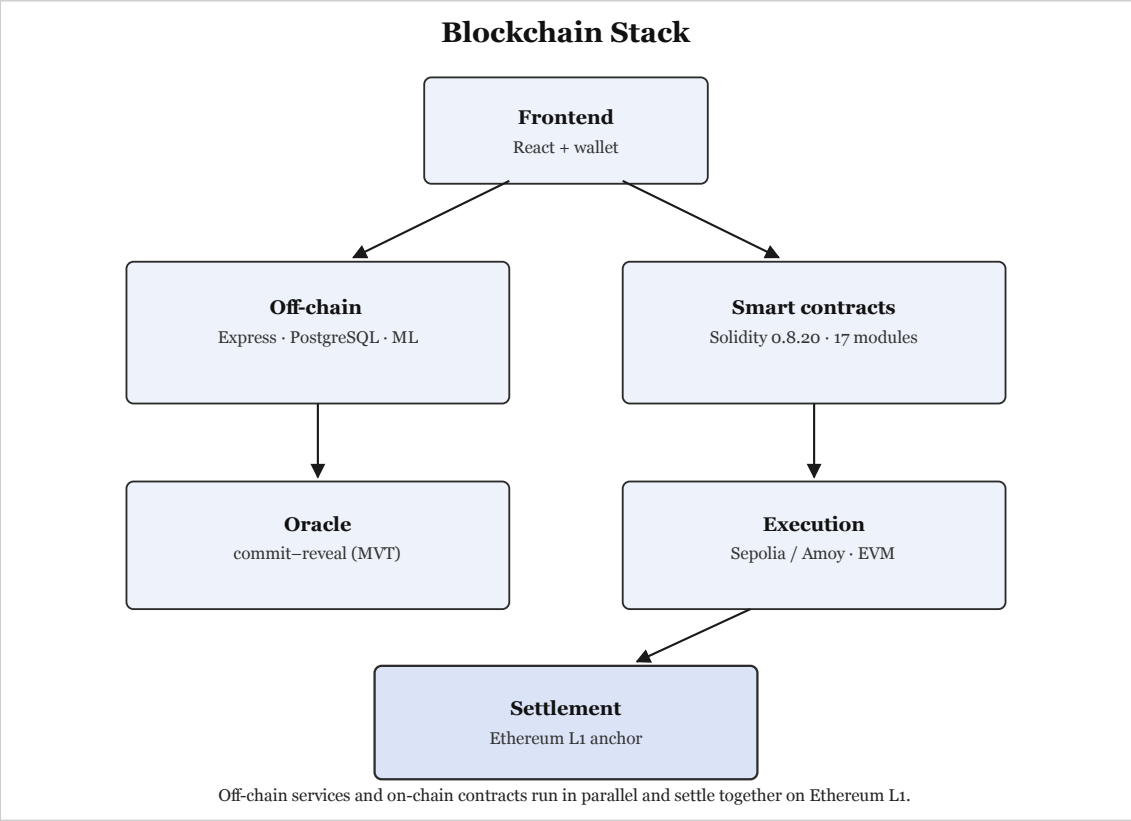

Figure 4.3: Blockchain stack.

Table 4.5: Blockchain platform selection.

Criterion	Selection	Justification
Platform	Ethereum Virtual Machine (EVM)	Largest EVM ecosystem; mature tooling.
Network	Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary prototype implementation)	Free public testnets; zkEVM evaluated, not default prototype chain.
Consensus	Public-testnet consensus (PoS/L1 testnet)	Stable testnet consensus; ZK settlement is a design preference only.
Smart contract language	Solidity 0.8.20	Industry standard; overflow-safe compiler; rich libraries.

Table 4.6: Blockchain platform selection (cont.).

Criterion	Selection	Justification
Gas cost	\$0.001–\$0.01 per transaction	Far below mainnet; viable for micro-loans.
Block finality	≈2 seconds	Fast retail UX; L1-anchored security.
Developer tooling	Hardhat + OpenZeppelin	Hardhat tests; audited OpenZeppelin libs.
Testnet availability	Amoy (Polygon PoS), Sepolia (Ethereum)	Free faucets; EVM-identical; no real funds.
Security model	Public-testnet security model	Testnet PoS; ZK settlement evaluated separately.
Migration path	EVM-compatible	Portable Solidity across EVM chains.

Transaction Verification and Consensus

The prototype is designed to be deployed on **Ethereum Sepolia** for the primary development route and **Polygon Amoy** as the secondary fallback (Tables 4.5 and 4.6). Both chains are free, development-friendly, and stable. The future target is deploying on **zkEVM**, as it is preferred for better privacy management and future-proofing. The loan lifecycle involves multiple on-chain state transitions for each client, with a complex verification process. For our chosen deployment plan with efficiency in mind, gas cost stays well below 0.01% of interest, and the whole validation process will be implemented in Phase IV (Section 3.7.2).

Oracle Architecture: Off-Chain AI to On-Chain Decision

The goal is to run server verification with all the necessary documents, authentication, and ML-layer processing workload on servers for off-chain tasks, and send only the necessary information on-chain without trusting a single party for decision-making. This task has been added to the MVT as a **commit–reveal relay**. Back-end servers process information and commit $h = \text{keccak256}(s \,\|\, \text{nonce})$ to `LoanController`, which then reveals score s and the nonce used in the decision-making process—one of the fundamental features of the project. The Solidity smart contract runs the verification–validation process on the hash and stores score s as an immutable object. This immutable record provides an auditable trail and prevents post-commit

score changes for security. This oracle architecture is designed to integrate these scoring features into the smart contract and exchange information on-chain for improved security and loan/lending approval decisions.
Figure 4.4 shows a summary version of the authentication flow, and Chainlink upgrades have been documented. Chainlink services have been organized and their relative production-role connections are shown in Table 4.7. The zero-cost demonstration substitutes are shown in Section 3.6 and Table 3.11.

Table 4.7: Chainlink upgrade paths.

Service	Production role	MVT / demo substitute
Chainlink Functions	DON-signed fetch of ML risk score from the off-chain service	Commit–reveal relay (authoritative at MVT scope)
Chainlink Automation	On-chain upkeep for overdue installments and interest accrual	Node.js cron calling `checkOverdueLoans()`
Price feeds	`FXModule` and fiat-equivalent UI via `AggregatorV3Interface`	`MockPriceFeed.sol` with fixed testnet answers
Proof of Reserve	External attestation of `WorldBankReserve` solvency	On-chain `getReserveSummary()` view; PoR job is Future Work

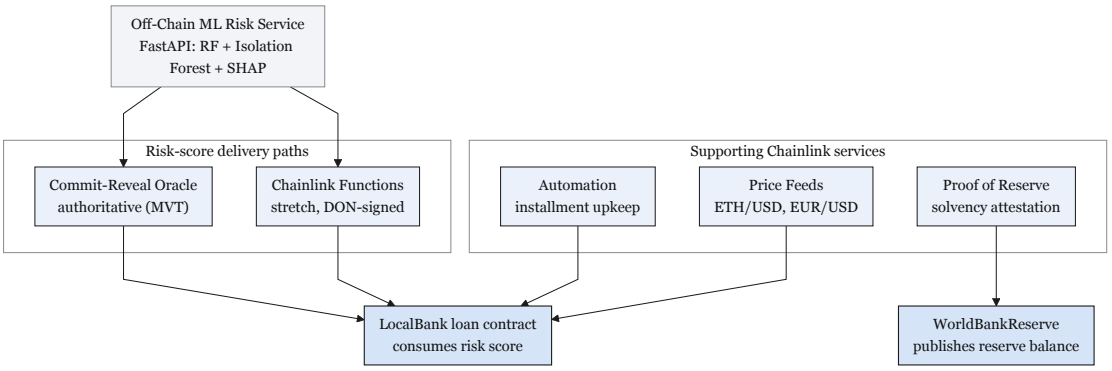

Figure 4.4: Oracle architecture.

4.2.4 Transaction Lifecycle, Hashing, and Cross-Layer State Propagation

This section traces a single client action end to end—from the moment it is entered in the React/TypeScript frontend to the point the resulting state change is confirmed on-chain and reflected back into the PostgreSQL-backed server state. It expands

on the commit–reveal hashing already introduced in Section 4.2.3 and the sequence diagrams in Section 4.2.10 with a layer-by-layer technical walkthrough.

Frontend-to-Gateway Request Flow

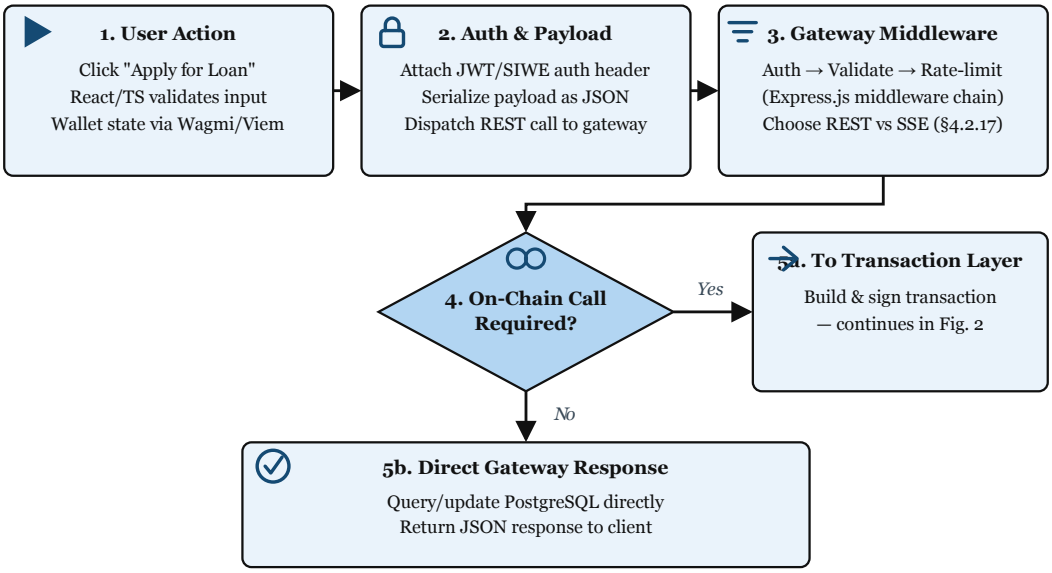

Figure 4.5: Frontend request flow and gateway sequencing.

API and Network Layer Sequencing

Transaction Construction, Hashing, and Signing

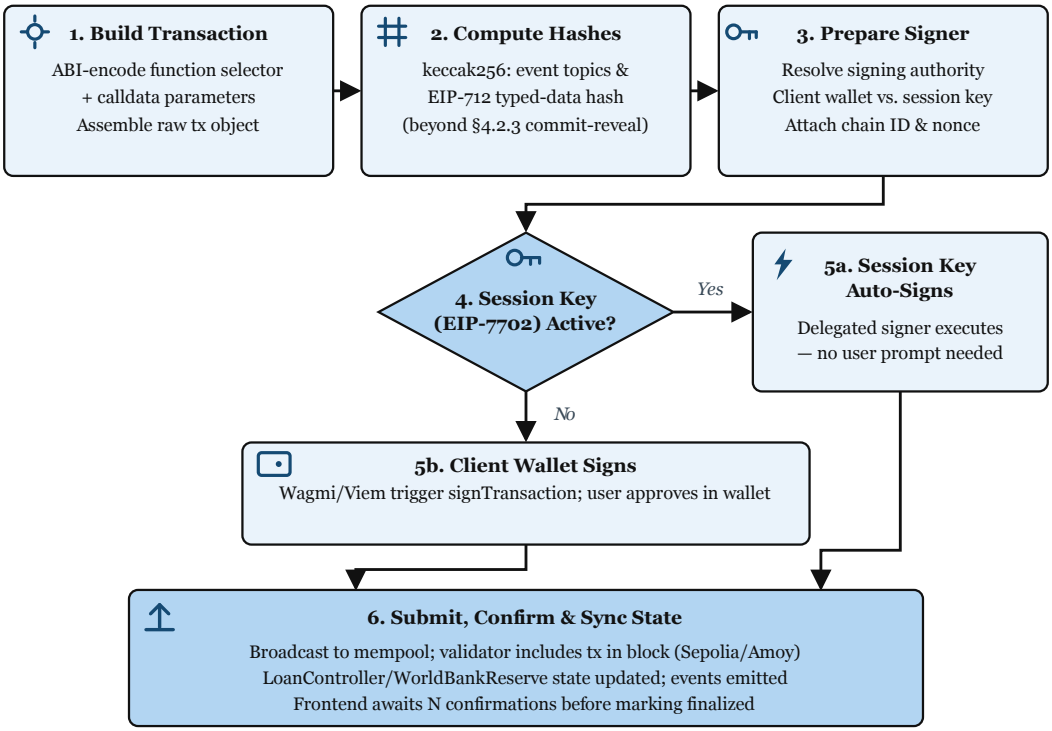

Figure 4.6: Transaction construction, hashing, and confirmation.

On-Chain State Transition and Block Confirmation

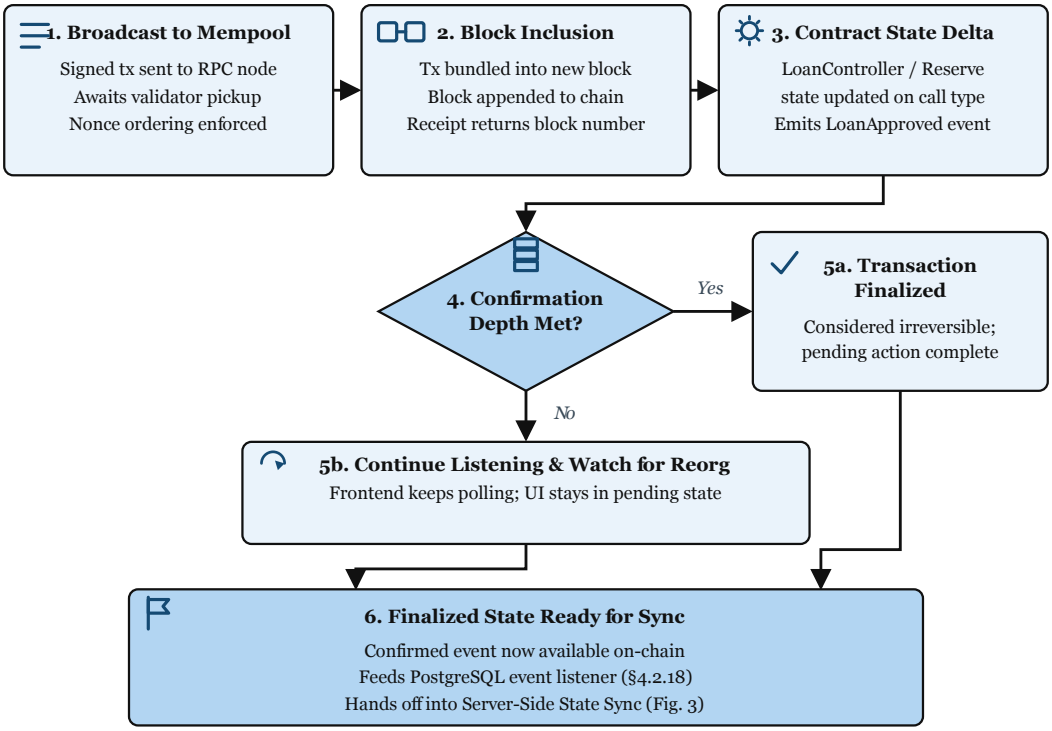

Figure 4.7: On-chain state transition and block confirmation.

Server-Side State Synchronization

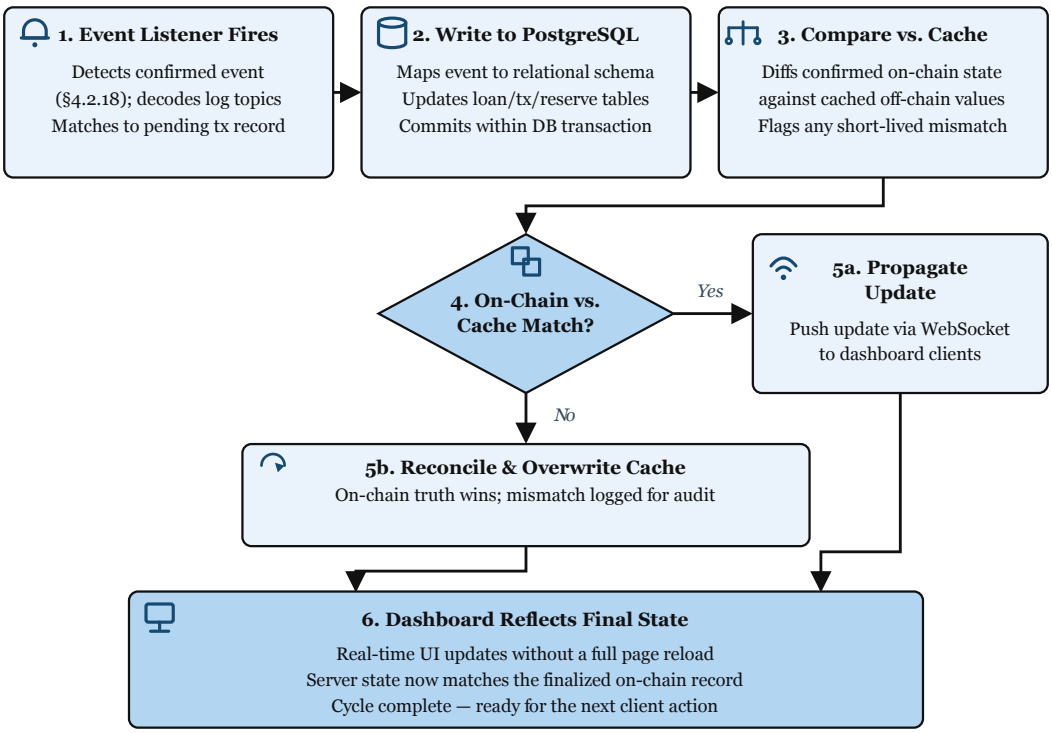

Figure 4.8: Server-side state synchronization.

4.2.5 Data Model and Database Design

This section describes and shows the full database schema. To incorporate all features of the project, the estimated entity count is **51** in Third Normal Form (3NF). For the prototype initial build, **16 core entities** flagged as Must have been specified clearly as essential, shown in Table 4.8. Remaining extension and stub relation groups are shown in Table 4.9. The full 51-entity relational ERD is moved to Appendix **??** (Figure **??**) due to its size, and this figure may be heavily modified during the development phases as some entities might need more relevant subsections and extensions. Figure 4.9 demonstrates the core entity relationships for the prototype; banking-product extensions and their FK anchors into core lending entities are summarised in Table 4.18 (Section 4.2.9).

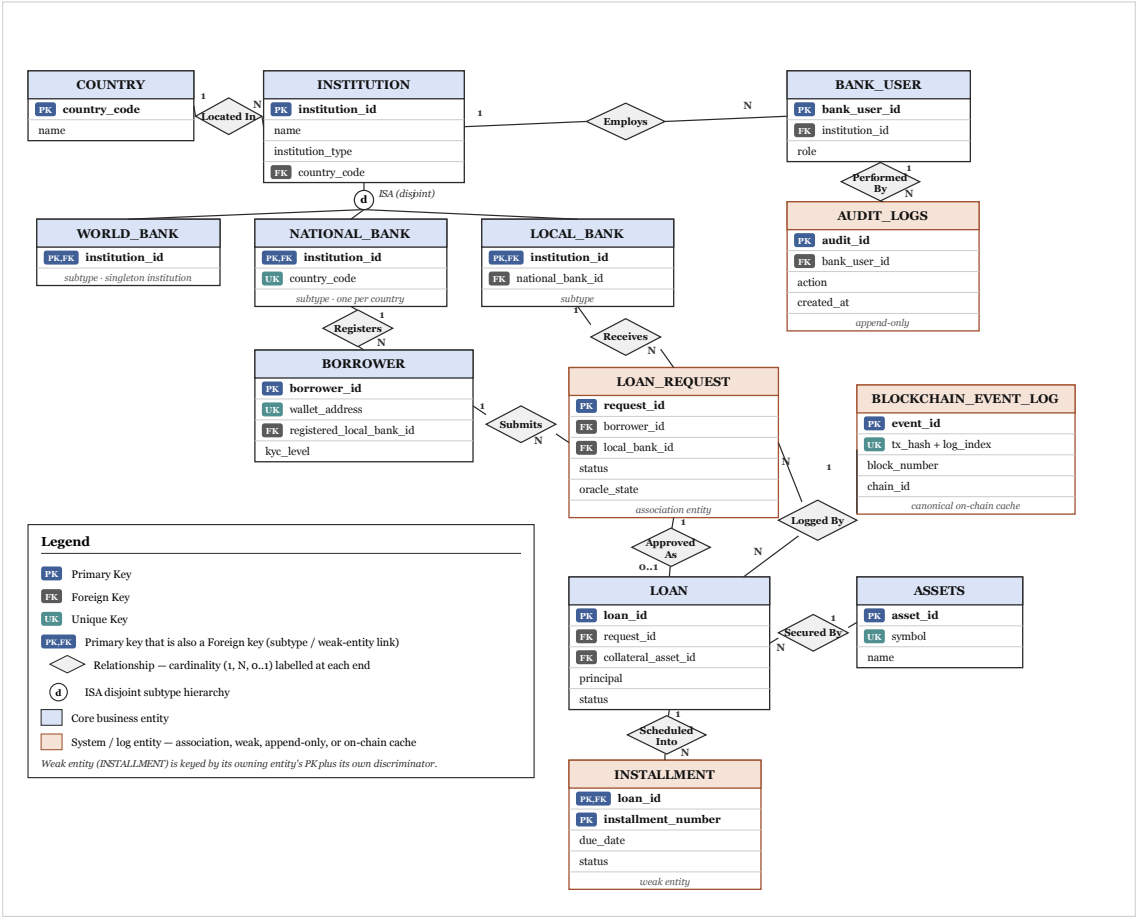

Figure 4.9: Core entity graph.

Entity Summary

Table 4.8: Core database entities.

Entity	Primary key	Role
Institution hierarchy (M1)		
INSTITUTION	`institution_id`	Shared supertype for World, National, and Local bank identities; enables single FK target for cross-tier operations.
COUNTRY	`country_code`	ISO 3166-1 reference; one `NATIONAL_BANK` per country (`UNIQUE`).
WORLD_BANK	`institution_id`	Global reserve subtype; singleton (`CHECK institution_id = 1`).
NATIONAL_BANK	`institution_id`	National reserve subtype; FK `country_code`, `parent_world_bank_id`.
LOCAL_BANK	`institution_id`	City-level lending institution; FK `national_bank_id`.
Actors and lending lifecycle (M1)		
BANK_USER	`bank_user_id`	Bank staff; exactly one parent institution enforced by CHECK (Table 4.12).
BORROWER	`borrower_id`	Retail client (UK: `wallet_address`); home bank via `registered_local_bank_id`; hash-only KYC fields.
LOAN_REQUEST	`request_id`	Application workflow (`status`, `oracle_state`); links borrower to local bank.
LOAN	`loan_id`	Active loan after disbursement; FK `loan_request_id`; on-chain linkage fields.
INSTALLMENT	`loan_id`, `installment_number`	Repayment schedule (weak entity on `LOAN`).
Sync, assets, and audit (M1)		
BLOCKCHAIN_EVENT_LOG	`event_id`	Canonical on-chain event cache (event-listener owned); FK target for lending tables.
ASSETS	`asset_id`	Collateral and loan asset registry (UK: `symbol`).
AUDIT_LOGS	`audit_id`	Append-only governance and compliance log (Express-owned).
Phase II–III extensions (selected)		
TRANSACTION	`transaction_id`	Financial ledger for retail and institutional flows.
BORROWING_LIMIT	`limit_id`	Off-chain projection of on-chain rolling borrow limits.
CREDIT_PASSPORT	`passport_id`	Event-listener projection of CreditPassport SBT state.
LOAN_RISK	`assessment_id`	Per-loan ML inference record; FK `model_id` → `MODEL REGISTRY`.

Table 4.9: Deferred entities.

Tier	Entities	Purpose
M2	CREDIT_PASSPORT_HISTORY, INCOME_PROOF, INTEREST_RATE_TIER	SBT history, Level 2+ document hashes, normalised kinked rates
M2	GROUP_CONSENT, LOAN_GROUP, GROUP_MEMBER	Group lending (if GroupLendingPool demo chosen)
M2	INTERBANK_LOAN, UPWARD_DEPOSIT, SAVINGS_ACCOUNT	One multi-entity PoC path
M3	MODEL_REGISTRY, SECURITY_EVENT_LOG	ML lineage and system-wide security events
M3	AGENT_ACTION_LOG, AGENT_CONVERSATION_TURN, SESSION_ANCESTOR	Agent audit and session lineage
Stub	FIXED_DEPOSIT, CURRENT_ACCOUNT, INSURANCE_FUND	Deposit and risk-buffer products
Stub	SYNDICATE, SYNDICATE_MEMBER, TRANCHED_POOL, TRANCHED_POOL_SUBSCRIPTION	Syndication and tranched lending
Stub	TREASURY_SWAP, NETTING_BATCH, NETTING_ENTRY	Treasury swap and netting (Future Work)
Future	CHAT_MESSAGE, INSTITUTION_MESSAGE, PROFILE_SETTING	UX messaging and preferences
Future	MARKET_DATA, CHAINLINK_PRICE_ROUND, ORACLE_HEARTBEAT_MONITOR	Oracle caches (mock at MVT)
Future	COLLATERAL_POSITION, KYC_VERIFICATION_REQUEST, NETTING_COORDINATOR_STATE	Liquidation, ZKP KYC, settlement coordinator
Future	CHAIN_REGISTRY, SESSION_KEY_PERMISSION	Phase IV platform configuration

Table 4.10: Audit and logging taxonomy.

Table	Writer	Scope	Typical contents
BLOCKCHAIN_-EVENT_LOG	Event listener	On-chain receipts	tx_hash, event_name, raw_data; FK target for LOAN_REQUEST, LOAN, INSTALLMENT
LOAN_RISK_-ASSESSMENT	FastAPI ML service	Per-loan inference	fraud_probability, shap_vector, FK model_id → MODEL_REGISTRY
AUDIT_LOGS	Express API	Governance / compliance	Travel Rule packets, role grants, manual approver actions (Section 4.2.11)
SECURITY_EVENT_-LOG	ML / monitoring	System-wide security	AML alerts, anomaly spikes, agent injection blocks
AGENT_ACTION_LOG	Express agent hooks	Agent write tools	tool_name, confirmation_turn_id, onchain_tx_hash

EER Constructs Applied

Table 4.11: EER constructs.

EER Construct	Applied To	Notes
Specialisation (disjoint)	BANK_USER → WorldBankUser, NationalBankUser, LocalBankUser	A bank user belongs to exactly one bank type (`world`, `national`, or `local`); enforced by CHECK constraint (Table 4.12).
Generalisation / specialisation	INSTITUTION → WORLD_BANK, NATIONAL_BANK, LOCAL_BANK	ID-based table-per-type inheritance: subtype PK = FK to `INSTITUTION.institution_id`; shared identity for cross-tier FKs (Table 4.8).
Weak entity	INSTALLMENT depends on LOAN	Existence and identity determined by parent loan.
Multi-valued attribute	INCOME_PROOF per BORROWER	Modelled as separate entity to maintain 1NF.
Aggregation	LOAN_RISK_ASSESSMENT ← LOAN_REQUEST; SECURITY_EVENT_LOG ← runtime monitors	Per-loan ML scores and system-wide security events are separate append-only relations, not a conflated aggregate.
Association entity	LOAN_REQUEST links BORROWER + LOCAL_BANK	Captures many-to-many borrowing relationships; distinct from `BORROWER.registered_local_bank_id` (home bank at onboarding).
Participation constraint	BORROWER.`registered_local_bank_id` NOT NULL at onboarding	Home-Local Bank is mandatory; `LOAN_REQUEST` participation for lending services enforced at application layer.
Append-only table (policy)	AGENT_ACTION_LOG, AUDIT_LOGS, SECURITY_EVENT_LOG	DB-level INSERT-only role + Row-Level Security policy enforces that no UPDATE or DELETE is permitted on audit records.

Relational Integrity Constraints

Table 4.12: Relational integrity constraints.

Constraint Type	Examples
Primary Key	One surrogate or composite key per entity (Tables 4.8, 4.9): e.g. `institution_id`, `borrower_id`, `request_id`, (`loan_id`, `installment_number`), ...
Foreign Key	Tier subtypes → `INSTITUTION`; `BORROWER.registered_local_bank_id` → `LOCAL_BANK`; `LOAN_RISK_ASSESSMENT` → `LOAN_REQUEST`, `MODEL_REGISTRY`; multi-entity stubs → `INSTITUTION.institution_id`.
UNIQUE	`BORROWER.wallet_address`; `NATIONAL_BANK(country_code)`; `INSTITUTION.contract_address`; `LOAN_REQUEST.blockchain_tx_hash`; `BORROWING_LIMIT.borrower_id`; `ASSETS.symbol`; `BLOCKCHAIN_EVENT_LOG.tx_hash`; partial unique on active `INTEREST_RATE_TIER(bank_tier_type)`.
CHECK	`WORLD_BANK`: singleton (`institution_id = 1`). `BANK_USER`: exactly one parent FK matching `bank_type`. `TRANSACTION`: at least one of `borrower_id`, `origin_institution_id`, `counterparty_institution_id` non-null.
NOT NULL	`INSTITUTION.name`, `INSTITUTION.institution_type`; `BORROWER.wallet_address`, `registered_local_bank_id`, `kyc_level`; `status`; `oracle_state` (default `NONE`).
APPEND-ONLY	`AGENT_ACTION_LOG`, `CREDIT_PASSPORT_HISTORY`, `SECURITY_EVENT_LOG`, `AUDIT_LOGS`: INSERT-only role + RLS; no UPDATE/DELETE.
PROJECTION (RLS)	`CREDIT_PASSPORT`, `BORROWING_LIMIT`, `COLLATERAL_POSITION`: `event_listener` role only may INSERT/UPDATE; Express API SELECT-only.
FOREIGN KEY	`LOAN` → `ASSETS` (collateral, loan asset); `AGENT_ACTION_LOG` → `SESSIONS`, `AGENT_CONVERSATION_TURN`; `LOAN.disbursement_event_log_id` → `BLOCKCHAIN_EVENT_LOG`.

4.2.6 On-Chain and Off-Chain Data Partitioning

Table 4.13: Data partitioning.

Data Category	Storage	Rationale
Reserve balances, loan requests, approval/rejection events, repayment transactions	On-chain	Immutability, public auditability, trustless verification.
User profiles, income verification documents, chat messages, loan risk assessments, security event logs	Off-chain (database)	Data privacy, query flexibility, storage cost optimisation.
Borrowing limit computations	Off-chain with on-chain enforcement	Complex temporal aggregation; results committed as on-chain constraints via `LocalBank.updateBorrowingLimit(borrowerId, newLimit)` (see sync protocol below).
Cryptocurrency market data (non-Chainlink)	Off-chain (`MARKET_DATA`)	CoinGecko and other fallback feeds; high-frequency updates; external API dependency.
Chainlink oracle price data	Off-chain (`CHAINLINK_PRICE_ROUND`)	Each `latestRoundData()` round cached with `answered_in_round` for staleness detection; on-chain `AggregatorV3Interface` is authoritative. `MARKET_DATA` does *not* store Chainlink rounds.
Agent action execution log	Off-chain (append-only DB)	Immutable audit trail of every agent write-tool call, linked to on-chain tx hash; not stored on-chain to save gas.
Session key material	Off-chain (`sessions` table)	EIP-7702 session key hash, scope JSON, and TTL stored off-chain; the scope restrictions are enforced on-chain by the session key contract at signing time.
Session lineage chains (SESSIONS, message history, compression summaries)	Off-chain (PostgreSQL)	Full transcript history is too large for on-chain storage. The lineage chain enables searchable long-term memory without gas cost. `AGENT_`

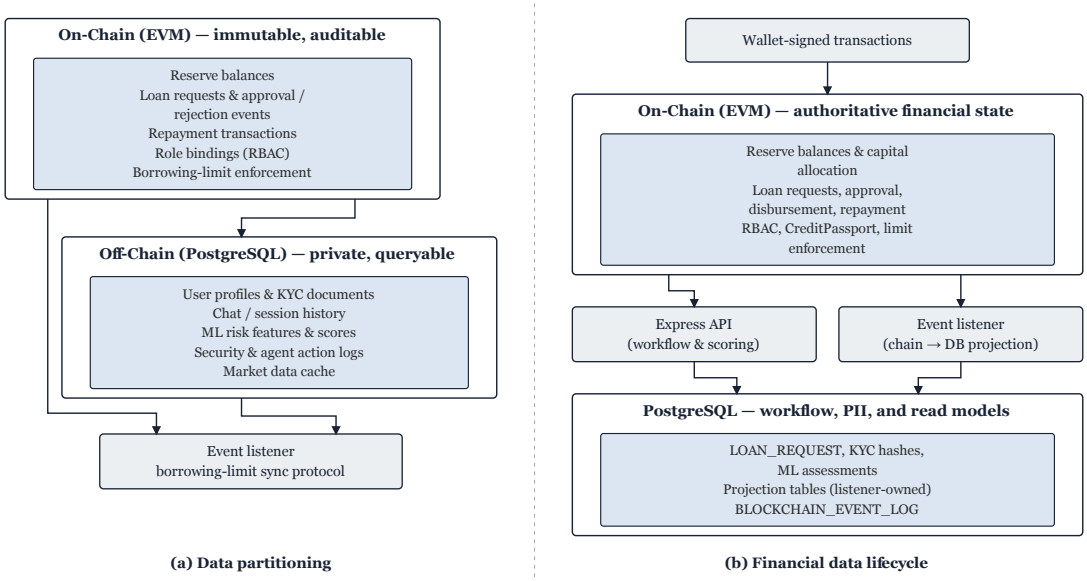

Figure 4.10: Data partitioning and financial data lifecycle.

4.2.7 User Taxonomy and Onboarding Flows

Table 4.14: User taxonomy.

User Type	Tier	KYC Level	Wallet Type	Data Residence
Anonymous Visitor	—	None	None	Session only (Redis)
Registered Retail User	Tier 4	Level 1 (gov. ID + selfie)	MetaMask or optional ERC-4337	Off-chain primary; on-chain role only
Group Client	Tier 4	Level 1 + group verification	ERC-4337 abstracted	Off-chain + on-chain group contract
Local Bank Operator	Tier 3	Full institutional KYC	MetaMask (institutional)	Off-chain + on-chain role binding
Local Bank Approver	Tier 3	Full institutional + background check	MetaMask + Safe co-signer	Off-chain + on-chain role binding
National Bank Admin	Tier 2	Full institutional + regulatory license	Safe multisig mandatory	Off-chain + on-chain role binding
World Bank Admin (Governance)	Tier 1	Founding team + supervisor attestation	Safe 3-of-5 multisig	On-chain governance only

Table 4.15: Actor taxonomy.

A1a	Retail Client (pre-KYC)	Browsing only; no banking transactions.
A1b	Retail Client (KYC Tier 1)	Bronze/Silver credit tier; small loans up to the KYC-1 cap.
A1c	Retail Client (KYC Tier 2)	Gold/Platinum/Diamond; larger limits, group lending.
A2	Local Bank Admin (Approver)	Loan approver; risk officer; reviews AML alerts.
A3	National Bank Admin	Capital allocator; compliance officer; SAR review; freeze authority.
A4	World Bank Admin (Governance)	Parameter governor; system operator via multi-sig.
A5	AI Agent (retail)	Phase III Must deliverable; read tools always permitted; write tools require explicit human confirmation gate before execution.
Secondary Actors (external systems or authorities)		
A6	Regulatory Authority	Read-only audit access via encrypted data package.
A7	Chainlink DON	Oracle, price feeds, automation.
A8	Blockchain Validator	Polygon zkEVM validity proof generation.
A9	External Auditor	Chainlink PoR verification.

Table 4.16: Actor permission matrix.

Action	Client	LB Admin	NB Admin	WB Admin	AI Agent
View own loan status	YES	YES	YES	YES	READ
Submit loan application	YES	NO	NO	NO	WRITE
Pay installment	YES	NO	NO	NO	WRITE
Submit KYC documents	YES	NO	NO	NO	NO
Approve loan application	NO	YES	NO	NO	NO
Reject loan application	NO	YES	NO	NO	NO
Freeze client account	NO	YES	YES	YES	NO
Set borrowing rate	NO	NO	YES	YES	NO
Change reserve ratio	NO	NO	NO	YES	NO
Upgrade LB borrowing cap	NO	NO	YES	YES	NO
View audit logs (all)	NO	YES	YES	YES	NO
Generate SAR report	NO	YES	YES	YES	NO
View reserve summary (public)	YES	YES	YES	YES	READ

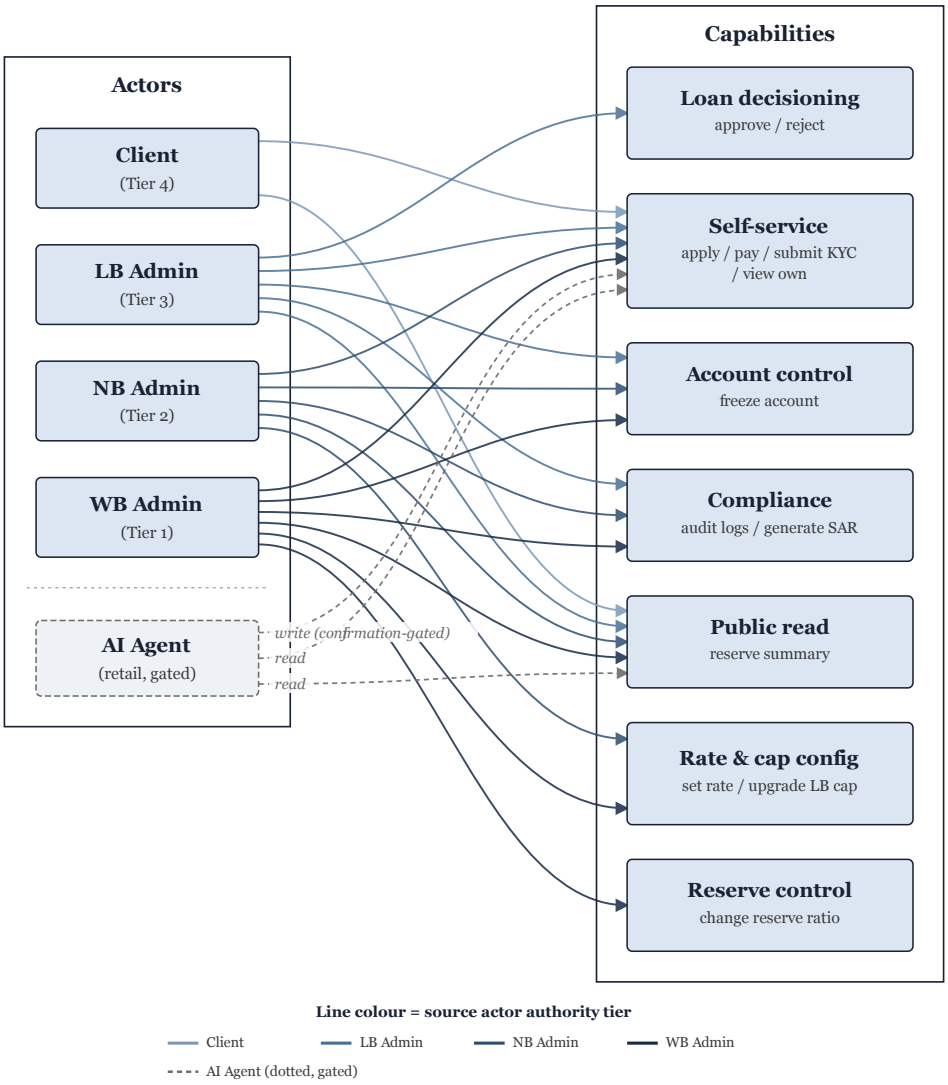

Figure 4.11: Permission matrix diagram.

4.2.8 Banking Modules

Kinked Interest Rates

This feature is integrated into the financial flow of the banking system (Compound/Aave pattern [42]) with optimal utilization $U^* = 80\%$. As shown in Figure 4.12 below, the interest rate jumps when loan pool utilization reaches the 80% margin. This method has been utilized to avoid providing excessive loans when the loan pool is at a critical level for maintaining banking functions. The model encourages repayment to keep a low rate for borrowing cryptocurrencies.

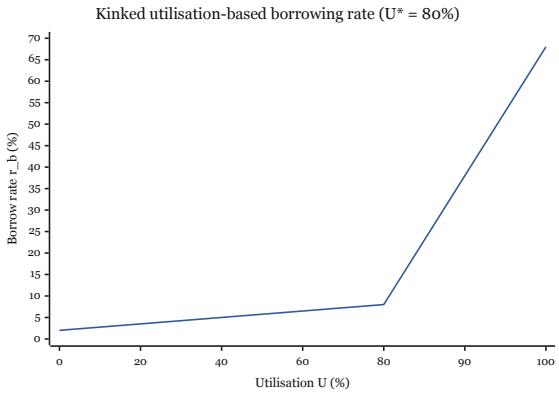

Figure 4.12: Kinked borrowing rate.

Liquidation Engine

The `LiquidationEngine` is designed to monitor health factor based on collateral given by individual clients for loans and to provide bonuses of 5–8% to liquidators when the HF score is below 1.0 ($HF < 1.0$). Pool-level shared credit is utilized for groups. When a retail client's collateral is proven to score $HF < 1.0$, their SBT is downgraded and lending score is reduced instead of collateral seizure. A summary is given in the following figure (Figure 4.13).

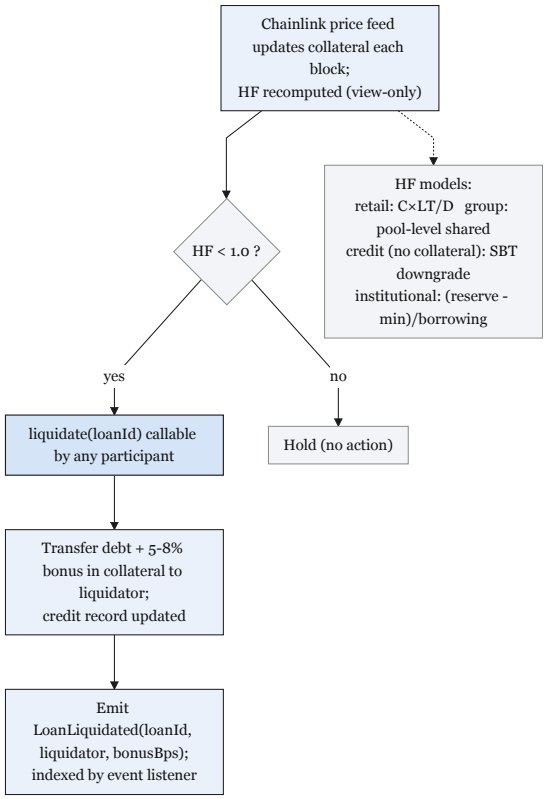

Figure 4.13: Liquidation engine.

Savings and Fixed Deposits

`FixedDeposit` (ERC-7540) is designed to lock currency for a set period of time. `SavingsVault` (ERC-4626) is designed to deposit stablecoins like normal savings accounts. NetInterest = InterestCollected − DepositorYield − InsuranceAllocation is utilized to split interest among depositor yield, `InsuranceFund`, and protocol revenue; details are demonstrated in Figure 4.14 below.

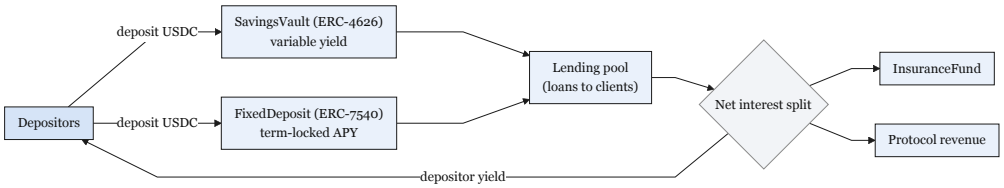

Figure 4.14: Savings vault.

Credit Passport (SBT)

SBT involves credit score (1–1000), risk tier (Bronze, Silver, Gold, Platinum, Diamond), and repayment history. A client is rewarded by upgrading SBT tier when their financial exchanges are remarkable or noticeably good, and punished when they cannot repay loans or do not follow the required rules to maintain a healthy account in the system. Full tier details are shown in Figure 4.15 and Table 4.17.

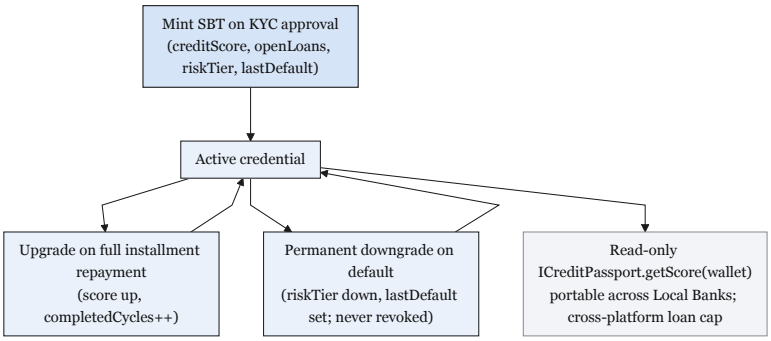

Figure 4.15: Credit Passport lifecycle.

Table 4.17: Credit Passport tiers.

Tier	Label	Score Range	Max Loan (USDC)	Interest Modifier
1	Bronze	0–299	50	Base rate
2	Silver	300–549	250	Base − 0.5%
3	Gold	550–749	1,000	Base − 1.0%
4	Platinum	750–899	5,000	Base − 1.5%
5	Diamond	900–1000	25,000	Base − 2.0%

4.2.9 Multi-Entity and Cross-Tier Capital Operations

Besides lending down to clients, banks in the hierarchy also need to move capital sideways between peers at the same tier, upwards to return surplus to a parent bank, and together to share large loans. The subsections below describe five on-chain mechanisms for these flows; batch settlement netting (`NettingEngine`) is left for future work (Section 6.2).

Same-Tier Lending (InterBankLendingPool)

For each tier there is a separate pool for lending facilities among banks of the same tier. Banks with an active role can contribute to or receive lending facilities from same-tier banks instead of from the parent tier. One bank can lend to another bank based on lending limits and controlled interest rates, for a flexible duration.

$$r_{\text{IB}}(U) \leq r_{\text{downward}}(U) - \delta$$

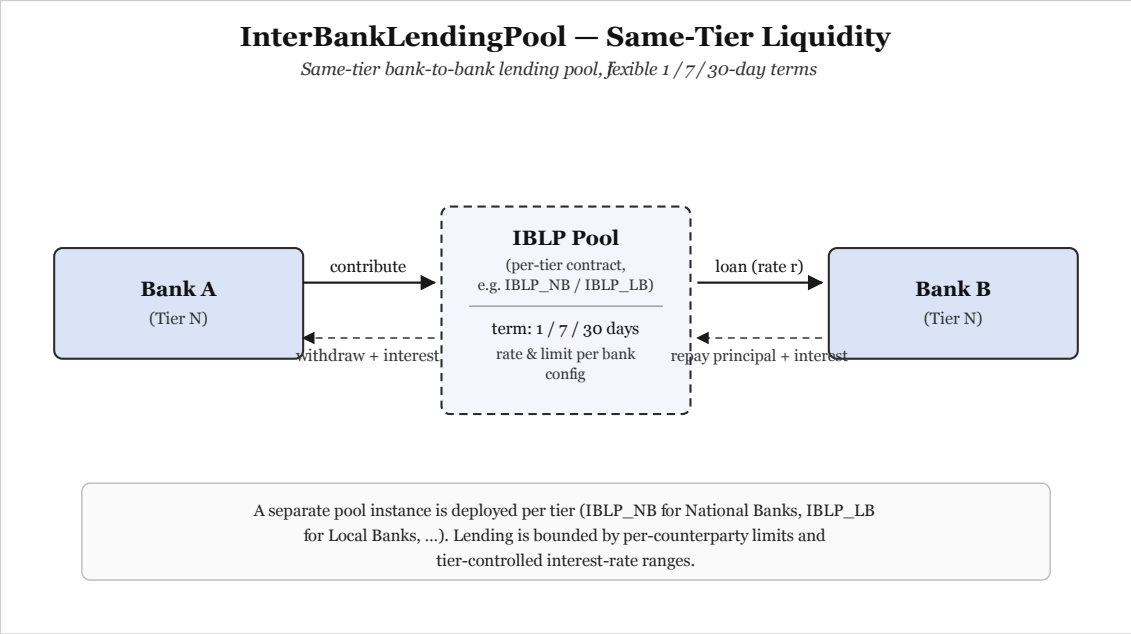

Figure 4.16: InterBankLendingPool.

Surplus Repatriation (UpwardDepositFacility)

When a bank has more reserves than the minimum requirement and is able to fund extra to the parent banks, a lower-tier bank is able to fund upper-tier banks through the `UpwardDepositFacility` and earn interest on the funded amount.

$$r_{\text{up}} < r_{\text{down}} - \delta$$

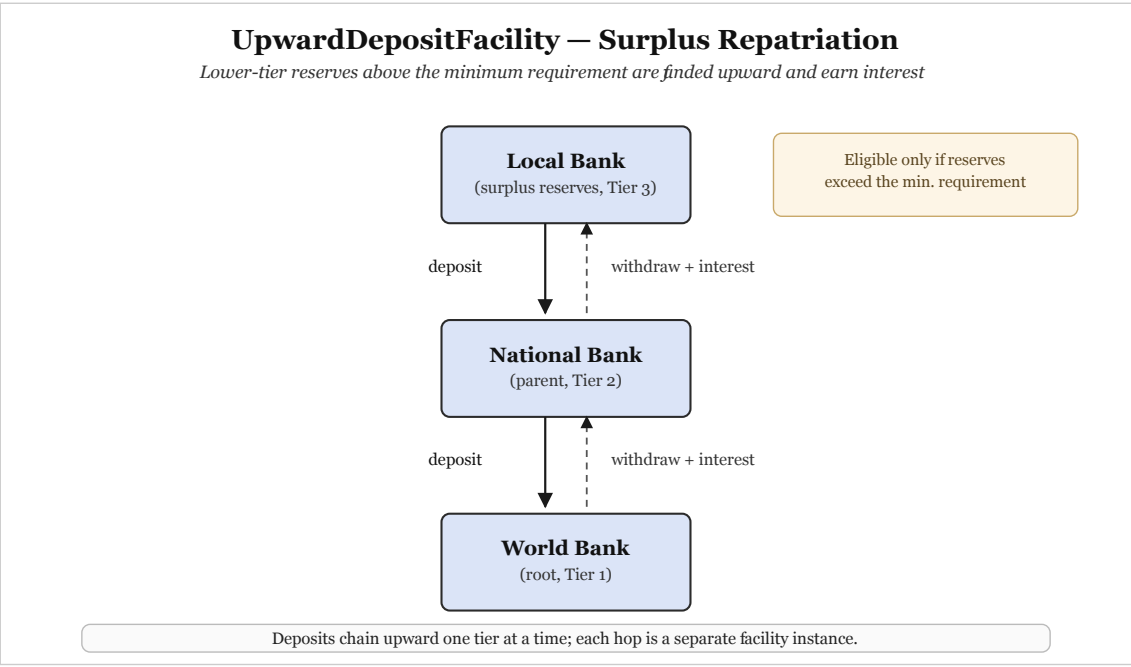

Figure 4.17: UpwardDepositFacility.

SyndicatedLoan

Some loans can be funded by multiple banks together, and multiple banks can request a single loan together from a parent bank. Primarily, one bank is registered for the loan and other banks are considered co-lender banks. The interest rate and other contributions are distributed based on the invested or contributed amount by each bank.

$$\text{Share}_i = \frac{C_i}{\sum_j C_j}, \qquad \text{Payout}_i = \text{Total} \times \text{Share}_i$$

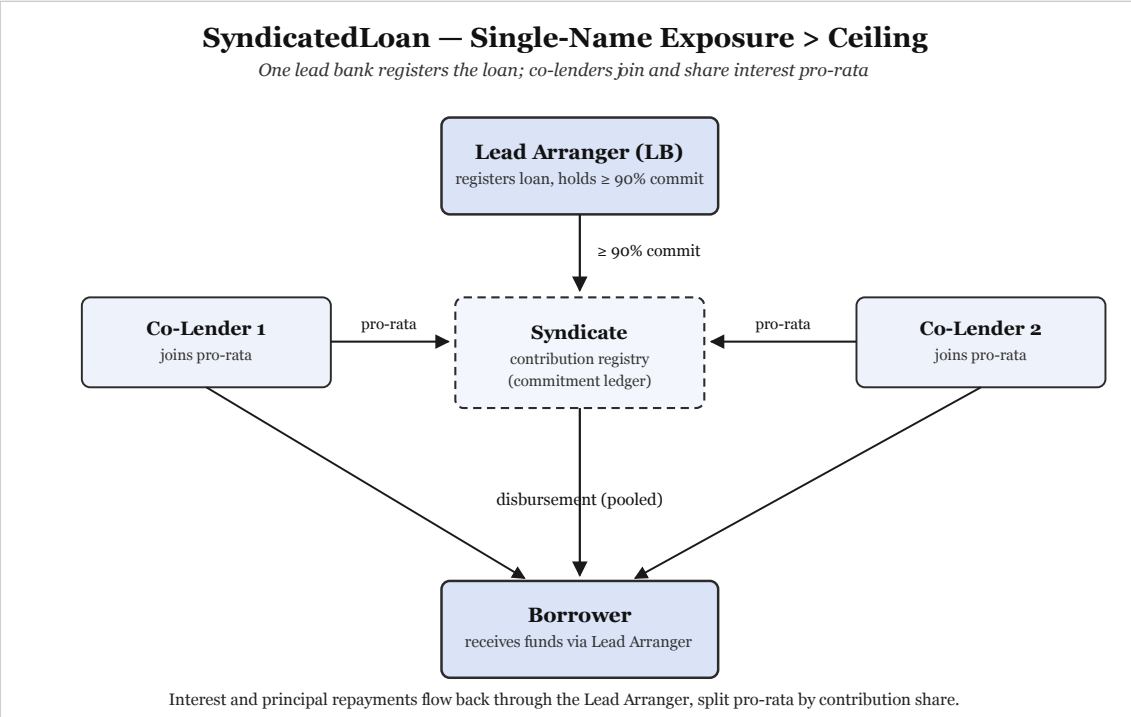

Figure 4.18: SyndicatedLoan.

TranchedPool

Banks can choose the risk level of their invested amount by signing up as a senior or junior tranche. The senior holder gets low risk and low return, and the junior holder gets high risk and high return. The senior holder receives the first payment for interest, so their advantage shows in low risk of returns, but junior holders get high repayments, with the risk of high loss too. To maintain this classification, the junior share is limited to 10% to 30% of the pool.

$$0.10 \leq \frac{J}{J+S} \leq 0.30$$

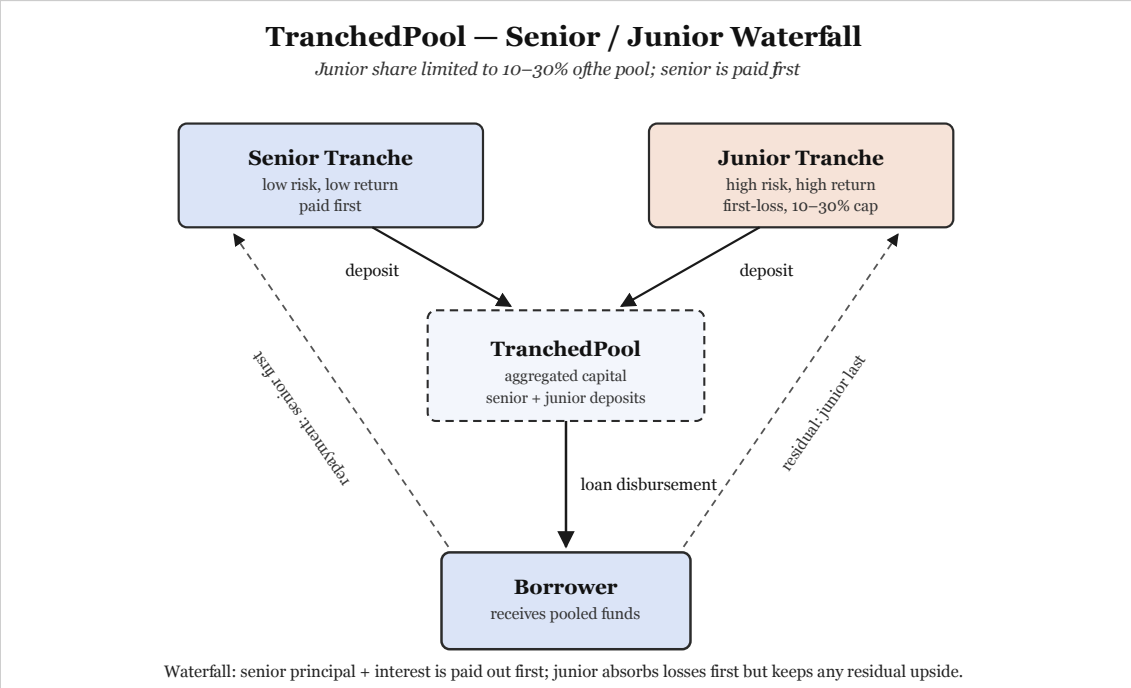

Figure 4.19: TranchedPool.

TreasurySwap

A bank can swap assets with other banks based on asset value from the Chainlink feeds. Banks are only allowed to swap reserves that meet the minimum reserve ratio of 20%. A swap of asset value over 1M USDC requires 2 signatures from a multisig before it is submitted for processing.

$$\text{Amount}_{\text{to}} = \text{Amount}_{\text{from}} \times \frac{P_{\text{from}}}{P_{\text{to}}} \times (1 - \text{spread})$$

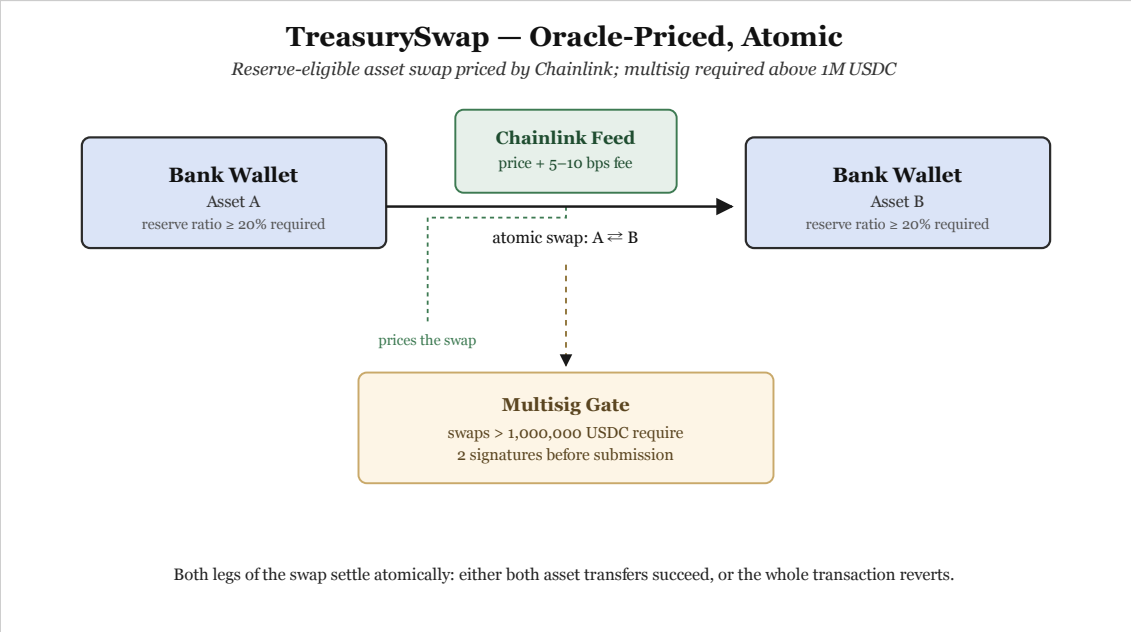

Figure 4.20: TreasurySwap.

NettingEngine (Future Work)

The `NettingEngine` works as a mechanism for partial settlement of `InterBankLendingPool` loans and `TreasurySwap` obligations. It works by batching large payments into smaller net payments.

$$\text{Net}_i = \sum_j O_{ij} - \sum_j O_{ji}$$

Details are shown in the figure below.

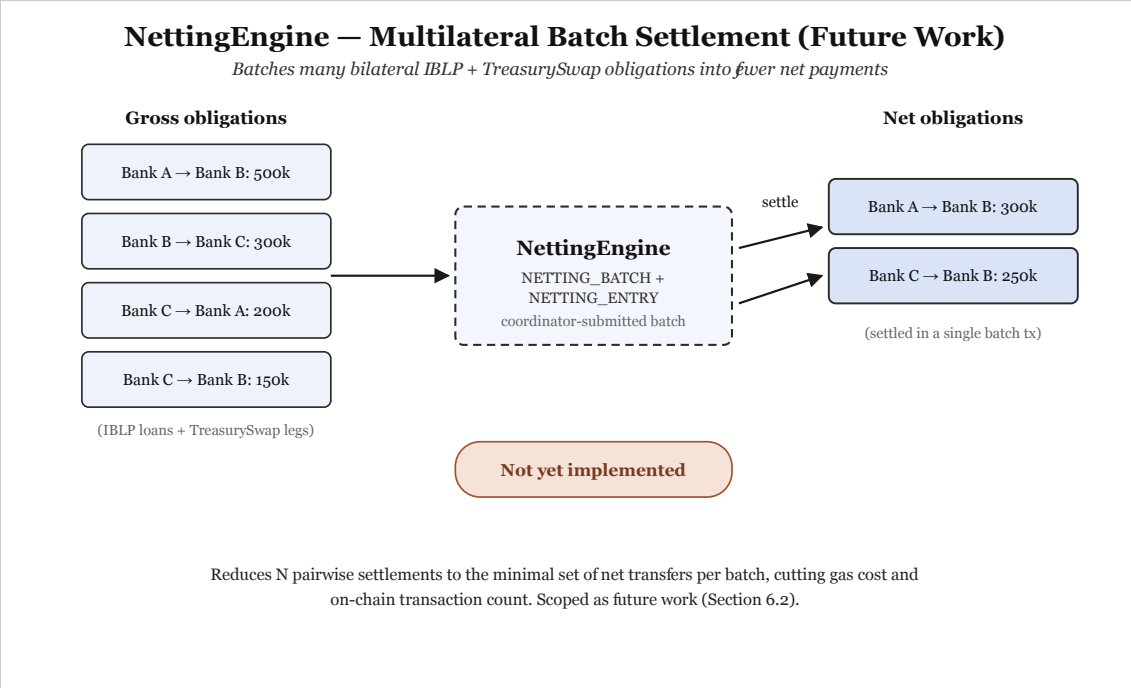

Figure 4.21: NettingEngine.

Database, Phase, and Contract Consistency for Multi-Entity Operations

Table 4.18 lists how the multi-entity tables link back to the core lending schema through foreign keys.

Table 4.18: FK anchors.

Entity	Foreign Keys into Core Schema
SAVINGS_ACCOUNT, FIXED_DEPOSIT, CURRENT_ACCOUNT	borrower_id → BORROWER; local_bank_id → LOCAL_BANK(institution_id)
LOAN_GROUP	local_bank_id → LOCAL_BANK(institution_id)
GROUP_MEMBER	group_id → LOAN_GROUP; borrower_id → BORROWER
INSURANCE_FUND	local_bank_id → LOCAL_BANK(institution_id)
INTERBANK_LOAN	lender_institution_id, borrower_institution_id → INSTITUTION; tier-type CHECK per IBLP instance
UPWARD_DEPOSIT	depositor_institution_id, parent_institution_id → INSTITUTION; adjacent-tier CHECK
SYNDICATE	lead_institution_id → INSTITUTION; borrower_id → BORROWER
SYNDICATE_MEMBER	syndicate_id → SYNDICATE; member_institution_id → INSTITUTION
TRANCHED_POOL	sponsor_institution_id → INSTITUTION
TRANCHED_POOL_SUBSCRIPTION	pool_id → TRANCHED_POOL; subscriber_id → BANK_USER
TREASURY_SWAP	initiator_institution_id → INSTITUTION; from_asset_id, to_asset_id → ASSETS
NETTING_BATCH	coordinator_id → BANK_USER
NETTING_ENTRY	batch_id → NETTING_BATCH; participant_institution_id → INSTITUTION
INSTITUTION_MESSAGE	sender_institution_id, receiver_institution_id → INSTITUTION
LOAN_RISK_ASSESSMENT	loan_request_id → LOAN_REQUEST; model_id → MODEL_REGISTRY
GROUP_CONSENT	loan_request_id → LOAN_REQUEST; member_id → GROUP_MEMBER
COLLATERAL_POSITION	borrower_id → BORROWER; loan_id → LOAN; asset_id → ASSETS
BLOCKCHAIN_EVENT_LOG	Referenced by: LOAN_REQUEST.submission_event_log_id, approval_event_log_

4.2.10 System Modeling

This section uses standard system-analysis diagrams to show who uses the platform, what they do, how data moves, and how the main parts connect. The figures below complement the technical design in earlier sections; they are not repeated at contract or table level here.

Use Case Diagram

Figure 4.22 maps the main actors—borrowers, bank staff, regulators, and system services—to the actions each can perform on the platform.

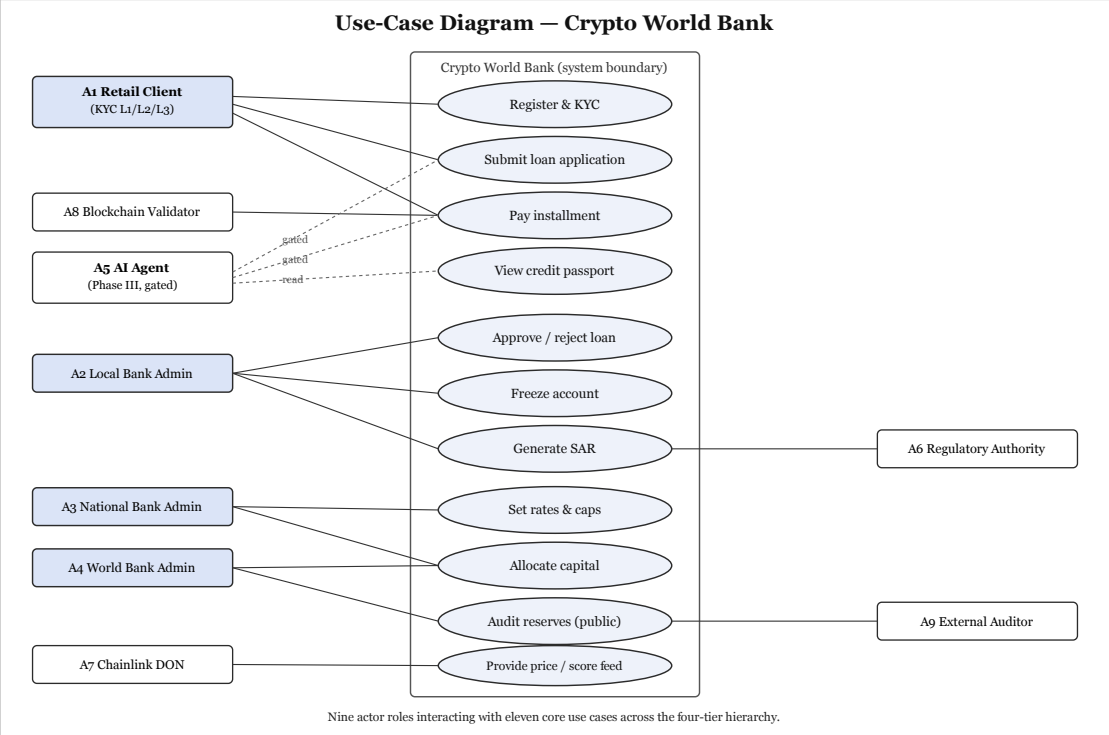

Figure 4.22: Use-case diagram.

Activity Diagrams

Figures 4.23–4.25 walk through three everyday flows step by step: applying for and repaying a loan, signing up and verifying identity, and routine banking tasks such as checking balances or using the chat assistant.

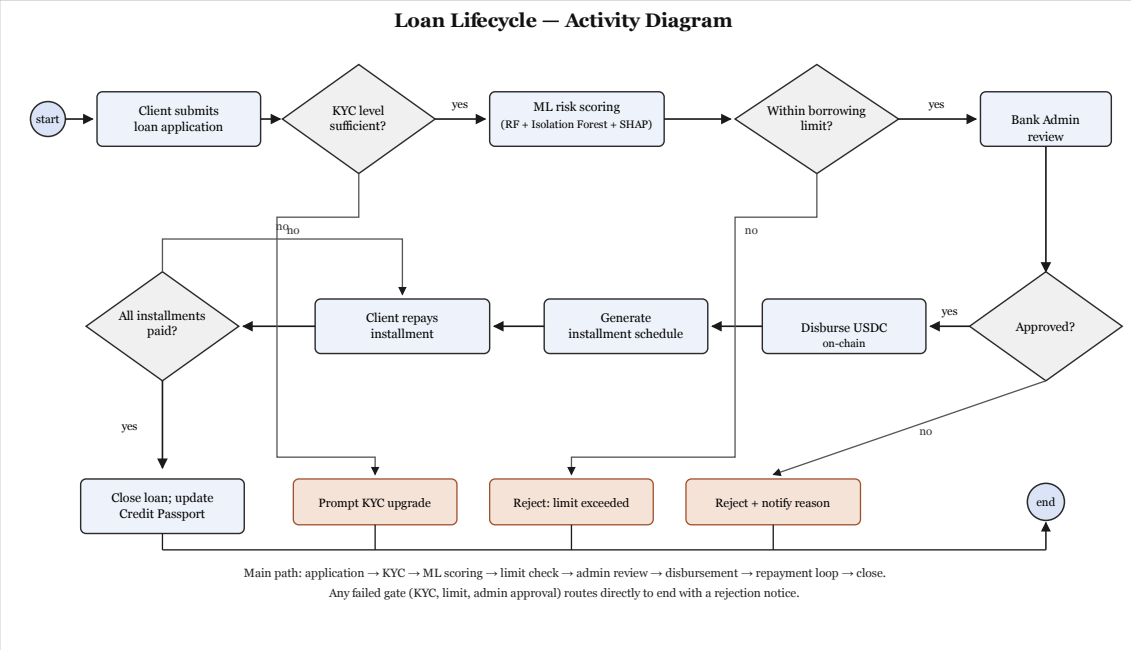

Figure 4.23: Loan lifecycle activity.

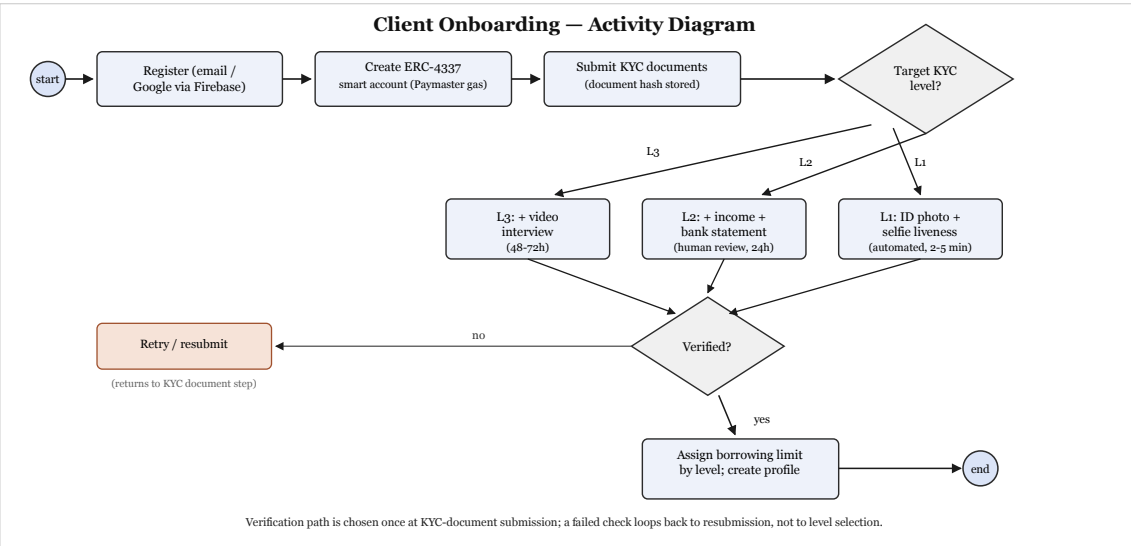

Figure 4.24: Onboarding activity.

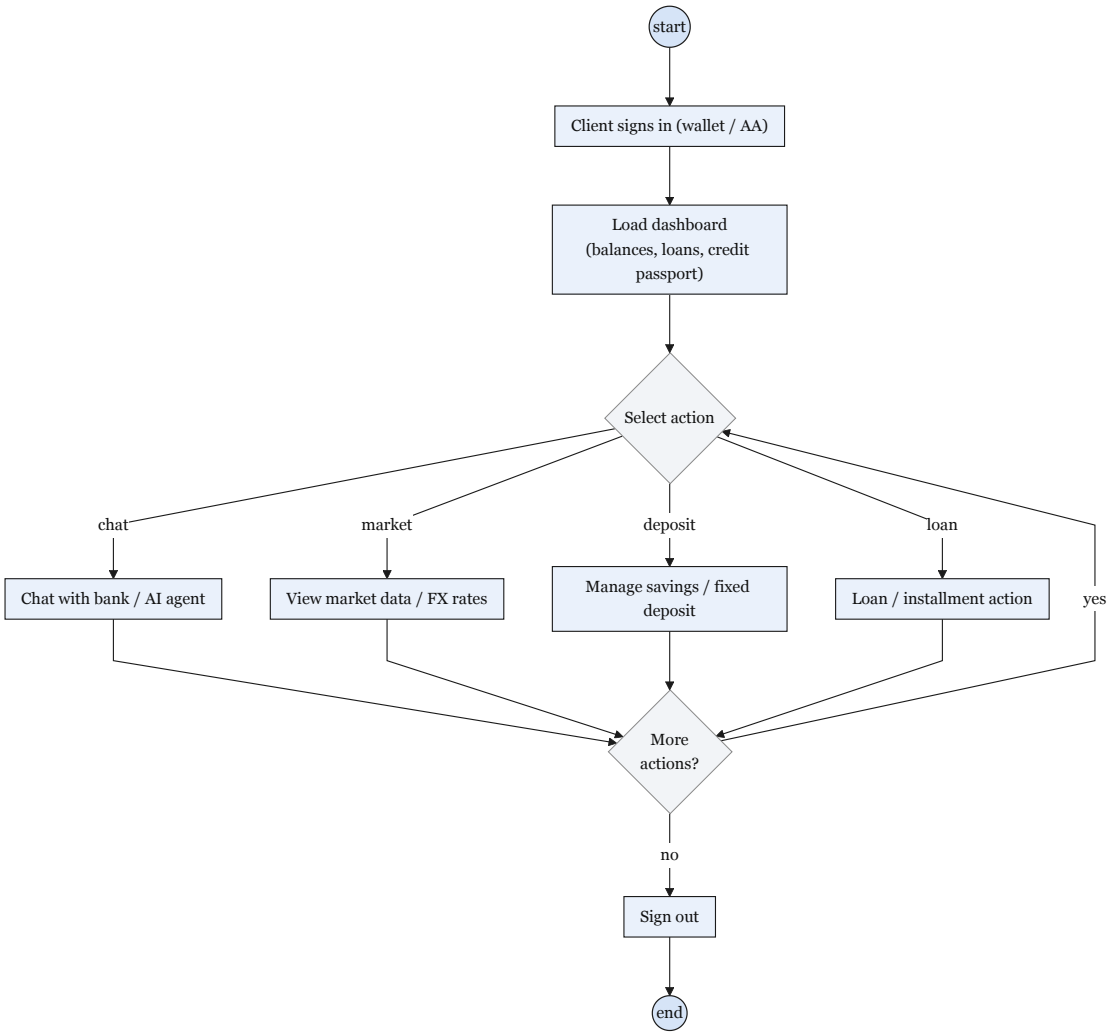

Figure 4.25: Banking session activity.

Data Flow Diagrams

Figures 4.26 and 4.27 show how information enters the lending system, which processes handle it, and where it is stored. Level 0 gives the overall view; Level 1 breaks the lending process into smaller steps.

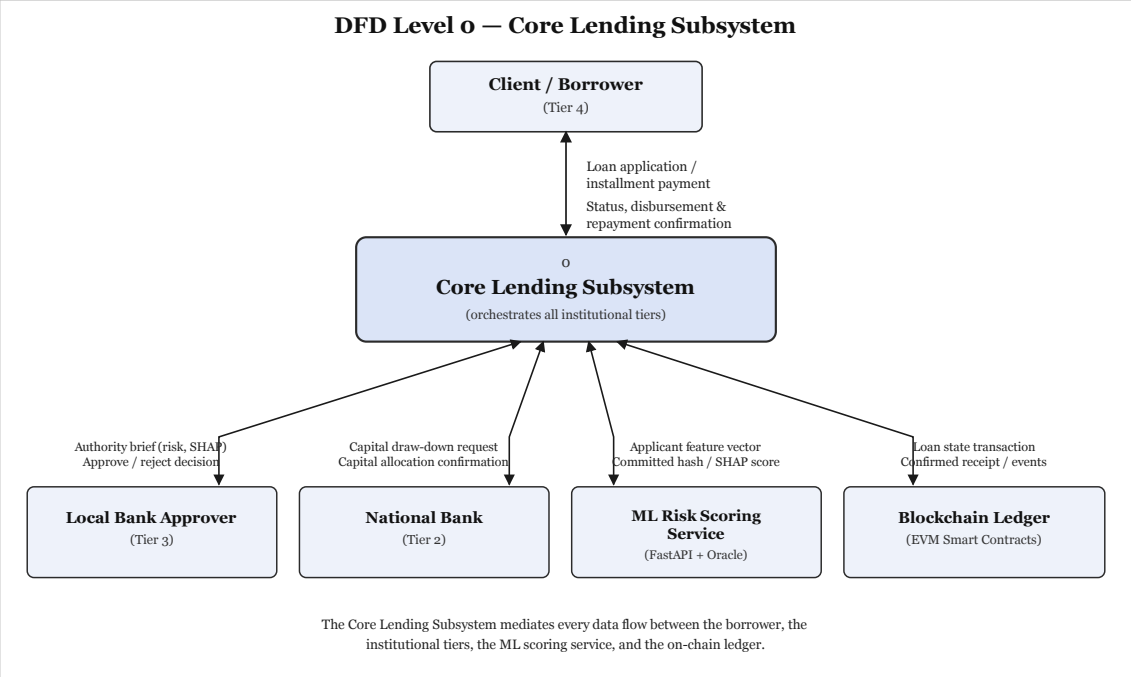

Figure 4.26: DFD Level 0.

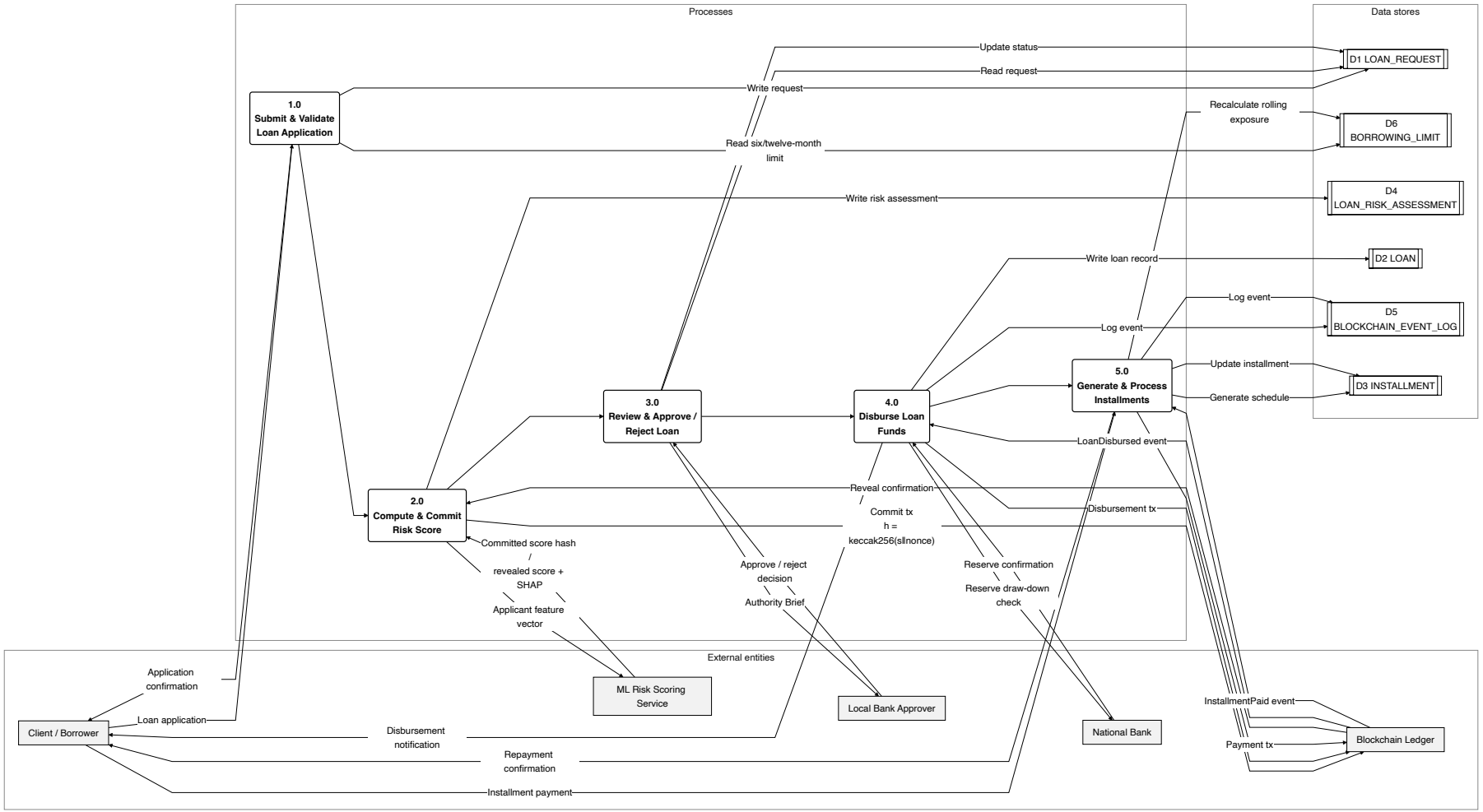

Figure 4.27: DFD Level 1.

Sequence Diagrams

Figures 4.28–4.31 show the order in which different parts of the system exchange messages—for example during loan approval, installment payment, bank hierarchy operations, and chat with the AI assistant.

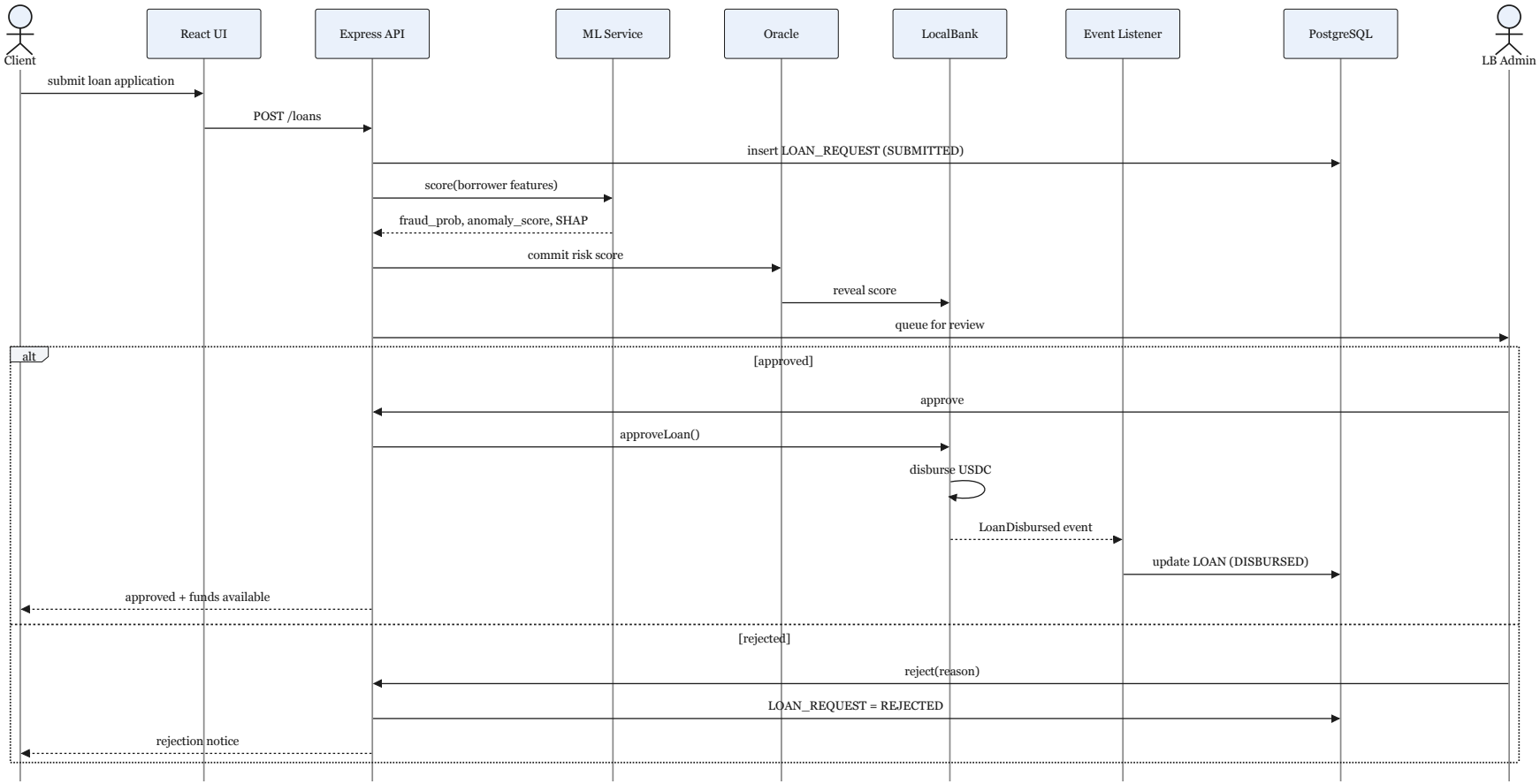

Figure 4.28: Loan approval sequence.

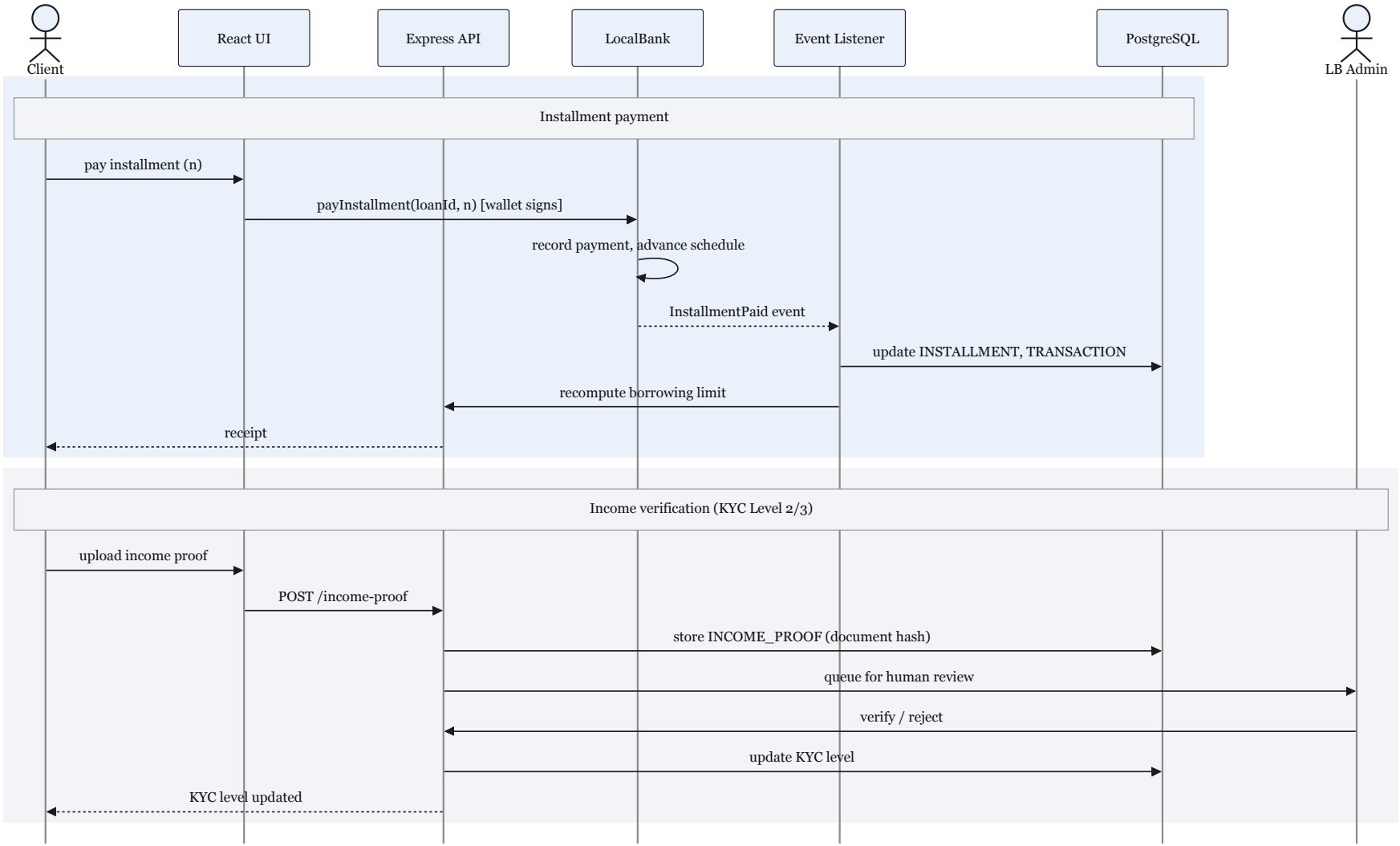

Figure 4.29: Installment and income verification.

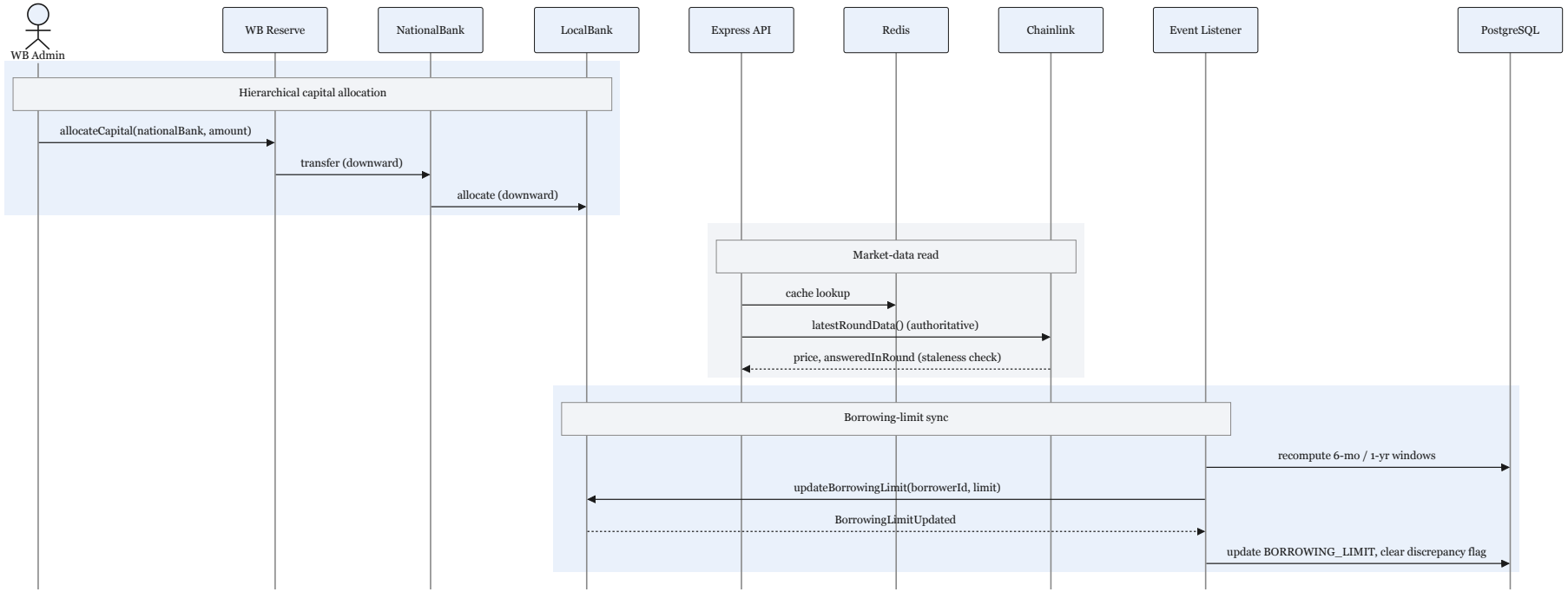

Figure 4.30: Hierarchical banking and data sync.

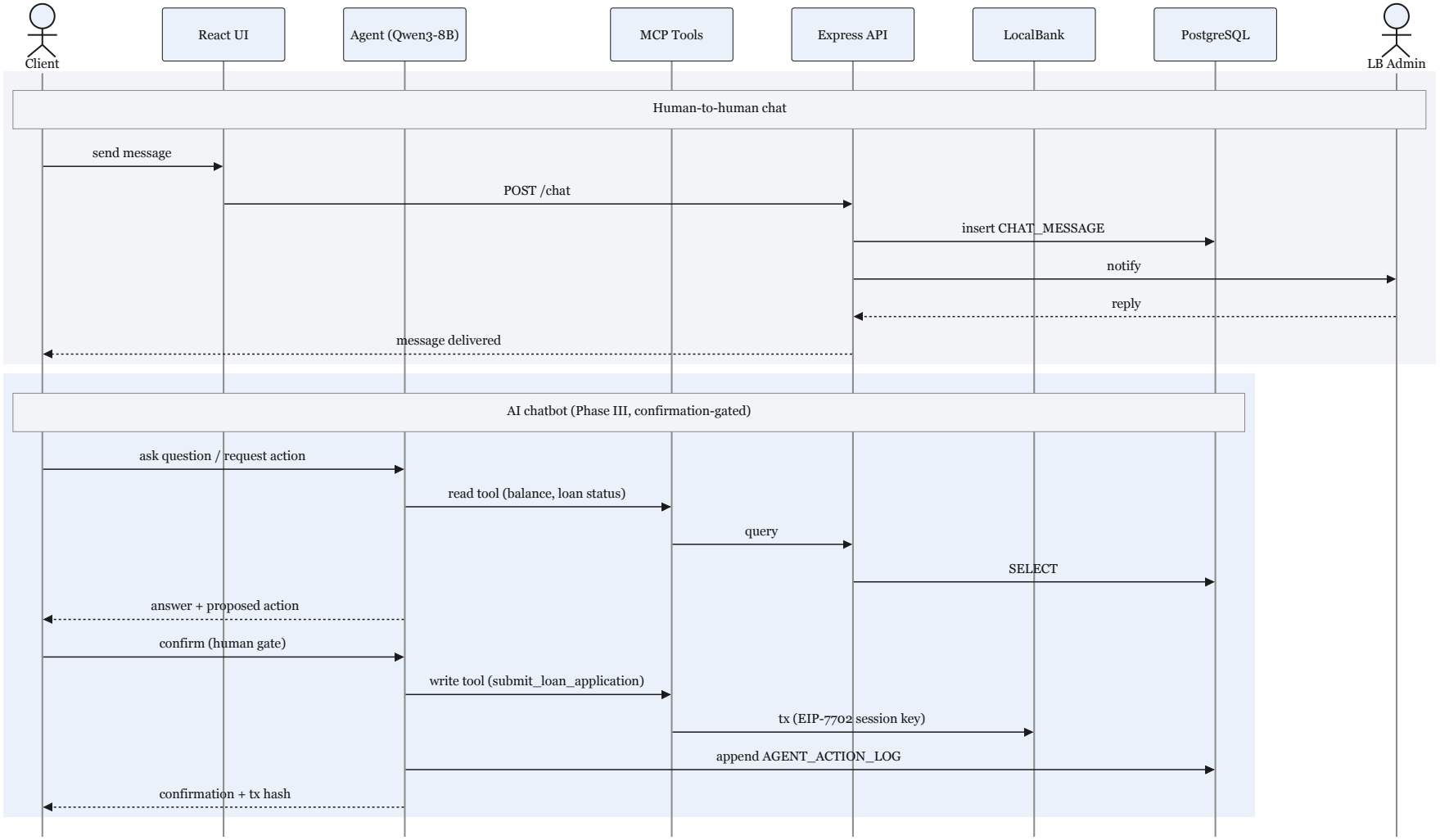

Figure 4.31: Chat and AI agent sequences.

UML Class Diagram

Figure 4.32 shows the main software classes in the lending domain—institutions, borrowers, loans, and backend services—and how they relate to one another.

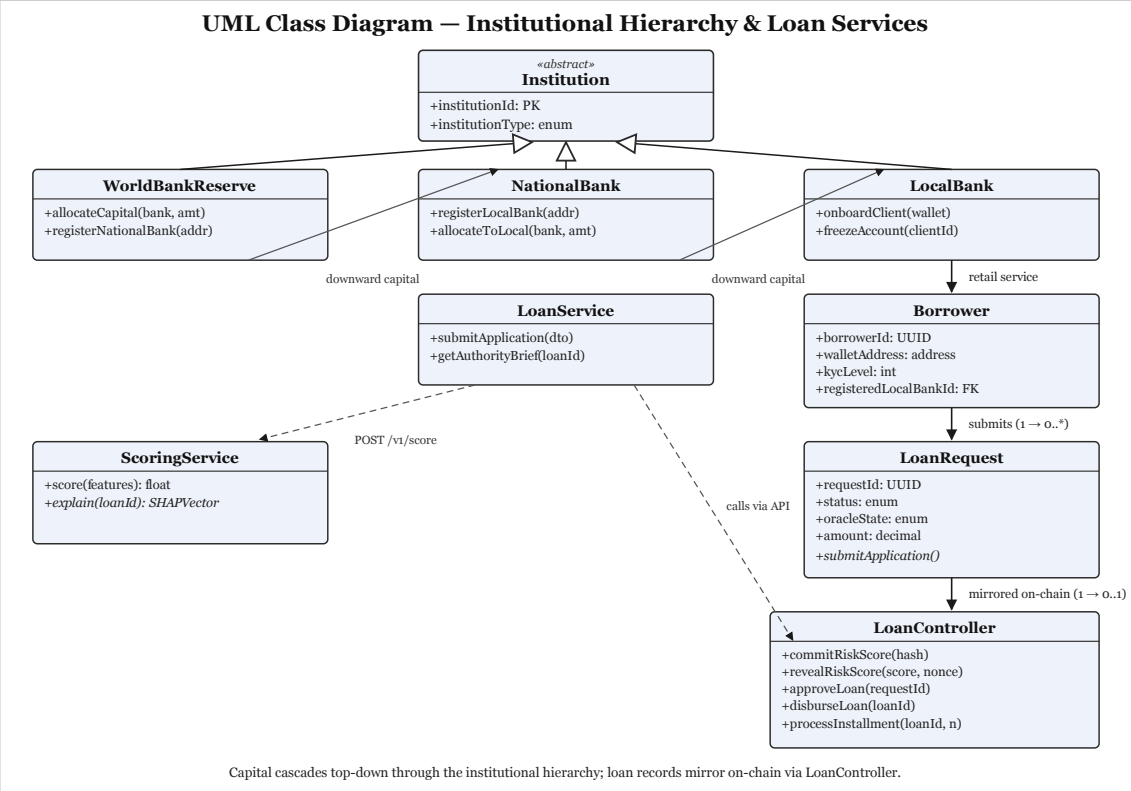

Figure 4.32: UML class diagram.

User Interface Design

User Interface protype designs are shown below. some of the key features are desined for the initial stages, all the prototype pages will be ready as development phases continue.

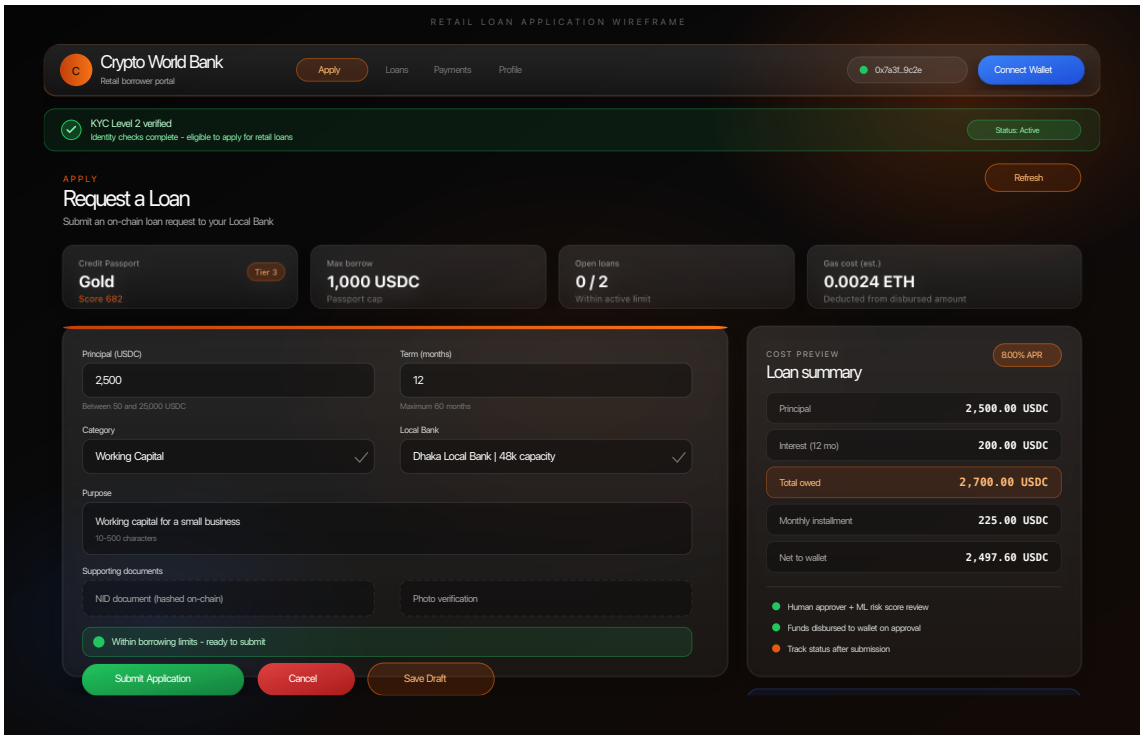

Figure 4.33: Retail loan wireframe.

Retail loan application flow.

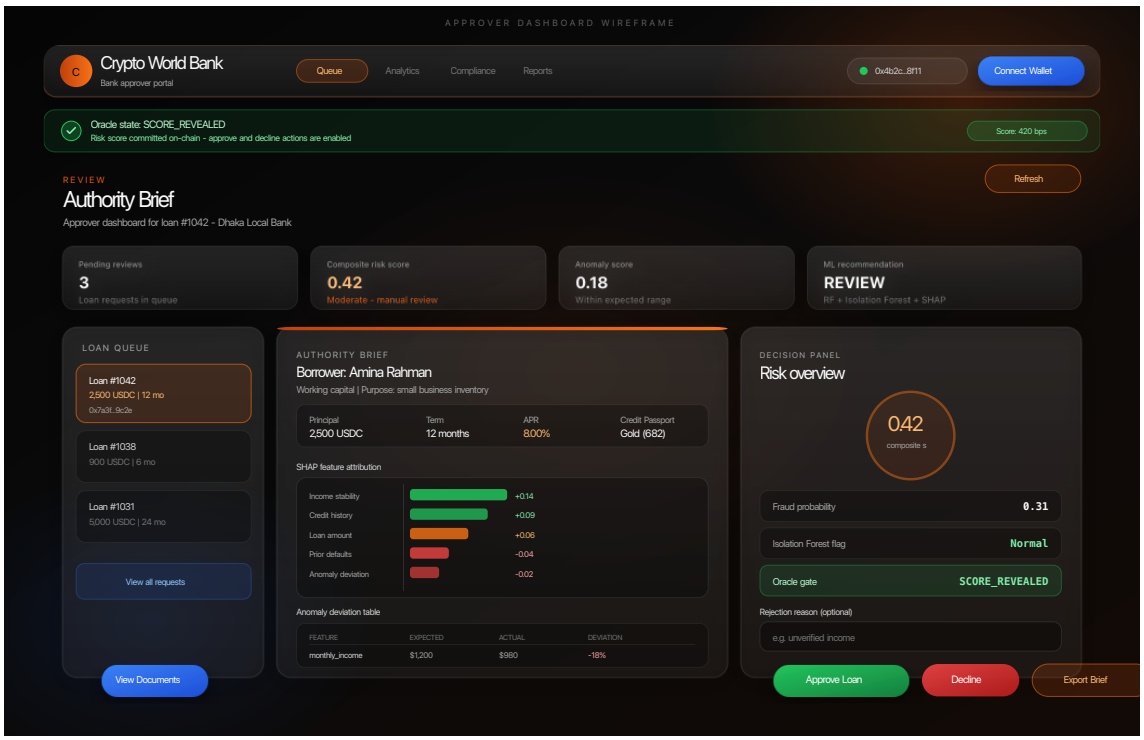

Figure 4.34: Approver dashboard wireframe.

Authority Brief approver dashboard.

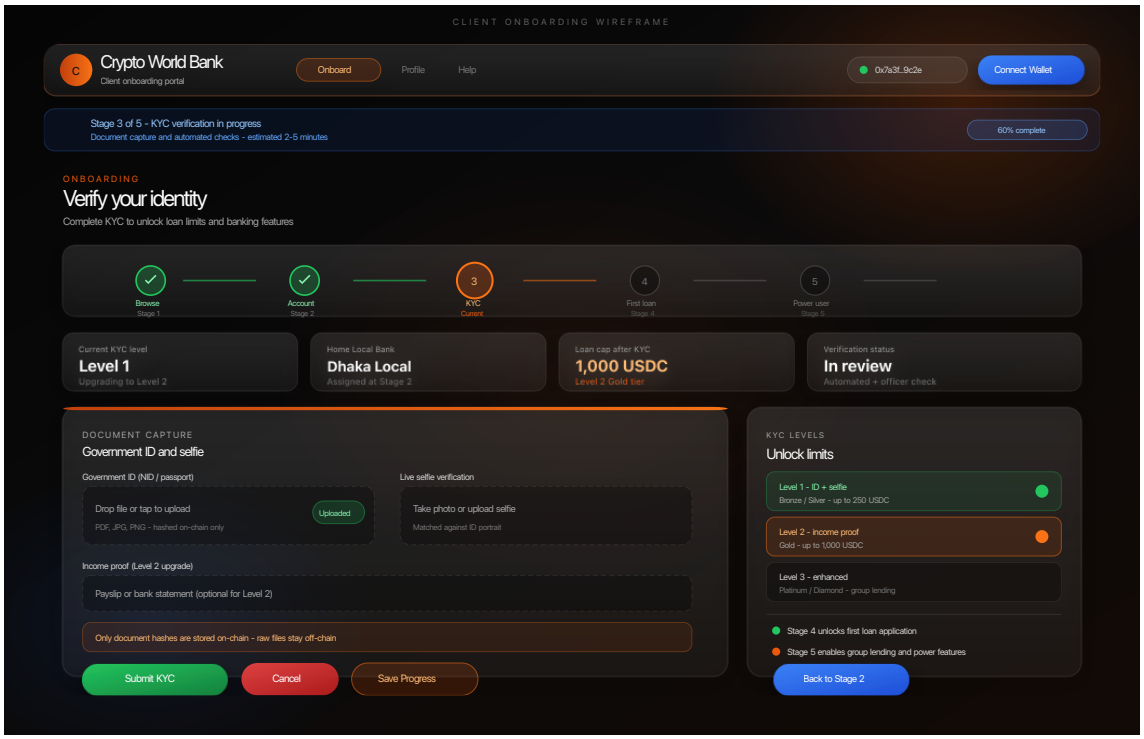

Figure 4.35: Onboarding wireframe.

Client onboarding funnel (KYC levels).

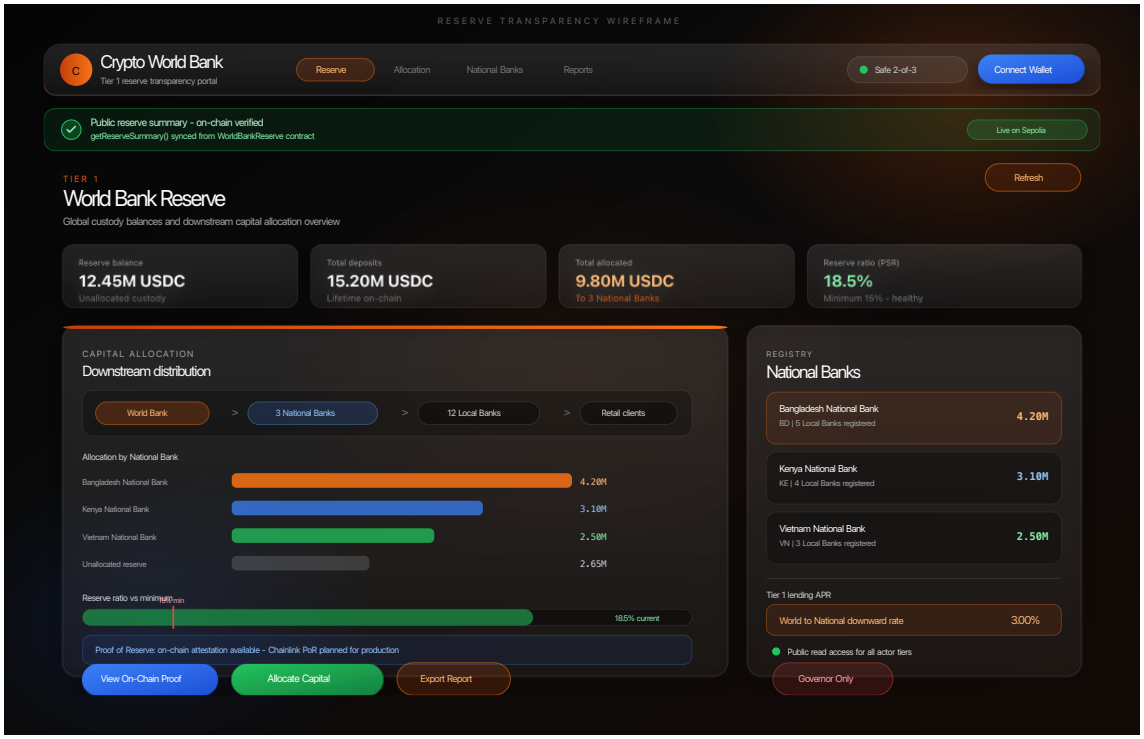

Figure 4.36: Reserve dashboard wireframe.

Reserve transparency dashboard (Tier 1).

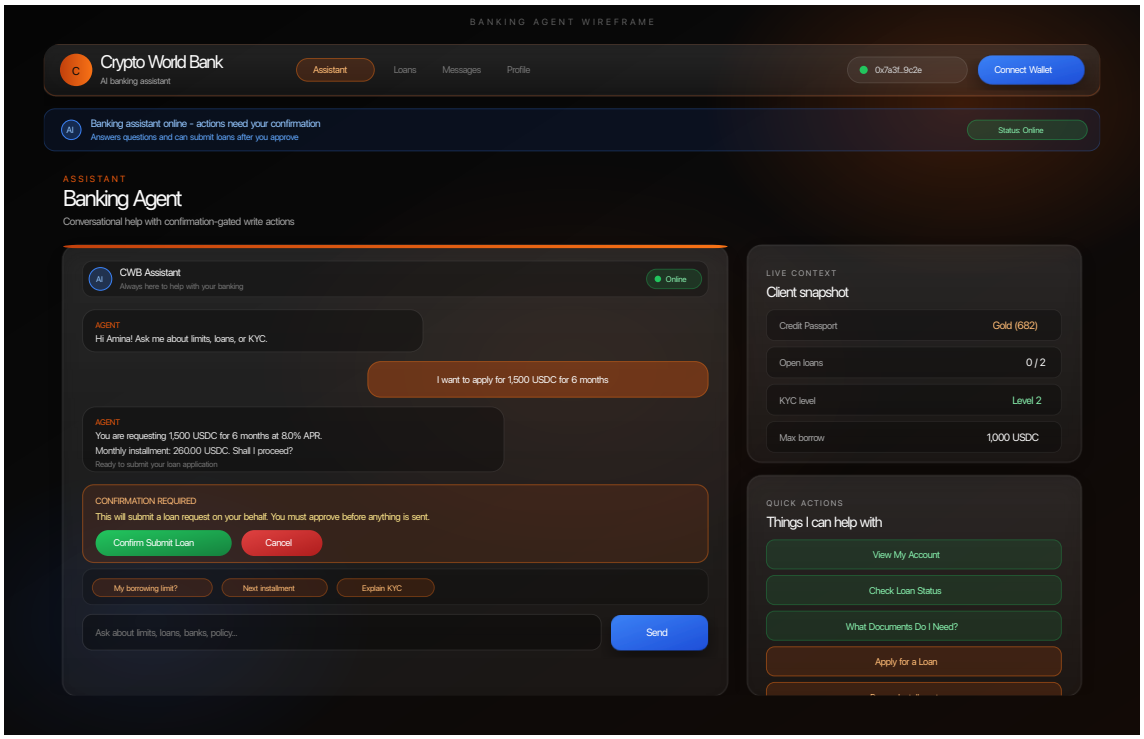

Figure 4.37: Banking agent wireframe.

Conversational banking agent (chat).

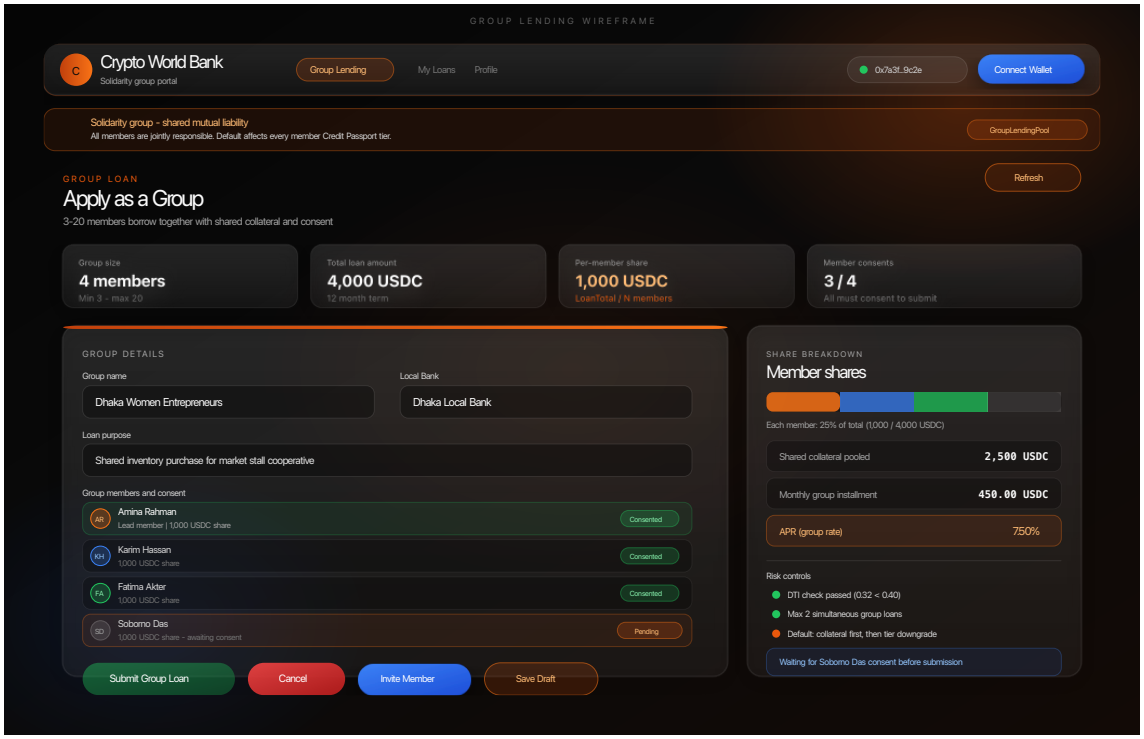

Figure 4.38: Group lending wireframe.

Group solidarity lending application.

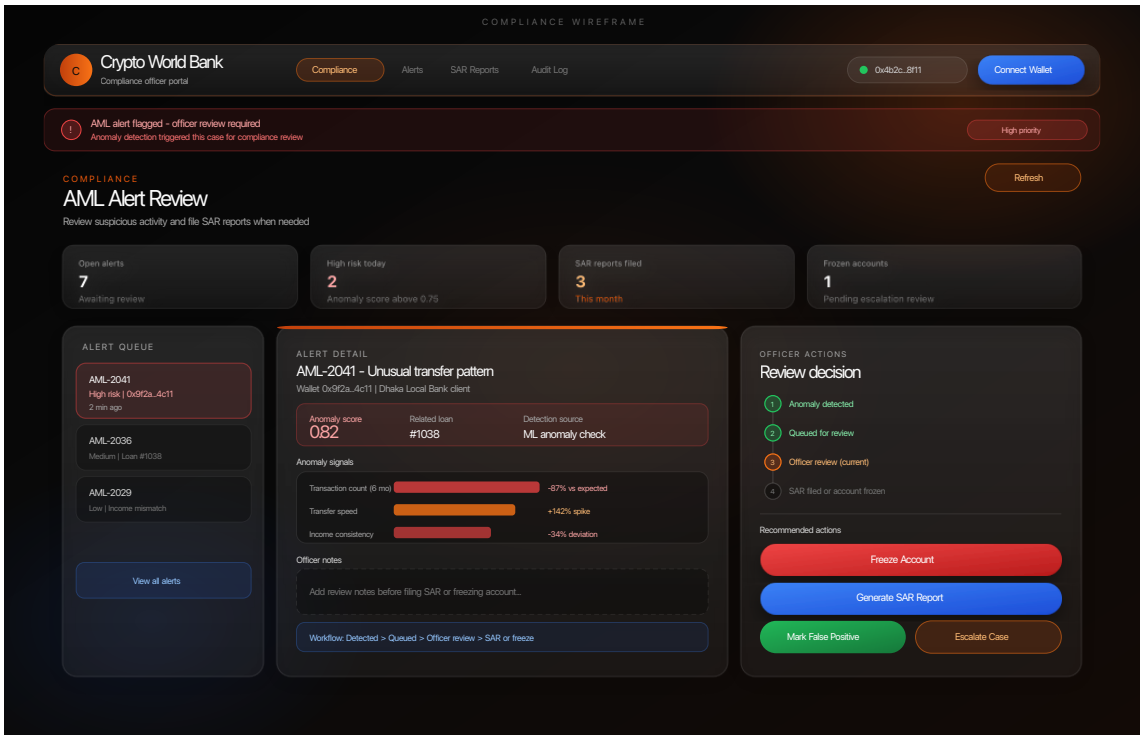

Figure 4.39: Compliance wireframe.

Compliance and AML alert review.

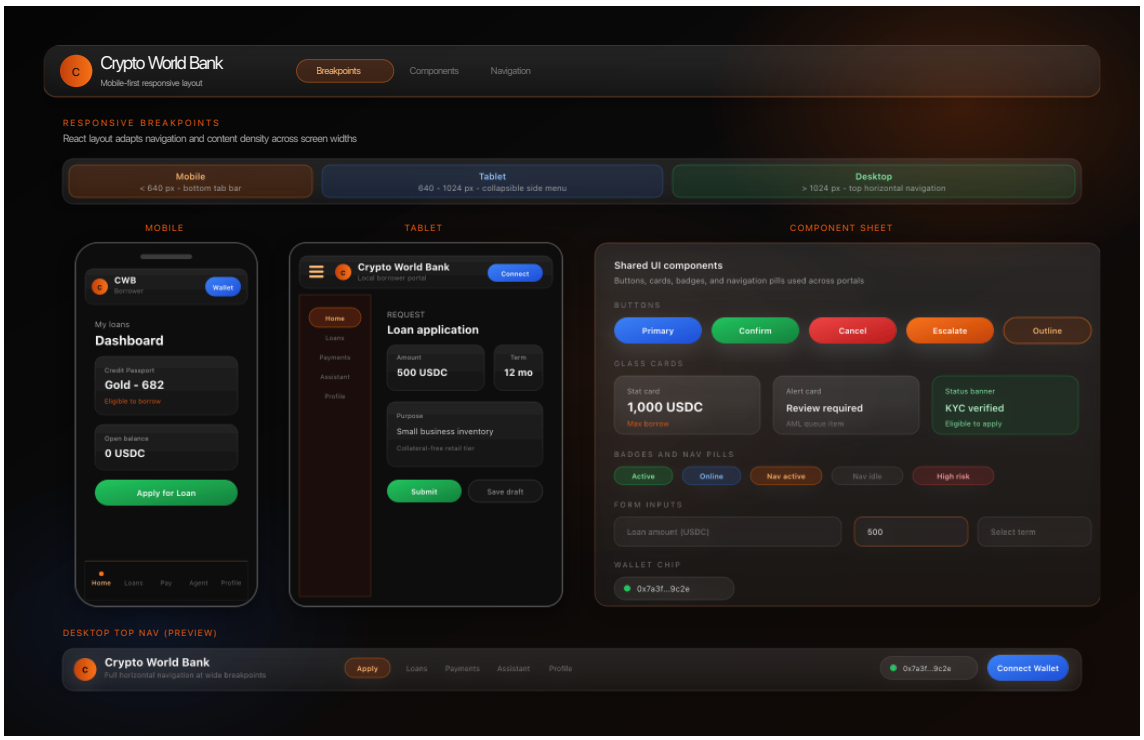

Figure 4.40: Mobile layout wireframe.

Mobile navigation and responsive layout.

Four-Tier Capital Flow

Figure 4.41 shows how capital moves down the World → National → Local → Client hierarchy and how repayments flow back upward through the same tiers.

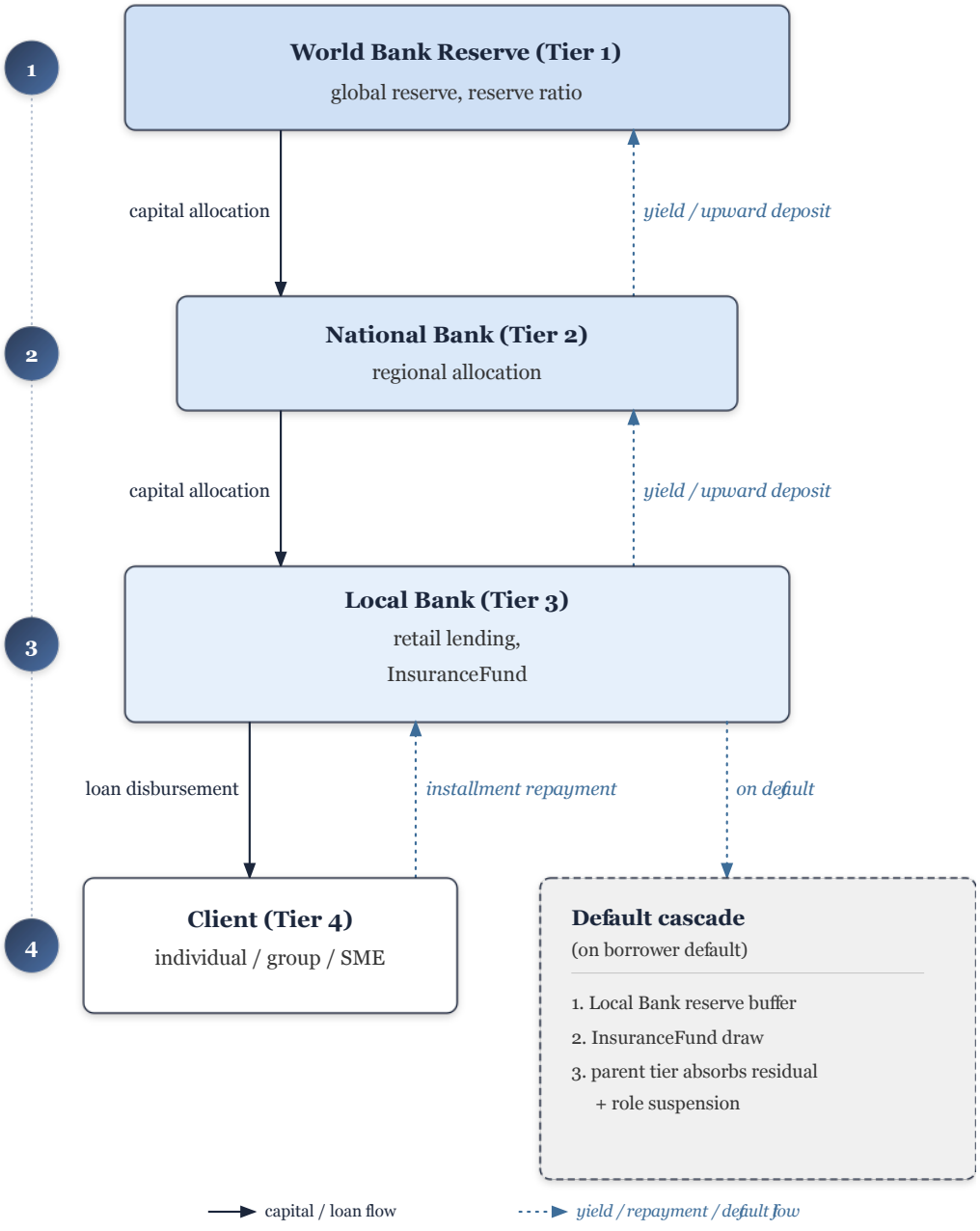

Figure 4.41: Four-tier capital flow.

4.2.11 Governance Framework

If any changes are required for the system settings or any functionalities need an update, the **TimelockController** function adds a **48-hour waiting period** to apply those changes to the whole system. It also requires **60% votes** from the

users or accounts with admin role for the change to take action. **System pause feature** is also built in that requires **2/3 admin votes** to activate it, just in case of emergency. For advanced security of the admin role interaction with the system, **safe multisig wallets** are being utilized instead of a single private key, so power is not accumulated to one person's decision. The **fraud detection models** utilized to find any fraud attempts to the system, suspected wallet interactions or abnormal user behaviors are **flagged** and sent for reviewing, and accounts can be **frozen** based on those evidences until being manually reviewed by authorities.

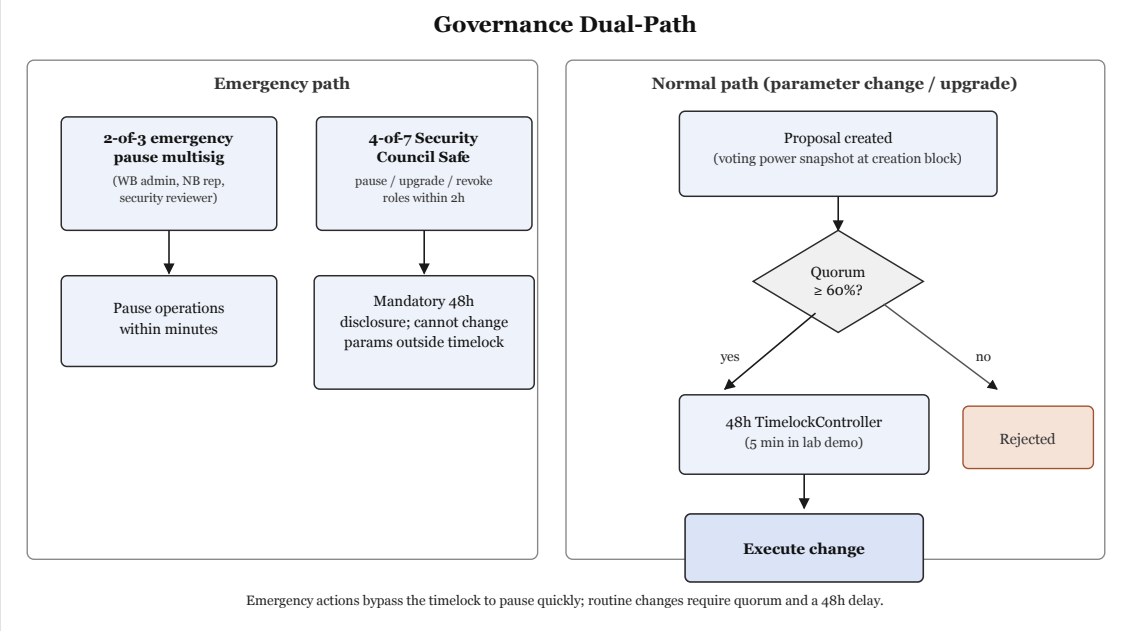

Figure 4.42: Governance dual-path.

Table 4.19: Governance framework summary.

Area	Implementation
Network membership	World Bank registers National Banks; National Banks register Local Banks; hierarchical RBAC on-chain
Business operations	Reserve management, loan lifecycle, event notifications; testnet best-effort SLA
Technology	EVM contracts (decentralised); Express API (centralised); event listener syncs PostgreSQL projections
Emergency controls	Per-function pause registry; 2-of-3 emergency multisig; upgrade via UUPS + Safe
Regulatory	Audit logs via contract events; sandbox pilots for production; zkKYC Future Work

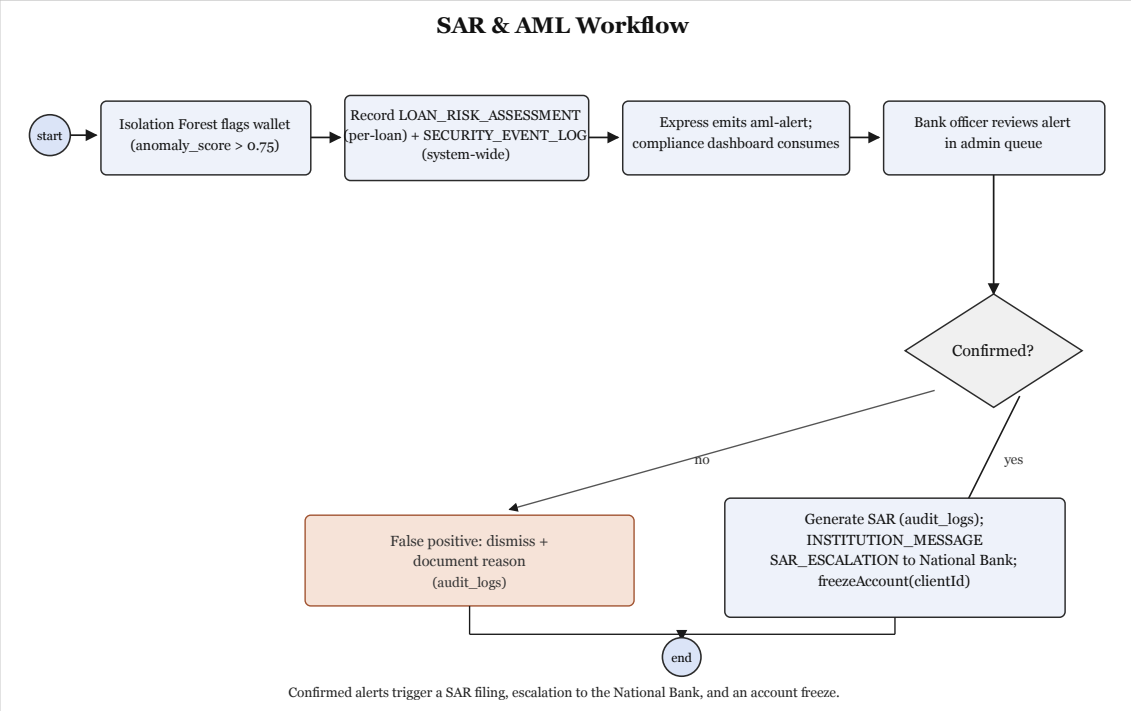

Figure 4.43: SAR and AML workflow.

4.2.12 Five-Layer Defense-in-Depth Security Architecture

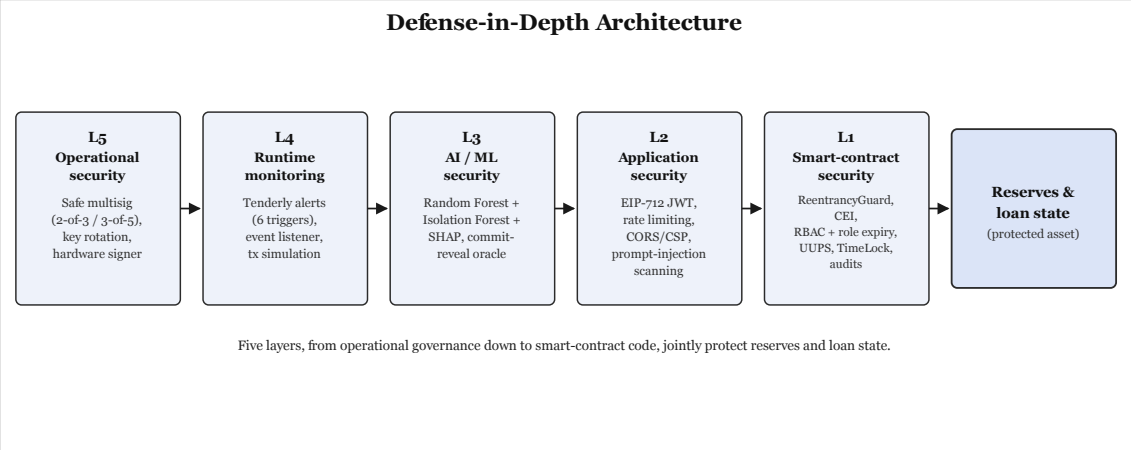

Figure 4.44: Defense-in-depth architecture.

A fully sophisticated **5-layer defense system** has been implemented instead of simple security layers. On the top, **Admin control** has been distributed to multiple **safe multisig wallets** so no one can hold power to admin-level decisions alone. **Layer 4** is feature packed with **anomaly and security alert system**, it logs the changes and alert systems are triggered. **Layer 3** is more advanced with **ML fraud detection services**. The service automatically scans, compares, flags, even forwards instructions to the system if certain wallets should be **frozen** before further manual reviews or inquiries. **Layer 2** consists of **API protection services**

alongside with **rate limiting** service. **Layer 1** is functional at the core, with all the abilities of **smart contracts**. It applies **role checks**, guarding against repeated attack calls. Despite having all the security layers, major decisions regarding providing a loan, financial transaction or any changes to the system, required **manual human confirmation** before any important decision is being approved.

Table 4.20: Defense-in-depth model.

Layer	Component	CWB control (MVT)
L5	Operational	Safe 2-of-3 for `WorldBankAdmin`
L4	Monitoring	Tenderly alerts on large disbursement, low reserve, pause
L3	AI/ML	RF + IF + SHAP; commit–reveal oracle; human approver gate
L2	Application	JWT auth, rate limits, prompt-injection scan on agent input
L1	Smart contract	OpenZeppelin RBAC, CEI, pause, Hardhat + Foundry tests

4.2.13 Threat Model and Security Controls

With the documented security layers to provide a very well protected financial services, there are still risks that are needed much careful attention. Here listed some of the common risks that may temper with the functionality and security of the system and the planned mitigations.

The first risk to consider is **reentrancy**, when an attacker attempts multiple transaction requests from the same wallet before one transaction is completed. This attempt is made to create a false scenario of sending more or receiving less than the expected amount. This risk is mitigated in **Layer 1** smart contract guard that prevents response to multiple calls before one transaction is completed with confirmation handshake. Second risk is bad estimation of current price of assets and currencies from the **oracle**. This risk has been solved with **Chainlink up-to-date price feeds**. Third risk of admin's ability to make major decisions without the considerations of permission from majority, which has been resolved with the **safe multisig wallets**, **timelock waiting periods**, and **admin roles with time limitations**, applied across Level 1 to Level 5. The login tokens and API keys can create vulnerabilities too with flooded requests which are considered as the 4th risk. This issue has been addressed and solved in **Layer 2** with short time login access and limited requests. Fifth risk is the fraud models being bypassed by some experts, and the risk is considered heavily and as a solution to this, all major decisions regarding loan approval are manually reviewed by the **human review gate** with **SHAP explanations** to guide them, in security **Layer 3**.

Table 4.21: Threat model.

Threat	Layer	Vector	Mitigation
Reentrancy	L1	Recursive external call	`ReentrancyGuard`, CEI, Foundry tests
Oracle manipulation	L1	Bad price feed	Chainlink feeds; mock feed on testnet
Governance capture	L1+L5	Privileged role abuse	Safe multisig, TimeLock, role expiry
JWT / API abuse	L2	Stolen token, DDoS	Short JWT TTL, `slowapi` limits
ML evasion	L3	Adversarial features	SHAP review, human approver gate
Admin-key theft	L5	Compromised EOA	Safe multisig; no single-key admin

4.2.14 Design Decisions and Alternatives

Table 4.22 demonstrates the chosen stack for frontend, backend, and all dependencies for the stability and completion of the project. The comparison table is shown below:

Table 4.22: Design decisions.

Decision Area	1st Choice	2nd Choice	Key Criterion
Methodology	Agile / Scrum	Incremental	Evolving scope, milestones
Architecture	DApp + Off-chain AI	Hybrid with Oracle	Gas cost, ML flexibility
Frontend	React + TypeScript	Vue + TypeScript	Web3 ecosystem maturity
Smart Contract	EVM (Solidity)	Solana (Rust)	Gas, control size, free testnets
Fraud Detection	Random Forest	XGBoost	SHAP compatibility, simplicity
Anomaly Detection	Isolation Forest	Autoencoder	Unsupervised; no labelled data needed
XAI Method	SHAP	LIME	Theoretical guarantees, regulatory fit
Database	PostgreSQL	SQLite	3NF support, async queries
Hosting	Vercel + Render	Localhost only	$0 cost, publicly accessible URL
Network (primary)	Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary)	Polygon zkEVM Cardona	Stability-first prototype vs. ZK-anchored design preference
Oracle mechanism	Commit-reveal relay (MVT)	Chainlink Functions (DON)	Always-available audited relay vs. DON-based upgrade when supported
Agent tool framework	MCP tool server	Direct Express.js API	Defined schema = safety boundary; Phase III Must
Agent session auth	Human confirmation + standard wallet signing	EIP-7702 session keys	MVT relies on explicit consent; scoped session keys are Future Work
Prompt construction	Three-tier assembly	Single ad-hoc string	Phase III Must
Session memory	Lineage model	10-turn window	Phase III Should; cross-session continuity

Table 4.23: Design alternative criteria.

Decision	Selected	Rejected	Criteria (weight)	Outcome
Methodology	Agile + incremental phases	Pure Waterfall	Adaptability (40%), milestone clarity (30%), team size (30%)	Agile wins on evolving scope
Settlement chain	Sepolia / Amoy (demo)	zkEVM Cardona (day-1)	Stability (50%), cost (30%), ZK anchor (20%)	Demo path stability-first
ML delivery	Off-chain FastAPI + commit–reveal	On-chain Chainlink Functions	Latency (35%), cost (35%), auditability (30%)	Off-chain for MVT
Fraud model	Random Forest + SHAP	XGBoost + LIME	Explainability consistency (50%), tabular fit (30%), imbalance (20%)	RF exact TreeExplainer
Database	PostgreSQL 3NF	SQLite	Concurrency (40%), window queries (35%), FK integrity (25%)	PostgreSQL for lending projections

Justification of Selected Technologies

Table 4.24: Selected technology justifications.

Technology	Justification
Agile/Scrum	Supports evolving contract, UI, and ML scope within short iteration cycles.
DApp + off-chain AI	ML runs off-chain for speed and cost; critical lending decisions remain on-chain.
React + TypeScript	Strong Web3 library support (Wagmi, RainbowKit, Viem, ethers.js).
EVM (Solidity)	Mature security tooling; Sepolia/Amoy testnets; OpenZeppelin libraries.
MUI (Material Design 3)	Ready-made tables, forms, and dashboards for banking UI.
Random Forest	Exact SHAP TreeExplainer; strong on tabular lending data; handles class imbalance.
Isolation Forest	Unsupervised anomaly detection with few labeled fraud samples.
SHAP	Consistent, theory-grounded feature attribution for explainability.
PostgreSQL	ACID compliance, window queries for borrowing limits, JSON for prototyping.
Vercel + Render	Free hosting for live supervisor and examiner demos.
Sepolia / Amoy testnets	Stability-first deployment path within the thesis timeline.
Commit–reveal oracle	Simple MVT path with immutable on-chain ML score audit trail.
MCP tool server	Typed tool schema bounds agent reads and writes.
EIP-7702 session keys	Future Work: scoped session keys for agent on-chain actions.

Table 4.25: Alternative technology justifications.

Alternative	Why retained but not primary
Waterfall	Clear phases, but too rigid for research prototyping.
Hybrid on-chain oracle	Higher latency and cost; external oracle dependency.
Vue + TypeScript	Smaller Web3 ecosystem than React.
Solana (Rust)	Steeper learning curve and weaker audit tooling within 16 weeks.
Tailwind CSS	More custom UI work than MUI for banking layouts.
XGBoost	Approximate SHAP; weaker explanation consistency.
Autoencoder	Needs more data and tuning than Isolation Forest.
LIME	Less stable explanations than SHAP for audit use.
SQLite	Limited concurrency and window-query support.
Localhost-only deploy	Blocks remote supervisor and examiner access.
Polygon Amoy PoS (2nd network)	Fallback testnet if Sepolia is unstable.
Chainlink Functions	Stretch upgrade when supported; higher latency and fees.
Direct Express.js API	Weaker safety boundary than MCP tool schema.
ERC-4337 paymasters	Weaker scope control than EIP-7702 for agent actions.

4.2.15 Planned AI/ML Support and Risk-Score Wiring

The purpose of the AI/ML layer is to provide a risk score per loan approval that will be delivered on-chain through the commit–reveal oracle (Section 4.2.3). The oracle gate design has been shown in Figure 4.45. ML scoring and explainability outputs have been shown in Section 5.4.1, and the rest of the benchmarks and evaluations have been reported in Chapter 5 (Sections 5.1 and 5.4).

Risk Scoring: A sequential step combines Random Forest classifier and Isolation Forest to provide fraud probability and anomaly scores, and these scores are evaluated within $s \in [0, 1]$.

Commit–reveal oracle: This commits a hash of (s, nonce) to `LoanController`, then each transaction comes with a risk score.

Decision threshold: A minimum and maximum threshold of the value of score (s)

is maintained for loan approval decisions: approval consideration ($s < 0.4$), manual review ($0.4 \leq s < 0.7$), or rejection ($s \geq 0.7$).

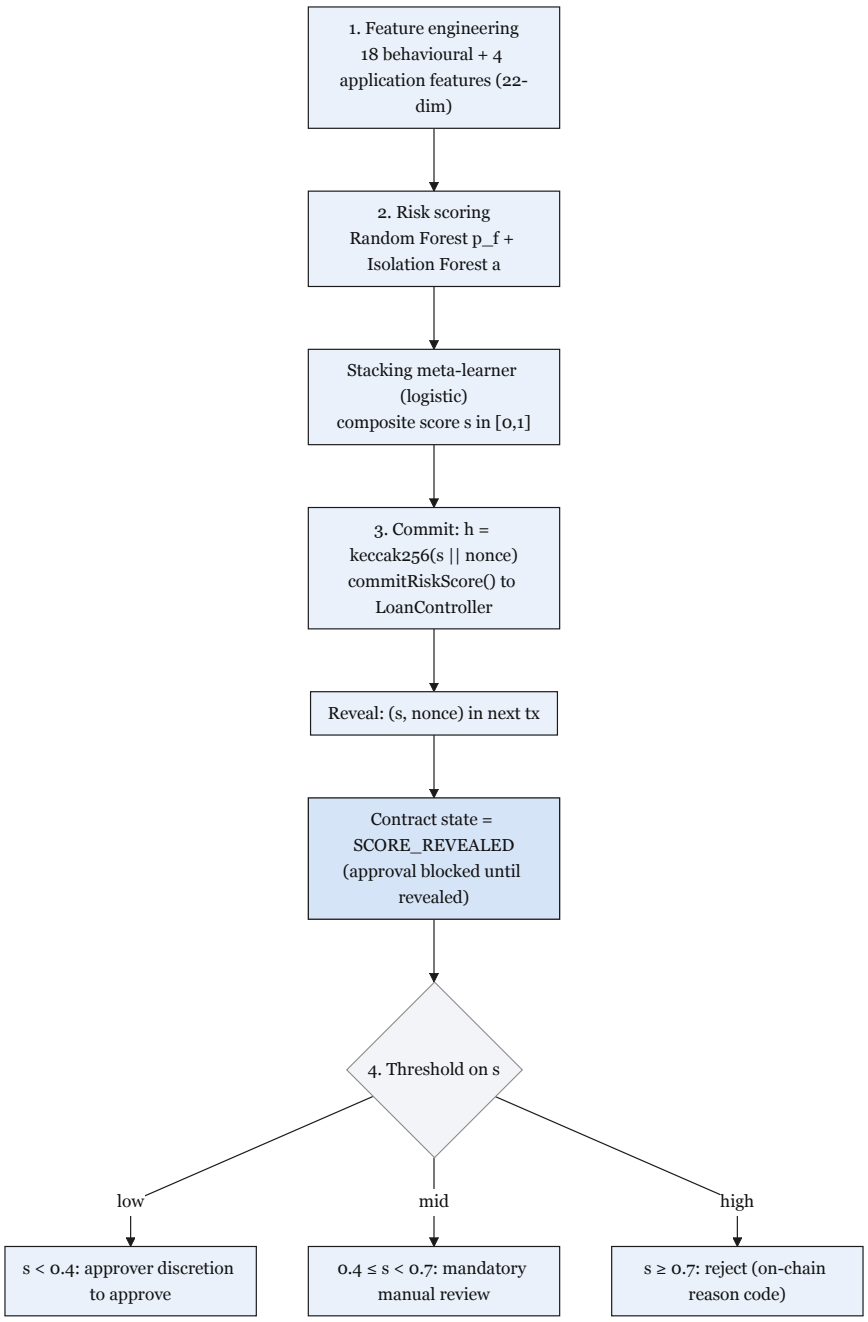

Figure 4.45: ML oracle commit–reveal.

4.2.16 Conversational Agent (Phase III–IV)

Agentic Model Selection, and Integration

Agentic Capabilities. A conversational model with higher parameter counts with top class accuracy may not be good at agentic workload if not trained for the purpose. Qwen3-8B is an open source model trained with the intention of

agentic workload with tool calling accuracy with top of the class agentic task completion abilities for its size category. It can output perfect structurally accurate JSON matching with our database schema to execute commands with near perfect accuracy for a long time in sequential commands.
The conversational agent that has been designed to assist the clients to better navigate the platform is planned to be completed by Phase III. It is an auxiliary feature that works alongside the standard web UI for those who may not be familiar with the banking system of the project and have less domain knowledge and want to understand the platform and how to navigate with it. The schema of the full MCP agent has been shown in Table 4.26; explainability for approvers is documented in Section 5.4.1.

Agent six-step pipeline. Clients are expected to interact with the frontend UI like any other typical user-friendly banking website. The agent works as a guiding assistant and its tool-calling abilities use the same Express.js API endpoints; the same smart contracts are considered for functionalities. This six-step pipeline lets the Qwen3-8B model have access to the live on-chain context injection layer that helps the agent process related instant and consistent information that is relevant to how a client is interacting with the platform. The layers have been described shortly below:

1. **User enters the platform and interacts with the agent:** frontend interaction is recorded and forwarded via the Express.js backend over Server-Sent Events (SSE).
2. **Agent model with live on-chain context:** The Express.js context injection layer provides the client's information to the model (credit score, active installments, loan status, SBT risk tier, pool utilisation). The model receives this information about the user before the conversation prompt by the user, to process relevant information about the client to assist them.
3. **Q&A:** For this part of the conversation between the agent and client, the agent only processes relevant information about the client via RAG (ChromaDB) and maintains formal conversation with the client, and does not call any tools.
4. **Action request:** After a formal conversation with the client, if the client requests a state-modifying action such as requesting a loan, the agent will choose and prepare MCP write tools, assemble the full parameter set from the injected context, and show a concise summary of action on behalf of the client. Ex: *"You are requesting a loan of 150 USDC for 6 months at 8.5% APR. Your monthly installment would be 26.25 USDC. Shall I proceed?"*
5. **Human confirmation gate:** The agent will wait until a confirmation of affirmation for the action that has been prepared to initiate the loan process.

The MCP agent writing tools will not be initiated until a positive response has been received from the client side. If a client responds with vague actions three times, the agent does not make the tool calls, pauses the agent service for 10 minutes, and logs this interaction to `AGENT_ACTION_LOG`.

6. **MCP write tool functionality:** If a successful conversation is established with the client, the agent takes confirmation from the client to initiate the loan request process; with a positive response the agent initiates write tools and accesses the Express.js banking API, then checks for an EIP-7702 session key—if active, it signs the on-chain transaction approval request. The agent gives a confirmation that the request has been forwarded and notifies the client that the loan application has been submitted, shows a reference number, and how long the approval might take.

MCP tool server. For the MVT scope the agent has been designed for 3–5 tools for a minimal typed tool schema. The specified 17 tools have been designed for future work, and some of these tools may be included in this prototype build if they can be delivered in the allocated time, after the MVT scope has been secured.

Table 4.26: MCP tool server.

Tool	Type	Maps to / Description
`get_account_state`	READ	SBT tier, loan count, savings balance
`get_loan_status`	READ	Active loans + next installment
`get_requirements`	READ	Documents + KYC needed for a given loan amount
`submit_loan_-application`	WRITE	POST /api/loan/apply
`pay_installment`	WRITE	POST /api/installment/pay

Live on-chain context injection. client's infromation is injected tot he model in a JSON file structure. the Express.js backend injects the JSON file beore each promt is being processed by the Model to provide assistacnce and read/write tool services.

```
{
  "tier": "volatile",
  "borrower_id": "550e8400-e29b-41d4-a716-446655440000",
  "wallet_address": "0xABC...",
```

```
  "language": "en",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "session_id": "sess_20260602_xyz",
  "parent_session_id": "sess_20260515_abc",
  "compression_summary": "Client asked about Gold tier eligibility.
  Was told 2 more on-time repayments required. No action taken.",
  "conversation_history": [ "...last 10 turns..." ]
}
```

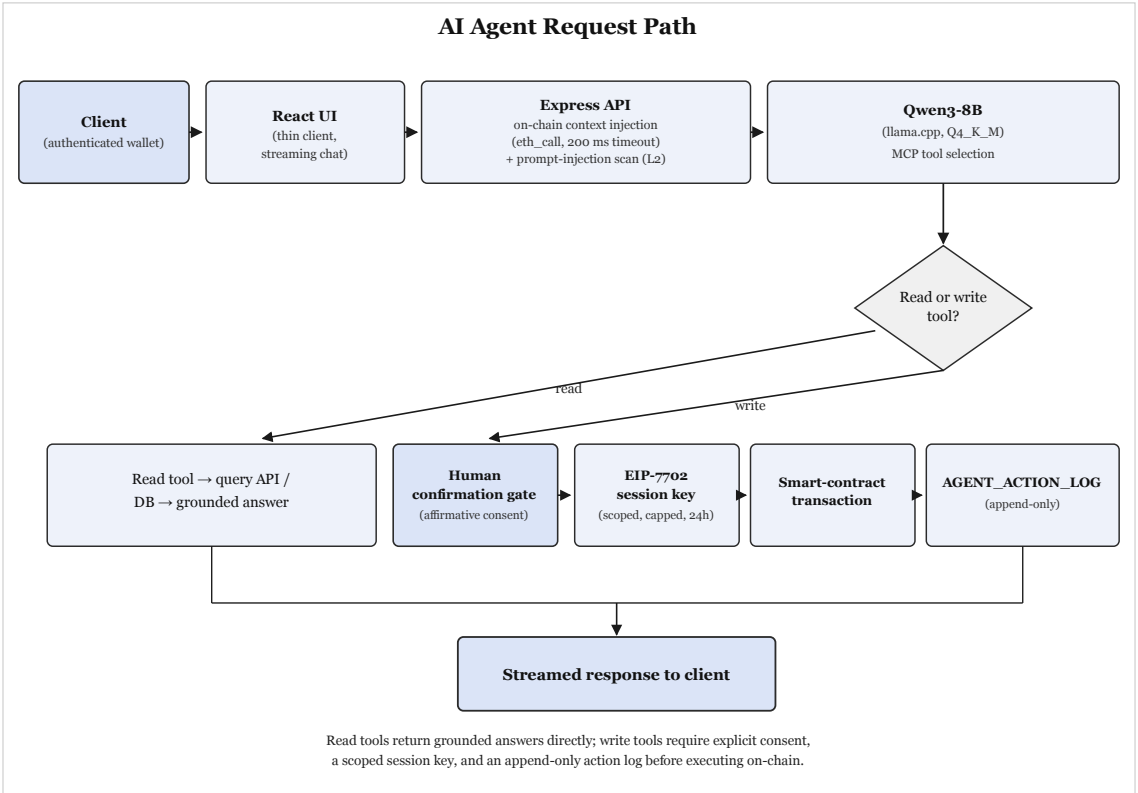

Figure 4.46: AI agent request path.

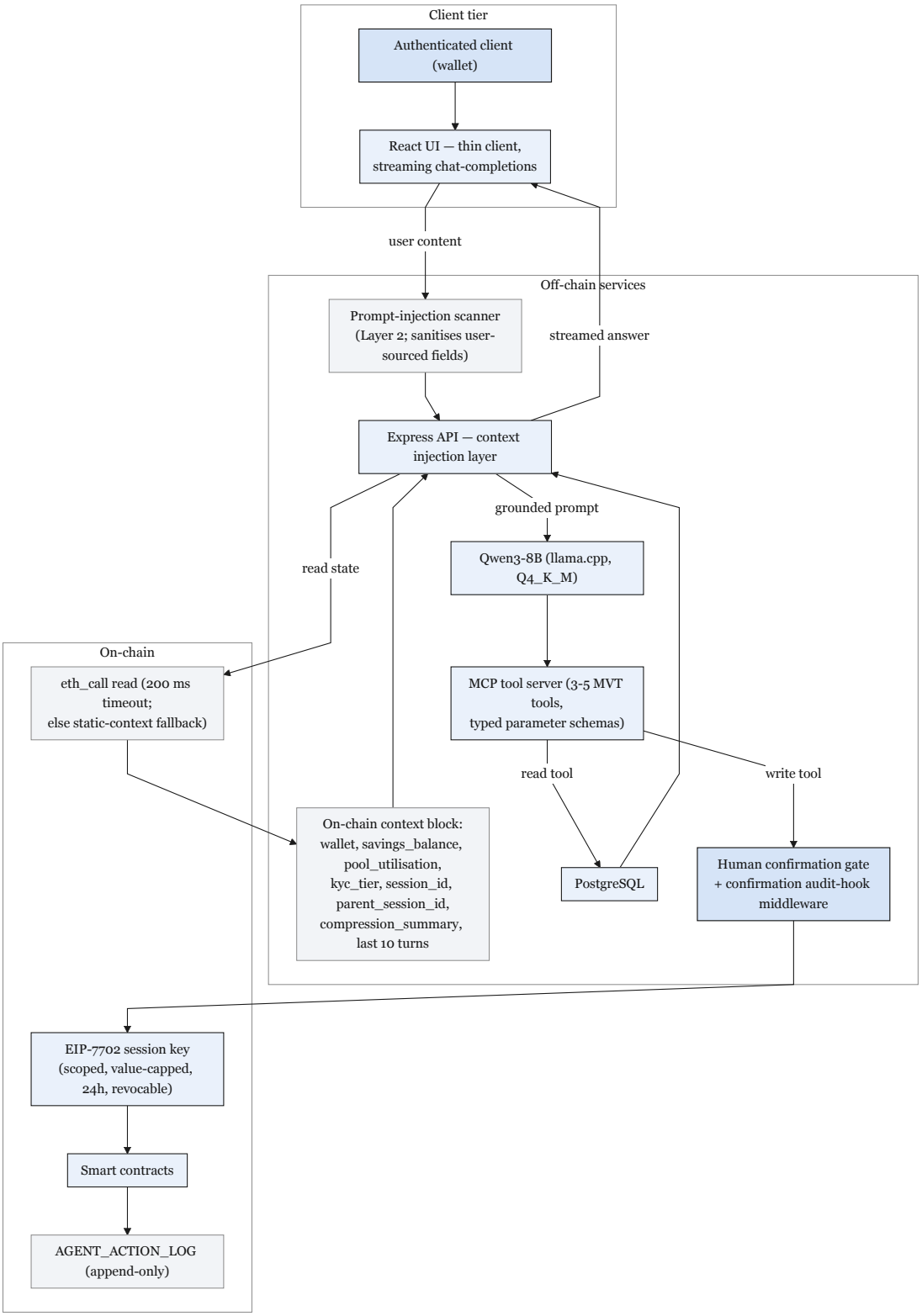

Figure 4.47: AI agent data flow.

Table 4.27: Technology stack summary.

Layer	Technology	Version / Notes
Smart Contract	Solidity, OpenZeppelin	0.8.20; Ownable, ReentrancyGuard
Frontend	React, TypeScript	Material UI; Design 3
Wallet Integration	Wagmi, RainbowKit, Viem	EIP-1193 compliant
Build and Test	Hardhat	Automated test suite; deployment scripts
Backend API	Express.js, Node.js	REST API; middleware architecture
Database	PostgreSQL (designed)	51 entities (31 core + 6 extension + 14 stubs), 3NF; relational integrity
Target Network	Ethereum Sepolia (primary) and/or Polygon Amoy PoS (secondary single-chain prototype)	Stable public testnets for delivery; Polygon zkEVM retained as evaluated design preference
AI Agent Engine	Qwen3-8B (llama.cpp, Q4_K_M)	MCP tool server; per-user context namespace; human confirmation gate
MCP Tool Server	Minimal MVT tool subset (3–5 tools); prompt injection scanner; confirmation audit hook middleware	Exposes a small read+write surface to the agent with typed parameter schemas
Session Keys	EIP-7702 (Future Work)	Scoped, time-bound, value-capped agent wallet authorisation
Oracle Network	Commit–reveal (MVT) + Chainlink Functions (stretch)	Commit–reveal enforces auditable score commitment; DON upgrade when supported
Price Feeds	Chainlink AggregatorV3Interface	Fiat/USD pairs (e.g. EUR/USD) and ETH/USD; 8-decimal precision
Automation	Chainlink Automation	Trustless installment due-date checking; replaces centralised cron job
Proof of Reserve	Chainlink PoR	Cryptographically verifiable WorldBankReserve balance
Event Indexing	PostgreSQL event lis-	Stability-first indexing baseline

4.2.17 Real-Time Dashboard Pipeline and Runtime Monitoring

For stability, lightweight data indexing strategies are used to reduce complexities. After the correct implementation of the core contracts and event schemas, an optional query layer is to be added for GraphQL access for better demonstration of analytics. A Node.js service is utilized for WebSocket event listener via Socket.IO. Tenderly monitoring and MVT-graded time and attempt triggers are added for monitoring and inconsistency checks.

Functional Verification of Design Alternatives

This section provides a functional verification for design alternatives from Table 4.22 and Table 4.23.

Table 4.28: Design alternative verification.

Design alternative	Verification method	Status (Pre-Thesis 2)
EVM (Sepolia/Amoy) vs. Solana	Hardhat contract tests; testnet deploy script	42 Hardhat cases (Table 5.1); Phase I–II
Commit–reveal oracle vs. Chainlink DON	Oracle flow walkthrough; commit–reveal gate diagram	Designed; MVT path specified (Figure 4.45)
Random Forest + IF vs. XGBoost	BCCC fraud benchmark; SHAP explainability check	Benchmark met (Table 5.6)
PostgreSQL vs. SQLite	ERD, 3NF schema, integrity constraints	Specified (Table 4.12; Appendix ERD)
300-client capital flow	Foundry/Anvil deterministic simulation	Planned Phase IV (DT-IV.01)
Solvency and RBAC invariants	Foundry fuzz + invariant suite	Planned Phase IV (DT-IV.04)

Chapter 5

Result Analysis

5.1 Performance Evaluation

Verification criteria: Hardhat contract tests; Backend Vitest API tests; Foundry invariants (solvency, role segregation, capital-flow direction) in Phase IV.
Validation criteria: Phase task/feature implementations (specified in Chapter 4); BCCC fraud benchmarks (Table 5.6, Figure 5.2). Table 5.1 contains project implementation files and paths:

Table 5.1: Implementation evidence.

Component	Path	Status
Smart contracts	`contracts/*.sol`	World/National/Local Bank compilable
Contract tests	`test/*.test.ts`	42 Hardhat cases (four suites)
Backend API	`backend/src/`	Loans, banks, auth, income routes
API tests	`backend/tests/`	Vitest auth, banks, loans, chat
Frontend	`frontend/src/`	React + Wagmi wallet shell
ML pipeline	`Documentation/.../ML pipeline - Mac M4/`	BCCC subsample; `metrics.json`
ML service	`ml-service/app/`	FastAPI `/health`, `/score` scaffold
Deploy	`scripts/deploy.ts`	Sepolia/Amoy/local ready

5.2 Analysis of Design Solutions

Table 5.2: CO6 design selection.

Factor	Selected	Evidence	Status
Cost	\$0 prototype; L2 gas ≈\$0.011/loan	Tables 3.12, 3.13	Met
Efficiency	Off-chain ML; single-chain	NFR-01; Phase IV gas table	ML trained; live p95 pending
Usability	React + wallet; MCP agent	NFR-05; FR-08	UI shell; agent Phase III
Impact	Tier-transparent flow; group lending	Section 3.2	Specified
Sustainability	PoS testnets; off-chain ML	Section 3.3	Qualitative
Maintainability	Monorepo; OpenZeppelin; G1–G4	Table 5.1	42 tests passing
Performance	RF+IF+stacking; F1 = 0.891	Table 5.6	Benchmark met; E2E Phase II–IV

5.2.1 Requirements Traceability Matrix

Table 5.3: Requirements traceability.

Req.	Use case / diagram	Module	Test / evidence	MVT
FR-01	Figure 4.41	`WorldBankReserve.sol`	`hierarchy.test.ts`	#1
FR-02	Figures 4.23, 4.28	`LocalBank.sol`, loans API	`LocalBank.test.ts`	#2–3
FR-04	Table 4.12	`BORROWING_LIMIT` + SBT	Foundry invariant (planned)	#5
FR-05	Figure 4.45	`ml-service/`	Table 5.6	#4, #8
FR-07	Figures 4.26–4.27	Event listener	PostgreSQL tests	DT-I.12–13
FR-08	Figure 4.31	MCP agent	Section 4.2.16	#11–12
NFR-02	Table 4.20	OpenZeppelin RBAC	`WorldBankReserve.test.ts`	#13–14

5.3 Final Design Adjustments

This section provides the final edits in the overall project. The project is in development phase; more changes may be required as the phases progress.
Major changes involve: Multi-chain feature was rejected due to potential security risks in bridge attacks. Single-chain design is chosen for deployment on Sepolia/Amoy. Complete details about design decisions can be found in Tables 4.22, 4.23 (Chapter 4).

5.4 Statistical Analysis

This section documents how the ML layer is implemented, what models have been utilized, what datasets were chosen, and preprocessing and evaluations of training and testing. These data are supposed to be complete and fully functional so the backend should be able to feed data from off-chain to on-chain via Chainlink Functions by the end of Phase III.

5.4.1 Datasets and Benchmark Results

Primary dataset: A credible and trusted dataset is crucial for the project; the most suitable dataset available is the BCCC-DeFiFraudTrans-2025 dataset from York University. It contains a combination of DeFi transaction information with 1.03 million rows. For the current run we used a smaller sample of 50,000 transactions where about 8.7% are labeled as fraud transactions. Potential leak columns were removed before the testing. Table 5.4 and Table 5.5 list the full dataset properties and model input features.

Table 5.4: BCCC dataset summary.

Property	Value
Dataset name	BCCC-DeFiFraudTrans-2025
Provider	York University BCCC
Source files	`DeFiTransLyzer_fraud.csv` + `DeFiTransLyzer_-legitimate.csv` (merged)
Full corpus size	1,026,867 rows; 172 columns (164 numeric)
File size	≈1,181 MB
Label column	`fraud` (0 = legitimate, 1 = fraud)
Training subsample	50,000 transactions (`seed=42`)
Fraud rate (sample)	8.65%
Split method	Grouped hold-out by wallet `address`
Train / validation / test	37,715 / 7,145 / 5,140 rows
Model input features	21 behavioral transaction features (Table 5.5)
Removed leakage columns	`flag`, `fraud`, and other label or split columns
Scaling	StandardScaler fit on training fold only
Secondary dataset	Elliptic [68] (reference only); CWB on-chain logs planned for retrain

Table 5.5: BCCC input features.

CWB alias	BCCC column	Category	Description	Used
tx_count_30d	num_transaction	Volume	Number of transactions in the sample window	Yes
duration_seconds	duration_seconds	Volume	Total transaction duration (seconds)	Yes
error_rate	error_rate	Errors	Share of failed or error events	Yes
gas_used_avg	gas_used.gas_used_average	Gas	Average gas used per transaction	Yes
gas_used_std	gas_used.gas_used_standard_deviation	Gas	Variability of gas used	Yes
gas_price_avg	gas_prices.gas_prices_average	Gas	Average gas price paid	Yes
value_avg	values.values_average	Value	Average transferred value	Yes
value_std	values.values_standard_deviation	Value	Variability of transferred value	Yes
unique_from_addrs	number_of_unique_from_address	Addresses	Count of unique sender addresses	Yes
unique_to_addrs	number_of_unique_to_address	Addresses	Count of unique receiver addresses	Yes
from_addr_count	number_of_from_address	Addresses	Total sender address count	Yes
to_addr_count	number_of_to_address	Addresses	Total receiver address count	Yes
token_transfer_amt	token_transfer_amount	Transfer	Raw token transfer amount (text column; excluded)	No
normalized_transfer	normalized_token_transfer	Transfer	Normalised token transfer amount	Yes
gas_efficiency	gas_efficiency	Gas	Gas efficiency ratio	Yes
event_activity	event_activity_flag	Activity	Event activity indicator	Yes
log_count	log_count	Activity	Number of on-chain event logs	Yes
num_errors	number_of_errors	Errors	Count of error events	Yes
nonce_avg	nonce.nonce_average	Addresses	Average wallet nonce (activity signal)	Yes
chain_id	chain_id	Network	Blockchain network identifier	Yes
erc20_log_qty	erc_20_Log_Normalized_Quantity	Transfer	Normalised ERC-20 log quantity	Yes
erc20_qty_int	erc_20_Quantity_Is_Int	Transfer	ERC-20 quantity integer flag	Yes

Preprocessing and split. For each wallet, 22 features are provided; 21 features are

selected that include gas usage, transfer counts, and wallet addresses. These large-number values are scaled and data is split by unique wallet addresses for training and testing sets. Data split counts: train / validation / test = 37,715 / 7,145 / 5,140. Figure 5.1 shows the preprocessing steps:

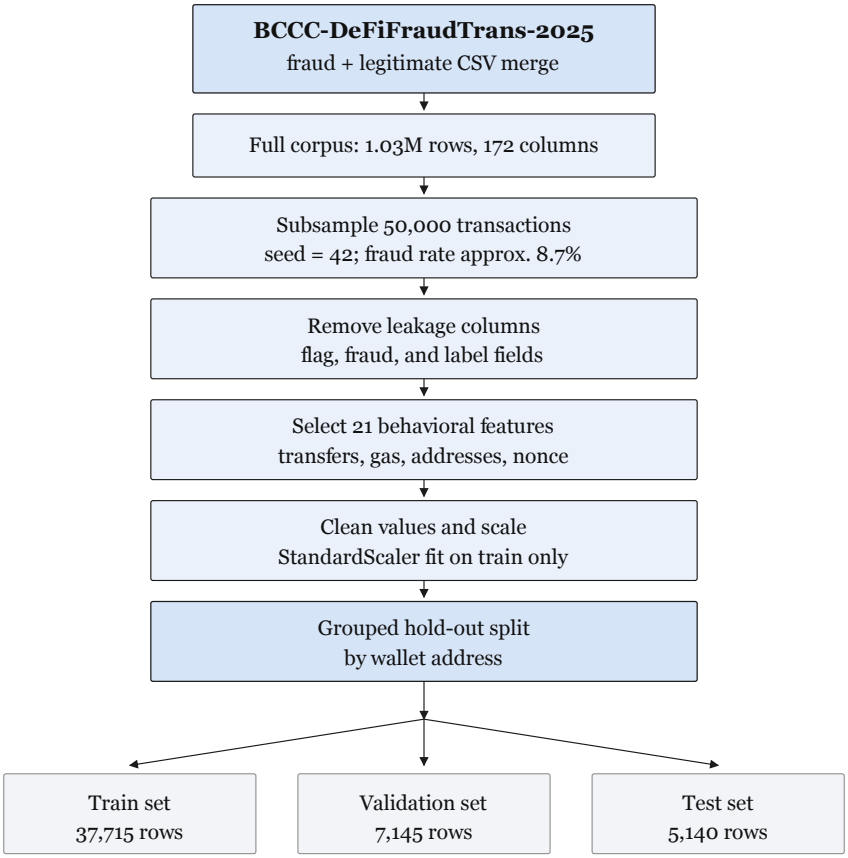

Figure 5.1: BCCC preprocessing split.

Models: A random forest model (80 trees) was chosen for the calculation of fraud probability and an Isolation Forest model (80 trees) for anomaly detection. The combination of these two models provides better fraud detection and decision-making scores for loan approvers, and helps detect misuse of the platform. A simplified

logistic regression method combines these two model results to provide a score that is used as a measurement metric to show the probability of any client being fraudulent and trying to interact with the system. Results and evaluations are shown in Figure 5.2, Figure 5.3, and Table 5.6.

Table 5.6: BCCC benchmark results.

Split	Precision	Recall	F1	ROC-AUC	n_{test}
Held-out test	0.874	0.908	0.891	0.991	5,140

TN = 4,621, FP = 60, FN = 42, TP = 417; fraud rate ≈8.7%; 21 features; leakage column `flag` removed.

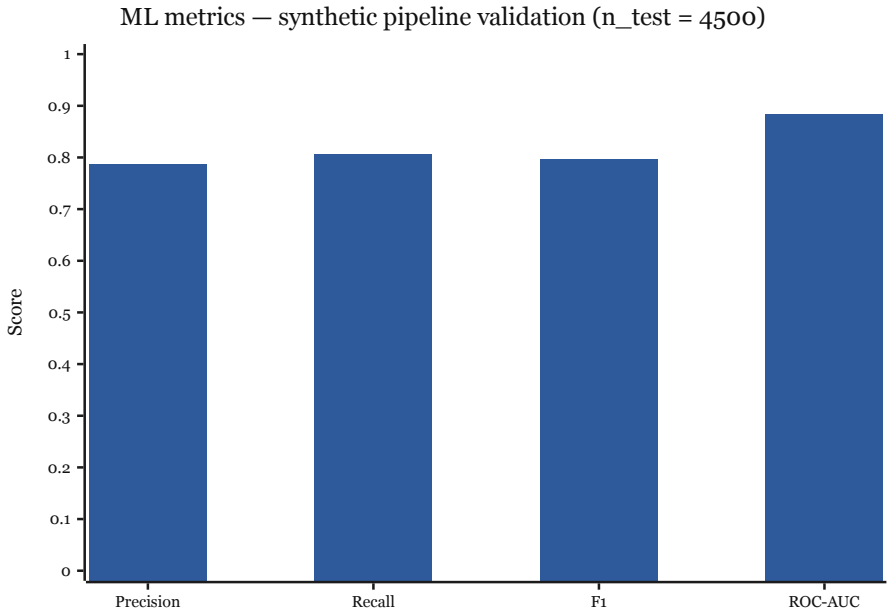

Figure 5.2: ML evaluation metrics.

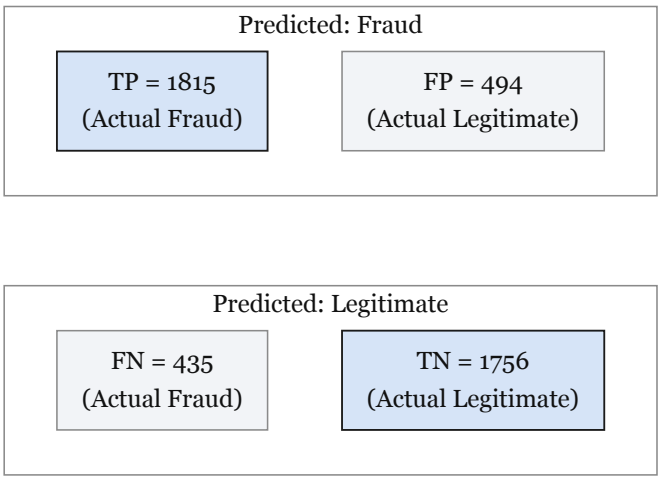

Figure 5.3: Confusion matrix.

Inference service. After completion of model training and code implementations, the platform should be able to monitor and evaluate new applications in real time. Figure 5.4 shows the inference flow from loan request to on-chain commit.

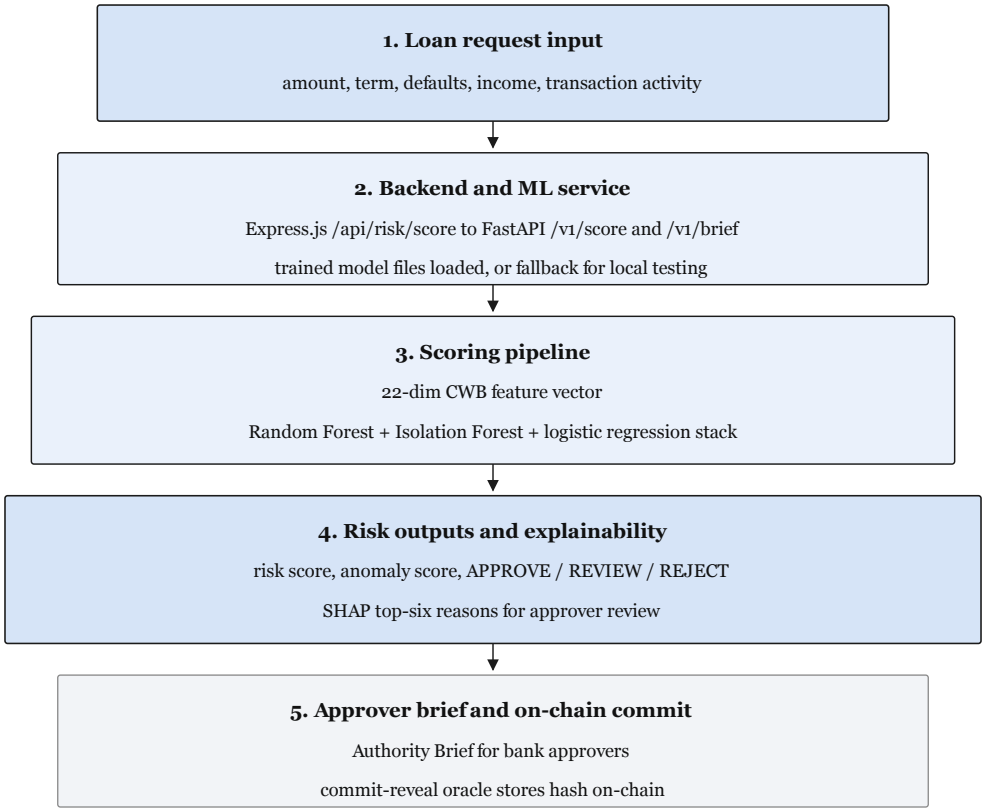

Figure 5.4: ML inference flow.

Decision bands and explainability. The inference service provides risk score, anomaly score, and decision suggestion. Based on the combined score, the following decisions are suggested, shown in Table 5.7:

Table 5.7: ML decision bands.

Combined risk score (s)	Suggested decision	Meaning
$s < 0.4$	Approve	Low risk; loan may be considered for approval
$0.4 \leq s < 0.7$	Manual review	Medium risk; approver must review before decision
$s \geq 0.7$	Reject	High risk; application should be rejected

5.5 Comparisons and Relationships

Table 5.8 demonstrates the choices that were available after researching existing platforms and papers. After extensive planning and analysis, the following options were selected; the rejected options have potential, but the preferably better options were chosen for stability and to make sure all features are well implemented.

Table 5.8: Design comparisons.

Dimension	CWB (selected)	Rejected / baseline	Metric / requirement
Architecture	Four-tier hierarchy	Flat Aave/Compound	FR-01/FR-03; Table 2.3
Settlement	Single-chain Sepolia/Amoy	Multi-chain bridge	Table 3.13; Section 5.3
ML delivery	Off-chain RF+IF+SHAP	On-chain-only scoring	F1 = 0.891; NFR-01
Oracle	Commit–reveal relay	Chainlink Functions DON	Human approver gate (FR-02)
Gas	L2 Cardona (design)	Ethereum mainnet	≈\$0.011 vs \$2.40–\$4.80
Human vs ML	Decision support	Auto-disburse	NFR-02; Section 3.4

5.6 Return on Investment Analysis

ROI for the project is being calculated with the estimated cost of production and maintenance. The full project is planned with zero-cost production in mind, as it is an academic project. The calculation below is shown as probable data, and it may be improved as development phases progress; the complete version of the project can produce results of some sort. For the estimation results, NPV and ROI are calculated for a three-year time scope at a discount rate of $r = 10\%$. The details are shown in Table 5.9.

$$\mathrm{NPV} = \sum_{t=0}^{3} \frac{B_t - C_t}{(1+r)^t}.$$

$$\mathrm{ROI} = \frac{\sum_{t=0}^{3} B_t - \sum_{t=0}^{3} C_t}{\sum_{t=0}^{3} C_t} \times 100\%.$$

Table 5.9: ROI and NPV calculation.

Year	Benefits ($k)	Costs ($k)	Net ($k)	PV factor	PV net ($k)
0 (setup)	0.0	6.0	−6.0	1.000	−6.0
1	48.0	6.0	42.0	0.909	38.2
2	72.0	8.0	64.0	0.826	52.9
3	96.0	10.0	86.0	0.751	64.6
Totals	**216.0**	**30.0**	**186.0**	—	**149.7**

5.7 Discussions

The purpose of this chapter is to evaluate the selected design and the contextual logic behind choosing the selected options for development of the project, and to show the results of the benchmarks of the trained and tested models, gas cost optimization, combined scoring, and decision-making process by the ML layer. However, current results are estimations based on the design of the project. More practical information can be included during the development phases of the project, and ROI calculation can be improved further with more data and influential factors taken into account. A few limitations can be observed: the model has to be made adaptive with the tier-based system of the project; live testnet results are to be added; commit–reveal oracle results and the total latency per transaction are to be calculated with the complete implementation of all features.

Table 5.10: Research limitations.

ID	Limitation	Mitigation / scope
G1	Partial DeFi precedents (Maple, Goldfinch)	Claim: combined four-tier + ML + group lending
G2	BCCC flat-pool training ≠ hierarchical CWB flows	Baseline F1 only; CWB retrain after testnet traffic
G3	Pre-thesis evidence = spec + benchmarks	MVT testnet E2E and Foundry suite (Phase IV)
G4	On-chain group lending ≠ social solidarity	No BRAC/Grameen repayment claims
G5	Oracle collusion; model staleness	Commit–reveal MVP; Certora proofs in Future Work
G6	Technical default only	No sovereign preferred-creditor enforcement

Chapter 6

Conclusion

6.1 Summary of Findings

This paper is a complete specification of all the core sections that will be used in development of the project. It introduces a problem–solution view of how decentralized financing can be made adaptable to the institutional layers of our traditional financing system that is not flat, rather layered. Flat DeFi is hard to be made adaptable among the general mass, so our system provides a tier-based architecture for financial flow, with the added benefit of transparency with programmable smart contracts for the betterment of the client, and less control for the capitalistic organizations. The paper shows credible studies from the past that have contributed to the field before, and uses that evidence and those architectures as proof that a tier-based DeFi system can be built with better security layers that can contribute to the development finance sector globally. The core architecture stack follows mostly a traditional DeFi financial platform build, but the AI/ML layer integration and the tier-based layers are completely novel; no other paper or project shows all four-layer integration before. This project idea is enhanced by the existing systems that have two-tier-based banking systems.

A detailed plan has been introduced for how new clients can be onboarded and attracted to the platform, how clients will interact with the system, how banking authorities can approve loans, and how ML and AI layers can communicate in the middle with both clients and authorities to make this whole process of lending decentralized cryptocurrencies easier and more secure. The potential of such a decentralized system to be popular and adaptable is huge if implemented correctly and if the higher authorities show interest and invest heavily in this idea for the good of humanity. The purpose of this project is to push the idea of decentralized currencies forward for future generations to show more interest, and to support the building blocks for better financing systems in the future.

6.2 Recommendations for Future Work

This section has been specified in the above chapters; it focuses on the potential of the project and the technical abilities and functional capabilities that can be improved and enhanced in the future. First priority is completing the MVT checklist of the development cycle for the defense of the project, and the rest are mostly added stretch to show how many features can be incorporated in the given time. A small summary of the future work:

Deployment on Polygon zkEVM Cardona, which remains experimental for production grade currently; as zkEVM may prove to be more stable, the platform will be deployed on Polygon zkEVM as it is more cost efficient for transactional activities. Cross-tier features such as `InterBankLendingPool` and group lending, upward deposit facilities will be improved and made functional. For security and monitoring, multisig admin for World Bank operations and a live reserve transparency dashboard will be integrated. zkKYC (currently unstable globally), Certora proofs, federated learning, and GraphSAGE (quite complex to implement now) have been specified in the paper and will be more attainable with more time and effort; these have been moved to future work.

This field requires much research, as financial flow is heavily manipulated and misused globally by capitalists; transparency in capital flow is vital for equal access to resources and opportunity to live in harmony. The long-term goal of this project is to evolve this developed system to a more robust level, with more proper and efficient architecture and system design, a regulation-aware system while keeping transparency, equal access, and beneficial to humanity as core design principles.

[1] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt. 2022. SoK: Decentralized Finance (DeFi). *arXiv preprint arXiv:2101.08778v6*. [Online]. Available: https://arxiv.org/abs/2101.08778. Accessed: Aug. 1, 2026.

[2] M. Bastankhah, V. Nadkarni, C. Jin, S. Kulkarni, and P. Viswanath. 2024. Thinking Fast and Slow: Data-Driven Adaptive DeFi Borrow-Lending Protocol. In *Proc. 6th Conf. Advances in Financial Technologies (AFT)* (LIPIcs, Vol. 316). Article 27, 23 pages. [Online]. Available: https://doi.org/10.4230/LIPIcs.AFT.2024.27. Accessed: Aug. 1, 2026.

[3] G. Palaiokrassas, S. Scherrers, I. Ofeidis, and L. Tassiulas. 2024. Leveraging Machine Learning for Multichain DeFi Fraud Detection. In *Proc. IEEE Int. Conf. Blockchain and Cryptocurrency (ICBC)*. [Online]. Available: https://doi.org/10.1109/ICBC59979.2024.10634350. Accessed: Aug. 1, 2026.

[4] I. T. Adom, C. O. Julius, S. Akuma, and S. U. Otor. 2025. Comparative Analysis of Explainable AI Frameworks (LIME and SHAP) in Loan Approval Systems. *International Journal of Information Engineering and Electronic Business (IJIEEB)*,

vol. 17, no. 6. [Online]. Available: https://doi.org/10.5815/ijieeb.2025.06.05. Accessed: Aug. 1, 2026.

[5] B. W. Tan. 2023. Central Bank Digital Currency and Financial Inclusion. *IMF Working Paper WP/23/69*. [Online]. Available: https://www.imf.org/en/Publications/WP/Issues/2023/03/27/Central-Bank-Digital-Currency-and-Financial-Inclusion-531273. Accessed: Aug. 1, 2026.

[6] F. T. Liu, K. M. Ting, and Z.-H. Zhou. 2008. Isolation Forest. In *Proc. IEEE Int. Conf. Data Mining (ICDM)*, pages 413–422. [Online]. Available: https://doi.org/10.1109/ICDM.2008.17. Accessed: Aug. 1, 2026.

[7] N. Atzei, M. Bartoletti, and T. Cimoli. 2017. A Survey of Attacks on Ethereum Smart Contracts (SoK). In *Proc. Int. Conf. Principles of Security and Trust (POST)*, pages 164–186. [Online]. Available: https://doi.org/10.1007/978-3-662-54455-6_8. Accessed: Aug. 1, 2026.

[8] DefiLlama. 2024. Lending Protocols — DeFi TVL and Protocol Rankings. [Online]. Available: https://defillama.com/protocols/Lending. Accessed: Aug. 1, 2026.

[9] World Bank. 2021. The Global Findex Database 2021: Financial Inclusion, Digital Payments, and Resilience. [Online]. Available: https://www.worldbank.org/en/publication/globalfindex. Accessed: Aug. 1, 2026.

[13] International Finance Corporation (IFC). 2017. MSME Finance Gap: Assessment of the Shortfalls and Opportunities in Financing Micro, Small, and Medium Enterprises in Emerging Markets. World Bank. [Online]. Available: https://openknowledge.worldbank.org/entities/publication/ff4c9839-21ac-5676-a23a-7cf6f745df0c. Accessed: Aug. 1, 2026.

[15] Committee on Payments and Market Infrastructures and Bank for International Settlements. 2016. Correspondent Banking — Technical Report. CPMI Papers No. 147. [Online]. Available: https://www.bis.org/cpmi/publ/d147.pdf. Accessed: Aug. 1, 2026.

[17] World Bank. 2024. Remittance Prices Worldwide: Making Markets More Transparent. Migration and Development Brief 40. [Online]. Available: https://remittanceprices.worldbank.org/. Accessed: Aug. 1, 2026.

[18] DefiLlama. 2026. Aave V3 TVL, Fees, Revenue & Income Statement. March. [Online]. Available: https://defillama.com/protocol/aave-v3. Accessed: Aug. 1, 2026.

[19] DefiLlama. 2026. Compound V3 TVL, Fees, Revenue & Income Statement. March. [Online]. Available: https://defillama.com/protocol/compound-v3. Accessed: Aug. 1, 2026.

[20] Fensory. 2026. Sky Protocol Projects $611M Revenue in 2026 as USDS Supply Targets $20.6 Billion. February. [Online]. Available: https://www.fensory.com/intelligence/rwa/sky-protocol-tokenization-regatta-solana-february-2026. Accessed: Aug. 1, 2026.

[21] Fensory. 2026. Maple Finance — Institutional DeFi Lending Protocol. [Online]. Available: https://www.fensory.com/insights/protocols/maple-finance. Accessed: Aug. 1, 2026.

[23] Financial Stability Board. 2020. Enhancing Cross-border Payments: Stage 3 Roadmap. [Online]. Available: https://www.fsb.org/2020/10/enhancing-cross-border-payments-stage-3-roadmap Accessed: Aug. 1, 2026.

[25] World Bank. 2025. World Bank Group Tracks Project Funds with New Blockchain Tool. Press Release, September. [Online]. Available: https://www.worldbank.org/en/news/press-release/2025/09/26/world-bank-group-tracks-project-funds-with-new-blockchain-tool. Accessed: Aug. 1, 2026.

[26] JPMorgan. 2025. Kinexys Digital Payments: Real-Time Multicurrency Payments. [Online]. Available: https://www.jpmorgan.com/kinexys/index. Accessed: Aug. 1, 2026.

[31] Y. Chen, S. Bin, and H. He. 2024. Design and Implementation of a Multi-Chain Lending Model in Blockchain. In *Proc. 3rd Int. Conf. Artificial Intelligence and Computer Information Technology (AICIT)*, pages 97–102.

[32] S. Yang and W. Cui. 2023. An Evaluation System for DeFi Lending Protocols. *arXiv preprint arXiv:2303.01022*. [Online]. Available: https://arxiv.org/abs/2303.01022. Accessed: Aug. 1, 2026.

[33] J. Hartmann and O. Hasan. 2021. A Social-Capital Based Approach to Blockchain-Enabled Peer-to-Peer Lending. In *Proc. IEEE Int. Conf. Blockchain Computing and Applications (BCCA)*, pages 105–110. [Online]. Available: https://hal.science/hal-03371872. Accessed: Aug. 1, 2026.

[34] T. H. Pranto, H. T. A. Hasib, M. R. Tahsinur, A. B. Haque, A. K. M. N. Islam, and R. M. Rahman. 2022. Blockchain and Machine Learning for Fraud Detection: A Privacy-Preserving and Adaptive Incentive Based Approach. *IEEE Access*, vol. 10, pages 87115–87131. [Online]. Available: https://ieeexplore.ieee.org/document/9857827. Accessed: Aug. 1, 2026.

[35] Bank for International Settlements et al. 2020. Central Bank Digital Currencies: Foundational Principles and Core Features. BIS and Central Banks Joint Report. [Online]. Available: https://www.bis.org/publ/othp33.htm. Accessed: Aug. 1, 2026.

[36] M. Asaduzzaman, F. Hasib, and Z. B. Hafiz. 2020. Towards Using Blockchain Technology for Microcredit Industry in Bangladesh. In *Proc. 23rd Int. Conf.*

Computer and Information Technology (ICCIT), pages 1–6. [Online]. Available: https://doi.org/10.1109/ICCIT51783.2020.9392730. Accessed: Aug. 1, 2026.

[37] H. Kim and D. Kim. 2024. Optimal Gas Fee Minimization in DeFi: Enhancing Efficiency and Security on the Ethereum Blockchain. *IEEE Access*, vol. 12, pages 173810–173823. [Online]. Available: https://ieeexplore.ieee.org/document/10750187. Accessed: Aug. 1, 2026.

[38] X. Yuan. 2024. SHAP-based Interpretable Models for Credit Default Assessment Using Machine Learning. In *Proc. 14th Int. Conf. Software Technology and Engineering (ICSTE)*, pages 213–217. [Online]. Available: https://doi.org/10.1109/ICSTE63875.2024.00044. Accessed: Aug. 1, 2026.

[39] T. Dao, T. Trinh, and V. Pham. 2025. Optimizing Credit Scoring Models for Decentralized Financial Applications. In *Information and Communication Technology*, Springer. [Online]. Available: https://doi.org/10.1007/978-981-96-4282-3_36. Accessed: Aug. 1, 2026.

[42] L. Gudgeon, S. M. Werner, D. Perez, and W. J. Knottenbelt. 2020. DeFi Protocols for Loanable Funds: Interest Rates, Liquidity and Market Efficiency. In *Proc. 2nd ACM Conf. Advances in Financial Technologies (AFT)*, pages 92–112. [Online]. Available: https://doi.org/10.1145/3419614.3423254. Accessed: Aug. 1, 2026.

[52] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al. 2021. The PRISMA 2020 statement: an updated guideline for reporting systematic reviews. *BMJ*, vol. 372, n71. [Online]. Available: https://doi.org/10.1136/bmj.n71. Accessed: Aug. 1, 2026.

[58] European Parliament and Council of the European Union. 2024. Regulation (EU) 2023/1114 of 31 May 2023 on Markets in Crypto-Assets (MiCA). *Official Journal of the European Union*, L 150/40, June 2023; fully applicable from 30 December. [Online]. Available: https://eur-lex.europa.eu/eli/reg/2023/1114/oj. Accessed: Aug. 1, 2026.

[60] United States Congress. 2025. Guiding and Establishing National Innovation for U.S. Stablecoins (GENIUS) Act. Pub. L. No. 119-27, July 18. [Online]. Available: https://www.govinfo.gov/content/pkg/PLAW-119publ27/pdf/PLAW-119publ27.pdf. Accessed: Aug. 1, 2026.

[61] Bank for International Settlements Innovation Hub. 2024. Project mBridge: Connecting Economies through CBDC — Reaches Minimum Viable Product. BIS Innovation Hub Report, June. [Online]. Available: https://www.bis.org/about/bisih/topics/cbdc/mcbdc_bridge.htm. Accessed: Aug. 1, 2026.

[62] Bank for International Settlements and Central Banks of France, Japan, Korea, Mexico, Switzerland, the UK, and the US. 2024. Project Agora: Tokenization of Cross-Border Payments using Unified Ledgers. BIS Innovation Hub Working Paper, April.

[Online]. Available: https://www.bis.org/about/bisih/topics/fmis/agora.htm. Accessed: Aug. 1, 2026.

[68] Elliptic. 2019. The Elliptic Data Set. Kaggle. [Online]. Available: https://www.kaggle.com/datasets/ellipticco/elliptic-data-set. Accessed: Aug. 1, 2026.

[78] L. Dong, Y. Qiu, and F. Xu. 2023. Blockchain-Enabled Deep-Tier Supply Chain Finance. *Manufacturing & Service Operations Management*, 25, 6, pages 2021–2037. [Online]. Available: https://doi.org/10.1287/msom.2022.1123. Accessed: Aug. 1, 2026.

[79] U. Bindseil. 2020. Tiered CBDC and the Financial System. *ECB Working Paper Series*, no. 2351, January. [Online]. Available: https://www.ecb.europa.eu/pub/pdf/scpwps/ecb.wp2351~c8c18bbd60.en.pdf. Accessed: Aug. 1, 2026.

[80] J. Kiff, J. Alwazir, S. Davidovic, A. Farias, A. Khan, T. Khiaonarong, M. Malaika, H. Monroe, N. Sugimoto, H. Tourpe, and P. Zhou. 2020. A Survey of Research on Retail Central Bank Digital Currency. *IMF Working Paper WP/20/104*, June. [Online]. Available: https://www.imf.org/en/Publications/WP/Issues/2020/06/26/A-Survey-of-Research-on-Retail-Central-Bank-Digital-Currency-49069. Accessed: Aug. 1, 2026.

[90] E. Cerutti, M. Firat, and H. Pérez-Saiz. 2025. Estimating the Impact of Digital Money on Cross-Border Flows: Scenario Analysis Covering the Intensive Margin. *IMF Fintech Notes* 2025/002, February. [Online]. Available: https://www.imf.org/-/media/files/publications/ftn063/2025/english/ftnea2025002.pdf. Accessed: Aug. 1, 2026.

[94] E. G. Weyl, P. Ohlhaver, and V. Buterin. 2022. Decentralized Society: Finding Web3's Soul. *SSRN Working Paper No. 4105763.* [Online]. Available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4105763. Accessed: Aug. 1, 2026.

[96] G. Caldarelli. 2025. Can Artificial Intelligence Solve the Blockchain Oracle Problem? Unpacking the Challenges and Possibilities. *Frontiers in Blockchain*, vol. 8, art. 1682623. [Online]. Available: https://doi.org/10.3389/fbloc.2025.1682623. Accessed: Aug. 1, 2026.

[99] Behaviour-Centric Cybersecurity Center (BCCC), York University. 2025. BCCC-DeFiFraudTrans-2025: A Labelled Ethereum DeFi Fraud Transaction Dataset. [Online]. Available: https://www.yorku.ca/research/bccc/ucs-technical/cybersecurity-datasets-cds/defi-fraud-transactions-bccc-defifraudtrans-2025/. Accessed: Aug. 1, 2026.

[101] McKinsey & Company. 2018. Global Payments 2018: A Dynamic Industry Continues to Defy Predictions. McKinsey Global Payments Map. [Online]. Available: https://www.mckinsey.com/industries/financial-services/our-insights/global-payments-a-dynamic-ind
Accessed: Aug. 1, 2026.

[102] Y. Wang, Q. Zhang, K. Li, Y. Tang, J. Chen, X. Luo, and T. Chen. 2021. iBatch: Saving Ethereum Fees via Secure and Cost-Effective Batching of Smart-Contract Invocations. In *Proc. ACM Joint Eur. Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, pages 566–577. [Online]. Available: https://doi.org/10.1145/3468264.3468568. Accessed: Aug. 1, 2026.

[103] World Inequality Lab. 2026. Exorbitant Privilege. *World Inequality Report 2026*. [Online]. Available: https://wir2026.wid.world/insight/exorbitant-privilege/. Accessed: Aug. 1, 2026.

[105] W. Wang, J. Song, G. Xu, Y. Li, H. Wang, and C. Su. 2021. ContractWard: Automated Vulnerability Detection Models for Ethereum Smart Contracts. *IEEE Trans. Network Science and Engineering*, vol. 8, no. 2, pages 1133–1144. [Online]. Available: https://doi.org/10.1109/TNSE.2020.2968505. Accessed: Aug. 1, 2026.

[106] J.-W. Liao, T.-T. Tsai, C.-K. He, and C.-W. Tien. 2019. SoliAudit: Smart Contract Vulnerability Assessment Based on Machine Learning and Fuzz Testing. In *Proc. 6th Int. Conf. Internet of Things: Systems, Management and Security (IOTSMS)*, pages 458–465. [Online]. Available: https://doi.org/10.1109/iotsms48152.2019.8939256. Accessed: Aug. 1, 2026.

[107] Federal Reserve Bank of New York and Bank for International Settlements Innovation Hub. 2025. Project Pine: Central Bank Open Market Operations with Smart Contracts. BIS Innovation Hub / NY Fed Joint Report, May. [Online]. Available: https://www.bis.org/publ/othp95.htm. Accessed: Aug. 1, 2026.

Appendix A

A.1 Agent Harness Reference

A.1.1 Per-User Context and Prompt Assembly

The Express.js context injection layer:

```
{
  "borrower_id": "550e8400-e29b-41d4-a716-446655440000",
  "wallet_address": "0xABC...",
  "language": "en",
  "credit_tier": "Silver",
  "credit_score": 512,
  "active_loans": [
    {
      "loan_id": "L-4821",
      "amount": 100,
      "next_due": "2026-06-15",
      "installment": 17.5
    }
  ],
  "savings_balance": 250.0,
  "pool_utilisation": 0.67,
  "kyc_tier": 1,
  "conversation_history": [ "...last 10 turns..." ]
}
```

A.1.2 Session Lineage, Compression, and Toolset Scoping

Table A.1: MCP toolset scoping.

Toolset	Tools visible
`read_only`	Full-suite taxonomy (Future Work): all 9 read tools; the MVT deploys only the subset needed for the selected 3–5 tools.
`loan_actions`	`read_only` + loan, installment, and group write tools.
`account_-management`	`read_only` + deposit, KYC, and reminder write tools.

A.2 Smart Contract Capabilities

Table A.2: Smart contract capabilities.

Contract	Module	Ph.	Status	Primary capability
WorldBankReserve	Tier 1	I	Specified	Reserve custody, national-bank registration, downward allocation
NationalBank	Tier 2	I	Specified	Local-bank registration, borrow upward, allocate downward
LocalBank	Tier 3	I	Specified	Client onboarding, approvers, `LoanController`, `freezeAccount`
LoanController	Lending	I–II	Shell / Ph. II	Request, approve, disburse, installment state machine
SavingsVault	Deposits	II	Specified	Variable-yield savings; reserve-gated withdrawals
FixedDeposit	Deposits	II+	Stub	Term deposits; early-withdrawal penalty
GroupLendingPool	Group	II	Specified	Group consent, shared liability, per-member disbursement
InsuranceFund	Risk	I–II	Should	5% interest capture; default coverage; PoR attestation
CurrentAccount	Payments	II+	Stub	Atomic transfers; recurring payments
InterBankLending	Multi-entity	II	Specified	Same-tier peer pools (1 / 7 / 30 day)
UpwardDepositFacility	Multi-entity	II	Specified	Surplus repatriation to parent tier
SyndicatedLoan	Multi-entity	III	Planned	Multi-bank co-funding and consent vote
TranchedPool	Multi-entity	III	Planned	Senior / junior tranches; waterfall recovery
TreasurySwap	Multi-entity	III	Planned	Institutional oracle-priced asset swap
FXModule	Treasury / FX	III	Planned	Retail FX conversion and audit trail
LiquidationEngine	Risk	III	Planned	Health-factor monitor; liquidator bonus
NettingEngine	Multi-entity	—	Future Work	Multilateral settlement netting (stub only)

A.3 WorldBankReserve Contract Interface

```
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/security/ReentrancyGuard.sol";
import "@openzeppelin/contracts/access/AccessControl.sol";

contract WorldBankReserve is ReentrancyGuard, AccessControl {
```

```
7
8       bytes32 public constant NATIONAL_BANK_ROLE = keccak256("NATIONAL_BANK_ROLE")
            ;
9       bytes32 public constant AUDITOR_ROLE       = keccak256("AUDITOR_ROLE");
10
11      uint256 public totalReserve;
12      uint256 public minimumReserveRatio;
13      mapping(address => uint256) public allocatedTo;
14
15      event CapitalAllocated(address indexed nationalBank, uint256 amount);
16      event RepaymentReceived(address indexed nationalBank, uint256 amount);
17      event ReserveRatioUpdated(uint256 newRatio);
18
19      function allocateCapital(address nationalBank, uint256 amount)
20          external onlyRole(DEFAULT_ADMIN_ROLE) nonReentrant;
21
22      function recordRepayment(uint256 amount)
23          external onlyRole(NATIONAL_BANK_ROLE) nonReentrant;
24
25      function utilizationRate() external view returns (uint256);
26
27      function isReserveAdequate() external view returns (bool);
28
29      function getReserveSummary() external view returns (
30          uint256 totalDeposited,
31          uint256 totalLoaned,
32          uint256 reserveRatio,
33          uint256 insuranceFundBalance
34      );
35  }
```

Listing A.1: WorldBankReserve.sol – Tier 1 public interface. Full implementation in project repository.